UNIVERSIDAD DE CHILE
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
DEPARTAMENTO DE INGENIERÍA ELÉCTRICA

PLANIFICACIÓN MULTINIVEL PARA NAVEGACIÓN ROBÓTICA EN AMBIENTES DE EXTERIOR

TESIS PARA OPTAR AL GRADO DE
MAGÍSTER EN CIENCIAS DE LA INGENIERÍA, MENCIÓN ELÉCTRICA

MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL ELÉCTRICO

GONZALO ALFONSO OLGUÍN MONCADA

PROFESOR GUÍA:
JAVIER RUIZ DEL SOLAR SAN MARTÍN

PROFESOR CO-GUÍA:
FRANCISCO LEIVA CASTRO

MIEMBROS DE LA COMISIÓN:
MARTIN ADAMS
RODRIGO VERSCHAE TANNENBAUM

Este trabajo ha sido parcialmente financiado por el Proyecto FONDECYT 1251823

SANTIAGO DE CHILE
2026

RESUMEN DE LA TESIS PARA OPTAR AL
GRADO DE MAGÍSTER EN CIENCIAS
DE LA INGENIERÍA, MENCIÓN ELÉCTRICA
Y MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL ELÉCTRICO
POR: GONZALO ALFONSO OLGUÍN MONCADA
FECHA: 2026
PROF. GUÍA: JAVIER RUIZ DEL SOLAR
SAN MARTÍN

PLANIFICACIÓN MULTINIVEL PARA NAVEGACIÓN ROBÓTICA EN AMBIENTES DE EXTERIOR

La navegación autónoma en entornos exteriores no estructurados presenta desafíos de incertidumbre y variabilidad que limitan la eficacia de los enfoques clásicos basados en mapas métricos. En esta tesis se aborda el problema de navegación global sin mapa (*mapless navigation*) mediante una arquitectura híbrida que integra modelos generativos probabilísticos y modelos de visión-lenguaje (VLM).

A diferencia de los planificadores deterministas tradicionales, se propone un generador de trayectorias basado en Autoencoders Condicionales Variacionales (CVAE) con *prior* aprendido, capaz de proponer múltiples hipótesis de movimiento condicionadas por el entorno de navegación. Estas hipótesis son posteriormente jerarquizadas por un módulo de percepción de vocabulario abierto, el cual interpreta preferencias simples en lenguaje natural, para discriminar las rutas semánticamente preferibles.

Mediante evaluaciones experimentales en una plataforma robótica real tipo *skid-steer*, se demuestra que este enfoque supera la rigidez de los métodos puramente geométricos. Los resultados validan que la estrategia propuesta otorga al agente la capacidad de adaptarse a contextos dinámicos y seguir instrucciones simples, manteniendo la seguridad física y la diversidad de exploración en ausencia de mapas globales.

A mi mamá y mi papá.

Agradecimientos

Quiero agradecer en primer lugar a mi profesor guía, Javier Ruiz del Solar, por su constante apoyo, orientación y paciencia durante todo el desarrollo de esta tesis. Sus valiosos consejos y conocimientos han sido fundamentales para la realización de este trabajo.

También quiero expresar mi gratitud a mis compañeros y amigos del Laboratorio de Robótica, sobre todo a Ignacio y Mattías que fueron quienes más tiempo me acompañaron y con quienes discutí varios aspectos de la tesis. También a la gente de AMTC, en especial a Michelle por ayudarme con el Panther constantemente.

Finalmente, agradezco a mi familia. A mi mamá y papá por brindarme apoyo desde siempre y darme las facilidades para estudiar esta carrera. A mis hermanos, Alejandra y Rodrigo, por soportarme durante todos los años de universidad. Agradezco también a Consuelo, que me acompañó casi desde el inicio del proceso universitario.

Tabla de Contenido

1. Introducción **1**

1.1. Motivación 1

1.2. Hipótesis 2

1.3. Objetivo general 3

1.4. Objetivos específicos 3

1.5. Principales Contribuciones 3

1.6. Estructura del documento 4

2. Marco teórico **5**

2.1. Navegación autónoma en robótica 5

2.1.1. Navegación Global: El nivel deliberativo 5

2.1.2. Navegación Local: El nivel reactivo 6

2.1.3. Arquitectura Híbrida 7

2.2. Sistemas de Localización y Estimación de Pose 7

2.2.1. Odometría: Estimación Relativa 7

2.2.2. Sistemas de Localización Global 8

2.3. Mapas de Elevación y Representación 2.5D 8

2.3.1. Enfoque *Robot-Centric* y Fusión Sensorial 9

2.4. Autoencoders Variacionales (VAE) 9

2.4.1. Objetivo de entrenamiento 10

2.4.2. Inferencia 11

2.5. Autoencoders Variacionales Condicionales (CVAE) 11

2.5.1. Formulación 1: *Vanilla* CVAE 12
2.5.2. Formulación 2: CVAE con *Prior* Aprendido (*Learned Prior*) 13
2.6. MTG: *Mapless Trajectory Generation* 15
2.6.1. Diversidad Estructural mediante DLOW 15
2.6.2. Cobertura y Transitabilidad 16
2.7. Procesamiento de Nubes de Puntos: PointNet 17
2.8. Modelos Fundacionales / Modelos de Visión Lenguaje 18
2.8.1. CLIP: *Contrastive Language-Image Pre-training* 19
2.8.2. CLIPSeg: Segmentación basada en Prompts 19

3. Estado del Arte **21**
3.1. Estimación de Transitabilidad y Percepción Semántica 21
3.1.1. Enfoques Geométricos y Aprendizaje Autosupervisado 21
3.1.2. Segmentación Semántica y Fusión de Mapas 22
3.2. De la Navegación Reactiva a la Generativa 22
3.2.1. Aprendizaje Reforzado y Aprendizaje por Imitación 22
3.2.2. Modelos Generativos de Trayectorias 23
3.3. Modelos Fundacionales de Visión-Lenguaje (VLMs) 23
3.3.1. Representaciones Visuales Generalizables 24
3.3.2. VLMs como Razonadores de Navegación 24

4. Navegación Global: Generación y Selección de Trayectorias **25**
4.1. Descripción general de la solución 25
4.2. Generación de trayectorias con CVAE 26
4.2.1. Formulación del Problema 26
4.2.2. Arquitectura propuesta (NaVAE) 27
4.2.3. Pre-entrenamiento del Módulo de Percepción (PointNet) 28
4.2.4. Encoder *prior* $p_\theta(z|\mathbf{x})$ 31
4.2.5. Encoder *posterior* $q_\phi(z|\mathbf{x}, \mathbf{y})$ 33

4.2.6. Decoder $p_\psi(\mathbf{y}|z,\mathbf{x})$ . . . 34
4.2.7. Función de pérdida . . . 35
4.3. Selección de trayectorias . . . 36
4.3.1. Filtrado de colisiones . . . 37
4.3.2. Construcción de mapa de costos con VLM . . . 38
4.3.3. Cálculo de costos por trayectoria . . . 39
4.3.4. Actualización de costos y trayectorias . . . 40
4.3.5. *Recovery behavior* . . . 42
4.4. Modificación *baseline* MTG . . . 44
4.4.1. Arquitectura . . . 44
4.4.2. Función de Pérdida y Entrenamiento . . . 44

5. Resultados **46**
5.1. Base de datos . . . 46
5.2. Pre-entrenamiento de PointNet . . . 48
5.3. Entrenamiento NaVAE . . . 50
5.4. Evaluación en el mundo real . . . 51
5.4.1. *Setup* experimental . . . 51
5.4.2. Pruebas y métricas . . . 52
5.4.3. Evaluación del sistema de navegación global . . . 55
5.4.4. Variación parámetros de selección . . . 61
5.4.5. Variación del planificador local . . . 62

6. Análisis y discusión de resultados **64**

7. Conclusiones **68**
7.1. Trabajo Futuro . . . 69

Bibliografía **77**

Índice de Tablas

5.1. Desglose de la información contenida en cada muestra temporal de la base de datos (ventana de 0,5 s). 48

5.2. Resultados del mejor modelo de PointNet en evaluación. 49

5.3. Parámetros usados para el entrenamiento del modelo generador NaVAE. . . 50

5.4. Orden de las pruebas de navegación realizadas y su largo aproximado por una demostración experta. 53

5.5. Parámetros generales usados para el módulo de selección de trayectorias. . . 55

5.6. Valores de las clases usadas para la construcción de mapas de costos. 55

5.7. Resultados cuantitativos de navegación global. 56

Índice de Ilustraciones

2.1. Diagrama para ilustrar la diferencia entre la planificación local vs. la planificación global. 6

2.2. Arquitectura general de un VAE. 10

2.3. Arquitectura de un CVAE *Vanilla*. 13

2.4. Arquitectura de un CVAE con *prior* aprendido. 14

2.5. Arquitectura general de MTG. 15

4.1. Diagrama general de la solución al problema de navegación global. 26

4.2. Arquitectura del modelo generativo propuesto, basado en CVAE con *prior* aprendido. 29

4.3. Proceso de generación de *Ground Truth* para la nube de puntos. 30

4.4. Arquitectura detallada del encoder *prior* $p_\theta(z|\mathbf{x})$. 31

4.5. Arquitectura detallada del encoder *posterior* $q_\phi(z|\mathbf{x}, \mathbf{y})$. 33

4.6. Arquitectura detallada del decoder $p_\psi(\mathbf{y}|z, \mathbf{x})$. 34

4.7. Diagrama general del sistema de selección de trayectorias. 37

4.8. Diagrama x-z del robot y la proyección de los *waypoints* al mapa de elevación. 38

4.9. Ilustración de un robot en un sistema $\mathcal{O}_{odom}$, que proyecta una trayectoria en el sistema $\mathcal{O}_{robot}$ al plano de la imagen $\mathbf{I}'$. 39

4.10. Referencia de una trayectoria seleccionada en un mapa de costos. 41

4.11. Diagrama de búsqueda de direcciones libres de colisiones. 42

4.12. Diagrama de la arquitectura MTG modificada (MTG') para la tarea de navegación. 45

5.1. Mapas semánticos de la base de datos generada. 47

5.2. Curvas de entrenamiento de *Loss* y *Accuracy* del modelo PointNet. 49

5.3. Ejemplos de segmentación de PointNet y mapas de calor $\mathbf{I}_{seg}$ generados en los datos de test. 49

5.4. Curvas de entrenamiento y evaluación del modelo generador NaVAE. 50

5.5. Ejemplos de trayectorias generadas por el modelo NaVAE en los datos de evaluación. 51

5.6. *Setup* experimental del robot Husarion Panther usado para las pruebas de navegación. 52

5.7. Vista satelital del sector usado para evaluar el módulo de navegación global. 52

5.8. Comparativa de trayectorias exitosas ejecutadas en las pruebas en el sector A. 57

5.9. Comparativa de trayectorias exitosas ejecutadas en las pruebas en el sector B. 58

5.10. Comparación entre las trayectorias ejecutadas y las generadas en el mapa de elevación y proyectadas a la imagen FPV. 59

5.11. Comparativa de trayectorias seleccionadas con trayectorias ejecutadas. 60

5.12. Ejemplos de trayectorias ejecutadas variando el costo de la clase *grass*. . . . 61

5.13. Ejemplos de mapas de costos y trayectorias al agregar la clase *sand* a la segmentación. 62

5.14. Ejemplos de trayectorias ejecutadas usando dos planificadores locales distintos. 63

Listado de Acrónimos

- AMCL: *Adaptive Monte Carlo Localization* (Localización Adaptativa por Monte Carlo)
- CLIP: *Contrastive Language-Image Pre-Training* (Preentrenamiento Contrastivo de Lenguaje e Imagen)
- CVAE: *Conditional Variational Autoencoder* (Autoencoder Variacional Condicional)
- EKF: *Extended Kalman Filter* (Filtro de Kalman Extendido)
- ELBO: *Evidence Lower Bound* (Cota Inferior de la Evidencia)
- FOV: *Field of View* (Campo de Visión)
- FPV: *First Person View* (Vista en Primera Persona)
- GNSS: *Global Navigation Satellite System* (Sistema Global de Navegación por Satélite)
- GPS: *Global Positioning System* (Sistema de Posicionamiento Global)
- IL: *Imitation Learning* (Aprendizaje por Imitación)
- IMU: *Inertial Measurement Unit* (Unidad de Medición Inercial)
- KL: *Kullback-Leibler* (Kullback-Leibler)
- LiDAR: *Light Detection and Ranging* (Detección y Medición por Luz)
- LLM: *Large Language Model* (Modelo de Lenguaje Grande)
- MTG: *Mapless Trajectory Generation* (Generación de Trayectorias sin Mapa)
- RGB: *Red Green Blue* (Rojo Verde Azul, referente a los canales de color en imágenes)
- RL: *Reinforcement Learning* (Aprendizaje Reforzado)
- ROS: *Robot Operating System* (Sistema Operativo Robótico¡)
- SLAM: *Simultaneous Localization and Mapping* (Localización y Mapeo Simultáneos)
- VAE: *Variational Autoencoder* (Autoencoder Variacional)
- VFM: *Vision Foundation Model* (Modelo de Fundación de Visión)
- VLM: *Vision-Language Model* (Modelo de Visión-Lenguaje)

Capítulo 1

Introducción

1.1. Motivación

La navegación autónoma en robótica móvil es uno de los desafíos más antiguos y complejos del área, lo que ha llevado al desarrollo de diversas soluciones a lo largo del tiempo. Los métodos tradicionales, aunque efectivos en entornos estructurados, dependen en gran medida de mapas predefinidos o modelos cinemáticos que no se adaptan bien a cambios repentinos o a la aparición de obstáculos imprevistos.

Actualmente, los métodos basados en aprendizaje han ganado mayor popularidad, ya que permiten capturar comportamientos complejos y adaptarse a situaciones diversas, logrando un rendimiento superior en entornos dinámicos o no estructurados [1]. Estos enfoques incluyen el uso de aprendizaje profundo para navegar en ambientes con obstáculos cambiantes, adaptarse a terrenos irregulares, o condiciones de iluminación variables, además del procesamiento de sensores visuales y de rango mediante redes neuronales para mejorar la percepción del entorno [2, 3, 4].

Sin embargo, la mayoría de los enfoques de aprendizaje supervisado directo enfrentan una limitación crítica: la unimodalidad. En escenarios de navegación real, a menudo existen múltiples trayectorias válidas para sortear un obstáculo (por ejemplo, rodear un árbol por la izquierda o por la derecha). Los modelos deterministas tradicionales tienden a promediar estas soluciones, resultando en comportamientos inseguros o inviables [5]. Aquí es donde los modelos generativos presentan un cambio de paradigma. Al aprender la distribución de probabilidad de las trayectorias posibles en lugar de una única salida determinista, estos modelos pueden proponer múltiples hipótesis de movimiento diversas y factibles, capturando la naturaleza estocástica de la navegación en el mundo real [6, 7].

A pesar de la capacidad de los modelos generativos para proponer rutas geométricamente viables, estos carecen a menudo de un entendimiento semántico profundo del entorno. Un robot en un parque podría generar una ruta válida a través del pasto y otra a través de un sendero pavimentado; ambas son geométricamente posibles, pero la preferencia entre una y otra depende de normas sociales o restricciones contextuales difíciles de codificar en una función de costo tradicional. Recientemente, los Modelos de Visión-Lenguaje (VLMs) han

emergido como una herramienta potente para dotar a los robots de razonamiento de sentido común [8, 9]. Estos modelos permiten interpretar escenas complejas y seguir instrucciones en lenguaje natural, actuando como un puente entre la percepción visual y la toma de decisiones lógica.

La integración de estas dos tecnologías representa una oportunidad para la navegación global sin mapa. Mientras que los modelos generativos pueden manejar la planificación y la viabilidad física proponiendo un abanico de acciones seguras, los VLMs pueden actuar como un traductor entre las preferencias del usuario y la selección de acciones dentro del abanico.

En vista de lo anterior, esta tesis propone desarrollar un módulo de navegación global que combine la capacidad exploratoria de los modelos generativos (específicamente CVAEs [10]) con las capacidades semánticas de los VLMs. A diferencia de los enfoques que dependen de mapas métricos globales costosos de mantener (ya sea por el tamaño de los mapas o el procesamiento sensorial para armarlos), esta propuesta busca un módulo capaz de planificar rutas de largo horizonte de forma adaptativa, utilizando datos recolectados en el mundo real para aprender patrones de movimiento y el lenguaje natural para dar guías simples de navegación en forma de segmentaciones de vocabulario abierto. Este módulo de navegación actúa como un nivel superior en una arquitectura de dos niveles, donde el nivel inferior corresponde a un planificador local del estado del arte (intercambiable), encargado de la ejecución comandos de velocidad.

El alcance de este trabajo se centra en validar el sistema multinivel en entornos urbanos del mundo real, utilizando una plataforma robótica *skid-steer* equipada con la sensorización adecuada. Si bien la implementación en entornos distintos o con plataformas de cinemática diversa podría requerir el ajuste de parámetros específicos, estos cambios son de carácter instrumental y no estructural. La metodología propuesta, basada en la interacción entre modelos generativos y de lenguaje, es agnóstica al *hardware*, lo que garantiza que los fundamentos algorítmicos presentados sean transferibles y generalizables a otros contextos robóticos sin alterar la arquitectura central del sistema.

1.2. Hipótesis

H1. El uso de modelos generativos condicionales con *prior* aprendido permite capturar la multimodalidad inherente a la navegación en entornos no estructurados, superando la tendencia al promedio de los planificadores clásicos mediante la **proposición** de un abanico de trayectorias geométricamente viables.

H2. La integración de modelos de visión-lenguaje (VLM) dota al sistema de navegación de una capacidad de discriminación semántica de vocabulario abierto, permitiendo **seleccionar**, entre las propuestas generadas, aquellas que satisfacen preferencias simples en lenguaje natural, incorporando criterios de decisión contextuales inaccesibles para enfoques puramente geométricos.

1.3. Objetivo general

Diseñar, implementar y validar experimentalmente un módulo de navegación global sin mapa (*mapless*), que integre modelos generativos probabilísticos para la propuesta de trayectorias y modelos de visión-lenguaje para su selección.

1.4. Objetivos específicos

Para llevar a cabo el objetivo general presentado y la hipótesis, se plantean los siguientes objetivos específicos:

1. **Generar un conjunto de datos multimodal** a partir de trayectorias ejecutadas por un operador experto en una plataforma robótica, sincronizando mediciones de odometría, nubes de puntos e imágenes para el entrenamiento de modelos de aprendizaje.
2. **Desarrollar un planificador global** basado en modelos generativos condicionales (CVAE), capaz de proponer múltiples hipótesis de movimiento geométricamente viables.
3. **Implementar un mecanismo de selección de trayectorias** basado en Modelos de Visión-Lenguaje (VLM), que permita discriminar y jerarquizar las propuestas del planificador generativo utilizando preferencias en forma de vocabulario abierto.
4. **Validar el desempeño del sistema integrado** mediante pruebas experimentales en entornos del mundo real, utilizando una plataforma robótica tipo *skid-steer* y comparando métricas de éxito y transitabilidad.

1.5. Principales Contribuciones

El desarrollo de esta tesis presenta contribuciones en la intersección de la robótica móvil, los modelos generativos profundos y la inteligencia artificial multimodal. Las principales contribuciones se detallan a continuación:

- **Arquitectura de planificación generativa con *prior* aprendido:** Se propone un planificador global basado en Autoencoders Variacionales Condicionales (CVAE). Esta arquitectura modela la distribución de trayectorias condicionada al contexto específico del entorno. Esto permite mitigar el problema del "colapso de modo", permitiendo la generación de múltiples hipótesis de movimiento geométricamente viables.
- **Mecanismo de selección semántica de vocabulario abierto:** Se propone un módulo de discriminación de trayectorias que utiliza modelos de Visión-Lenguaje (VLM), específicamente CLIP *Segmentation* (CLIPSeg), para interpretar preferencias simples de navegación en lenguaje natural. Este enfoque permite generar mapas de costo

semánticos, dotando al robot de la capacidad de seguir instrucciones simples sin la necesidad de reentrenar el sistema para nuevas clases de objetos o terrenos..

- **Estrategia de actualización de costos asincrónica y dinámica:** Se diseña un algoritmo que desacopla la frecuencia de generación de trayectorias de la frecuencia de su selección. Esta estrategia incorpora un mecanismo de histéresis y actualización parcial de costos para manejar la incertidumbre en zonas ocluidas. Esto permite al robot reaccionar a nueva información semántica sin tener necesidad de abandonar las trayectorias actuales.
- **Validación experimental en entornos no estructurados:** Se realiza la implementación y validación del sistema de navegación completo en una plataforma robótica real tipo *skid-steer*, demostrando la viabilidad de la navegación *mapless* en escenarios exteriores complejos. Esto incluye la generación de un conjunto de datos multimodal propio que sincroniza odometría, percepción visual y demostraciones expertas.

1.6. Estructura del documento

Los siguientes capítulos cubren los siguientes temas:

- En los Capítulos 2 y 3 se presentan los fundamentos teóricos y la revisión bibliográfica. Se analizan conceptos clave sobre navegación autónoma y se profundiza en la arquitectura de los Autoencoders Variacionales.
- El Capítulo 4 detalla el diseño e implementación del sistema propuesto, enfocándose en el módulo de navegación global basado en modelos de visión-lenguaje y generación de trayectorias.
- En los capítulos 5 y 6 se presentan los resultados experimentales, así como su análisis y discusión.
- Por último, en el Capítulo 7 se presentan las conclusiones extraídas del trabajo realizado y las posibles direcciones en las que se podría extender.

Capítulo 2

Marco teórico

En el presente capítulo se introducirán los principales tópicos de interés para abordar el problema de navegación autónoma en ambientes de exterior. En primer lugar se explica qué es la navegación robótica y algunos tópicos relacionados. Se continúa con una introducción a los Autoencoders Variacionales y sus variantes, junto con una arquitectura de generación de trayectorias para navegación robótica del estado del arte. Finalmente se entrega una breve explicación de Modelos Fundacionales y las arquitecturas relevantes para este trabajo.

2.1. Navegación autónoma en robótica

La navegación en robótica móvil se entiende como la capacidad de un agente para determinar su propia posición dentro de un marco de referencia y planificar una trayectoria hacia un objetivo espacial, ejecutando movimientos libres de colisiones y eficiente bajo algún criterio. Este problema fundamental suele descomponerse en cuatro bloques interconectados: percepción, localización, planificación de movimiento y control [11].

Si bien el objetivo principal es desplazar a un robot desde un punto de inicio a un punto de fin en el espacio, la complejidad radica en hacerlo en entornos que pueden ser parcialmente desconocidos o dinámicos. Para abordar esto, la literatura clásica suele distinguir dos niveles de jerarquía en la toma de decisiones: la navegación global (deliberativa) y la navegación local (reactiva).

2.1.1. Navegación Global: El nivel deliberativo

La navegación global opera bajo la premisa de que el robot posee un conocimiento previo del entorno, generalmente representado mediante un mapa estático (mapa de ocupación, topológico o geométrico) [11]. En esta etapa, el sistema busca resolver el problema de la "conectividad", encontrando una secuencia de poses libres de obstáculos que unan el estado actual con el objetivo de navegación.

Como se ilustra en la Figura 2.1, el resultado de esta etapa es el **Plan Global**. Este plan suele ser una trayectoria geométrica idealizada (representada por la línea segmentada azul) que ignora muchas veces las limitaciones cinemáticas finas del robot o los obstáculos pequeños y dinámicos que no aparecen en el mapa estático. Algoritmos clásicos de búsqueda en grafos, como A^* (A-Star) [12] o Dijkstra [13], y algoritmos basados en muestreo como RRT (*Rapidly-exploring Random Trees*) [14], son los estándares para generar estas rutas.

2.1.2. Navegación Local: El nivel reactivo

Mientras que el planificador global "piensa" en todo el mapa, la navegación local "actúa" en el entorno inmediato. Esta etapa es crucial para compensar la discrepancia entre el modelo ideal del mapa y la realidad física variable.

La navegación local se centra en la seguridad inmediata y la viabilidad cinemática. El **Robot** utiliza sus sensores a bordo (LiDAR, cámaras, radar, etc.) para realizar una **Medición sensorial** constante de su entorno cercano. Esta información permite detectar obstáculos no mapeados o en movimiento.

El funcionamiento típico de este nivel, detallado en el recuadro derecho de la Fig. 2.1, consiste en seguir un **Sub-objetivo local**. Este sub-objetivo es un punto intermedio sobre la ruta global que atrae al robot, mientras que los obstáculos detectados generan fuerzas de repulsión o restricciones de velocidad. El resultado es la generación de **Acciones de control** (típicamente velocidad lineal y angular ejecutadas por un controlador de bajo nivel, representadas por la flecha verde), que permiten al robot desviarse momentáneamente de la ruta ideal para evitar colisiones, retomando el camino cuando el obstáculo ha sido superado.

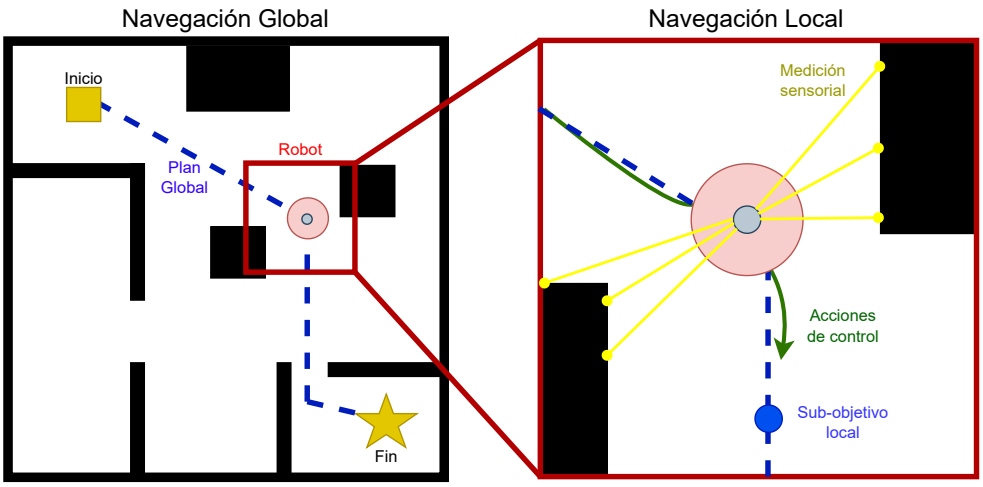


Figura 2.1: Diagrama para ilustrar la diferencia entre la planificación local vs. la planificación global. A la izquierda un plan global desde el inicio a fin dentro de un mapa conocido. A la derecha, un robot que ejecuta acciones de control con un planificador local para alcanzar sub-objetivos, haciendo uso de sus observaciones sensoriales.

2.1.3. Arquitectura Híbrida

En la práctica, la mayoría de los sistemas robustos implementan una arquitectura híbrida. El planificador global se ejecuta a baja frecuencia para asegurar que el robot no quede atrapado en "callejones sin salida" (mínimos locales), mientras que el planificador local se ejecuta a alta frecuencia (p. ej., 10-50 Hz) para garantizar una respuesta fluida ante imprevistos. Esta interacción asegura que el robot pueda navegar largas distancias de manera eficiente sin perder la capacidad de reaccionar ante un entorno cambiante.

2.2. Sistemas de Localización y Estimación de Pose

La localización es el proceso mediante el cual un agente robótico determina su posición y orientación (conocidas conjuntamente como *pose*) dentro de un sistema de coordenadas global o relativo. En el contexto de la navegación autónoma, conocer la pose es un prerrequisito fundamental para poder ejecutar un plan global. La literatura suele clasificar los métodos de localización en dos categorías: estimación relativa (generalmente denominado *Dead Reckoning*) y estimación absoluta.

La localización es el proceso mediante el cual un agente robótico determina su posición y orientación (conocidas conjuntamente como pose) dentro de un sistema de coordenadas global o relativo. En el contexto de la navegación autónoma, conocer la pose es un prerrequisito fundamental para poder ejecutar un plan global. La literatura suele clasificar los métodos de localización en dos categorías principales [15]: la estimación relativa (generalmente denominada *Dead Reckoning*), que calcula el avance a partir de mediciones propioceptivas continuas; y la estimación absoluta, que requiere referenciar la pose del robot respecto a un modelo espacial previo (como un mapa 2D), y se encuentra intrínsecamente ligada a la representación del entorno.

2.2.1. Odometría: Estimación Relativa

La odometría es el método más básico y ubicuo de localización. Consiste en estimar el cambio de posición del robot integrando las mediciones de velocidad y movimiento de sus actuadores (como *encoders* en las ruedas) o sensores inerciales (IMU).

Matemáticamente, si se conoce la posición inicial x_t, la odometría calcula x_{t+1} sumando el desplazamiento medido. Sin embargo, la principal limitación de este método es la acumulación de errores o deriva (*drift*). Pequeñas imprecisiones en las mediciones, deslizamientos de las ruedas o imperfecciones del terreno se suman incrementalmente, haciendo que la estimación de la posición diverja de la realidad a medida que pasa el tiempo. Por esta razón, la odometría se utiliza típicamente como una estimación local de alta frecuencia que debe ser corregida periódicamente por sistemas de localización global.

2.2.2. Sistemas de Localización Global

A diferencia de la odometría, los sistemas globales no dependen de la historia del movimiento previo y pueden determinar la posición del robot con respecto a un marco de referencia fijo, corrigiendo el error acumulado.

Sistemas de Navegación Global Satelital (GNSS)

En entornos exteriores, el uso de sistemas GNSS (como el GPS) es el estándar para obtener coordenadas absolutas (latitud, longitud, altitud). El GPS proporciona una referencia global que no sufre de deriva temporal. No obstante, presenta desafíos para la robótica móvil terrestre: su precisión estándar puede ser insuficiente para maniobras finas (margen de error de metros), la señal se degrada o pierde en interiores, y no proporciona información directa sobre la orientación del robot. Una solución a la precisión en espacios abiertos es el uso de RTK, donde una estación base con coordenadas conocidas se comunica con el GPS a bordo del robot y le envía correcciones de manera activa. Con esta configuración se pueden obtener precisiones de centímetros.

Localización de Monte Carlo Adaptativa (AMCL)

Para la navegación en interiores o en entornos donde el GPS no es fiable, se recurre a algoritmos probabilísticos que correlacionan las lecturas de los sensores exteroceptivos (como un LiDAR) con un mapa de ocupación conocido.

El algoritmo AMCL (*Adaptive Monte Carlo Localization*) [16] es una implementación del filtro de partículas. En lugar de calcular una única pose 2D (x, y, θ), AMCL mantiene una distribución de probabilidad representada por un conjunto de *partículas*. Cada partícula representa una hipótesis de dónde podría estar el robot en el espacio. El proceso de localización consta de tres pasos principales:

- **Predicción:** Las partículas se mueven según los datos de la odometría.
- **Corrección:** Se asigna un peso a cada partícula comparando qué tan bien coinciden las lecturas actuales del sensor láser con lo que el robot observaría si estuviese en la posición de esa partícula en el mapa.
- **Remuestreo:** Las partículas con bajo peso (hipótesis improbables) se eliminan, y las de alto peso se multiplican, convergiendo así hacia la posición real del robot.

2.3. Mapas de Elevación y Representación 2.5D

Si bien los mapas de ocupación bidimensionales son eficientes para la navegación en interiores con suelos planos, presentan limitaciones severas en entornos no estructurados

o con desniveles. En estos escenarios, asumir que el mundo es una proyección plana puede llevar a colisiones con el chasis del robot o a la imposibilidad de planificar rutas sobre terrenos transitables como rampas o terreno irregular.

Para solventar este problema sin incurrir en el alto costo computacional de mantener un mapa volumétrico 3D completo (como OctoMap [17]), se utilizan los Mapas de Elevación. Esta representación se clasifica como "2.5D", ya que consiste en una grilla bidimensional discreta donde cada celda (x, y) no almacena un estado de ocupación binario, sino una estimación de la altura z del terreno en ese punto.

2.3.1. Enfoque *Robot-Centric* y Fusión Sensorial

En el contexto de la navegación local y la evasión de obstáculos, no siempre es necesario mantener un mapa de elevación global de todo el entorno. El enfoque del estado del arte, desarrollado ampliamente por investigadores de ETH Zúrich, propone el uso de mapas de elevación centrados en el robot (*Robot-Centric Elevation Mapping*) [18].

Este método mantiene una grilla local que se desplaza en conjunto con el robot. A medida que este avanza, los datos sensoriales (provenientes de LIDAR o cámaras de profundidad) se fusionan en la grilla mediante un filtro de Kalman. Esto permite:

- **Manejo de incertidumbre:** Cada celda almacena no solo la altura, sino también la varianza del error, permitiendo descartar mediciones ruidosas.
- **Análisis de Transitabilidad:** Al conocer la geometría del terreno, es posible calcular gradientes y rugosidad para determinar si una celda es un obstáculo real (p. ej. una pared vertical) o una superficie navegable (p. ej. una pendiente suave).

Esta distinción es crítica para el cálculo de colisiones, ya que permite al planificador local evaluar si la base del robot puede pasar sobre una irregularidad del terreno sin que el chasis impacte con el suelo, algo imposible de determinar con un mapa local 2D convencional.

2.4. Autoencoders Variacionales (VAE)

Los Autoencoders Variacionales (VAE) [19] son una clase de modelos generativos que combinan técnicas de aprendizaje profundo con principios de inferencia variacional bayesiana. A diferencia de los autoencoders deterministas tradicionales, cuyo objetivo es minimizar directamente el error de reconstrucción bajo una función de pérdida como MSE entre la entrada y la salida, los VAE introducen un marco probabilístico que permite modelar la incertidumbre de los datos generados y obtener representaciones latentes más robustas.

La arquitectura de un VAE (véase la Fig. 2.2) consta de dos componentes principales:

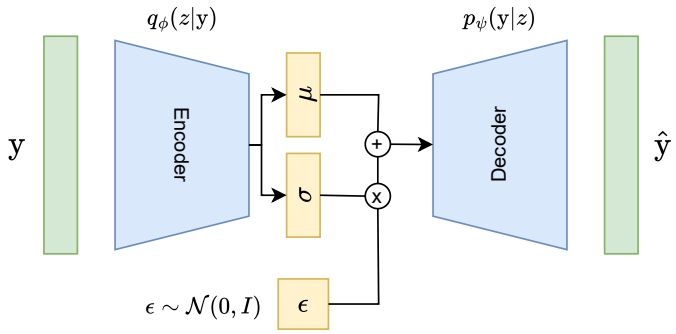


Figura 2.2: Arquitectura general de un VAE. Adaptado de [19].

1. **Encoder (red de reconocimiento):** recibe una muestra de entrada $\mathbf{y} \in \mathbb{R}^N$ y la mapea a los parámetros de una distribución gaussiana en el espacio latente. En particular, el encoder aprende a aproximar la distribución *posterior* verdadera $p(z|\mathbf{y})$ mediante una distribución variacional

$$q_\phi(z|\mathbf{y}) = \mathcal{N}\big(z; \mu_\phi(\mathbf{y}), \mathbf{\Sigma}_\phi(\mathbf{y})\big), \tag{2.1}$$

   donde ϕ denota los parámetros del encoder, $\mathcal{N}$ es la distribución normal, μ es el vector de medias y $\mathbf{\Sigma}$ la matriz de covarianza (generalmente diagonal) que define la incertidumbre de la representación latente.

2. **Decoder (red generativa):** toma como entrada una muestra $z \sim q_\phi(z|\mathbf{y})$ y genera datos en el espacio original a través de la distribución

$$p_\psi(\mathbf{y}|z), \tag{2.2}$$

   donde ψ corresponde a los parámetros del decoder. En la práctica, la distribución de salida $p_\psi(\mathbf{y}|z)$ se modela comúnmente como una distribución Gaussiana multivariada para datos continuos, o como una distribución de Bernoulli para datos binarios o discretos, dependiendo estrictamente de la naturaleza del dominio de los datos de entrenamiento [19, 20].

2.4.1. Objetivo de entrenamiento

El entrenamiento de un VAE se fundamenta en maximizar la *verosimilitud marginal* de los datos, también conocida como log-verosimilitud

$$\log p_\psi(\mathbf{y}) = \log \int p_\psi(\mathbf{y}|z)\, p(z)\, dz, \tag{2.3}$$

la cual resulta intratable en la práctica. Para abordarlo, se introduce una aproximación mediante el principio de inferencia variacional, optimizando la cota inferior de la evidencia (*Evidence Lower Bound*, ELBO)

$$\mathcal{L}_{\text{VAE}}(\psi, \phi; \mathbf{y}) = \mathbb{E}_{z\sim q_\phi(z|\mathbf{y})}\big[\log p_\psi(\mathbf{y}|z)\big] - D_{\text{KL}}\big(q_\phi(z|\mathbf{y}) \,\|\, p(z)\big), \tag{2.4}$$

donde el primer término corresponde a la verosimilitud esperada de reconstrucción y el segundo a la divergencia de Kullback-Leibler (KL) entre la distribución latente aproximada y el *prior* $p(z) = \mathcal{N}(\mathbf{0}, \mathbf{I})$.

En la práctica, la función de pérdida que se minimiza es la negativa de la ELBO,

$$\mathcal{J}(\psi, \phi; \mathbf{y}) = -\mathcal{L}_{\text{VAE}}(\psi, \phi; \mathbf{y}). \tag{2.5}$$

2.4.2. Inferencia

A diferencia de un Autoencoder tradicional, cuyo objetivo principal suele ser la reducción de dimensionalidad o la reconstrucción fiel de una entrada específica, el valor fundamental del VAE reside en su capacidad generativa. Una vez que el modelo ha sido entrenado y los parámetros del decodificador han convergido, el proceso de inferencia cambia drásticamente respecto a la etapa de entrenamiento.

Para generar nuevos datos, se prescinde completamente de la red codificadora (Encoder) y de los datos de entrada originales. En su lugar, se aprovecha la regularización impuesta sobre el espacio latente durante el entrenamiento. Dado que el VAE fuerza a que las variables latentes se aproximen a una distribución a priori conocida (típicamente una normal estándar $p(z) = \mathcal{N}(0, I)$), es posible realizar un muestreo directo sobre esta distribución.

El proceso de generación se resume en dos pasos:

1. **Muestreo del Espacio Latente:** Se genera un vector aleatorio z muestreando de la distribución normal estándar. Este vector actúa como una "semilla" estocástica que contiene las características abstractas del dato que se desea crear.
2. **Decodificación Generativa:** El vector z se introduce en la red decodificadora. En esta etapa, el decoder actúa como una función determinista $x_{gen} = D(z)$, que mapea el ruido aleatorio del espacio latente de vuelta al espacio de los datos originales, construyendo una muestra sintética completamente nueva.

Debido a la naturaleza continua del espacio latente en un VAE, es posible interpolar suavemente entre dos puntos aleatorios z_1 y z_2. Al decodificar los puntos intermedios de esta trayectoria, el modelo genera transiciones fluidas entre diferentes características de los datos (p. ej., transformar progresivamente la expresión de un rostro o la morfología de un dígito escrito a mano [19]), demostrando que el modelo ha aprendido la estructura subyacente de los datos y no solo ha memorizado los ejemplos de entrenamiento.

2.5. Autoencoders Variacionales Condicionales (CVAE)

A pesar de que los VAEs demuestran ser herramientas eficaces para aprender distribuciones complejas y generar datos sintéticos realistas, presentan una limitación fundamental en

su formulación original: carecen de un mecanismo intrínseco para controlar las características específicas de las muestras generadas. Dado que el proceso de generación depende exclusivamente del muestreo de una variable latente aleatoria z, la salida del modelo es estocástica y no dirigida.

Para abordar esta limitación, en [10] se propone el *Conditional Variational Autoencoder* (CVAE). Esta arquitectura introduce una variable de condicionamiento o contexto, denotada como $\mathbf{x}$, que permite modular tanto la proyección al espacio latente como el proceso de reconstrucción. Matemáticamente, el objetivo deja de ser modelar la probabilidad marginal $p(\mathbf{y})$, para pasar a modelar la probabilidad condicional $p(\mathbf{y}|\mathbf{x})$, donde $\mathbf{y}$ representa el dato objetivo y $\mathbf{x}$ un vector de contexto o codificación de éste.

Si bien existen diversas arquitecturas para integrar esta condicionalidad, esta tesis se enfoca en dos formulaciones específicas.

2.5.1. Formulación 1: *Vanilla* CVAE

La primera formulación, a menudo referida en la literatura como *Vanilla* CVAE debido a su simplicidad estructural, integra la condición mediante una concatenación directa en las entradas de las redes.

Sea $\mathbf{y} \in \mathbb{R}^{N_y}$ el dato de entrada que se desea reconstruir y $\mathbf{x} \in \mathbb{R}^{N_x}$ el vector de contexto que guía la generación. En esta arquitectura, el encoder recibe ambos vectores concatenados para aproximar la distribución posterior condicionada $q_\phi(z|\mathbf{x}, \mathbf{y})$. De manera análoga, el decoder recibe el vector latente z concatenado nuevamente con el contexto $\mathbf{x}$, modelando la distribución de verosimilitud $p_\psi(\mathbf{y}|z, \mathbf{x})$.

Bajo este esquema, la función de costo (ELBO) se adapta para incluir la dependencia de $\mathbf{x}$ en ambos términos

$$\mathcal{L}_{\text{CVAE}}(\psi, \phi; \mathbf{x}, \mathbf{y}) = \mathbb{E}_{z\sim q_\phi(z|\mathbf{x},\mathbf{y})}\big[\log p_\psi(\mathbf{y}|z, \mathbf{x})\big] - D_{\text{KL}}\big(q_\phi(z|\mathbf{x}, \mathbf{y}) \,\|\, p(z)\big). \tag{2.6}$$

Es importante notar que, en esta formulación simplificada, la distribución a priori $p(z)$ se mantiene generalmente como una normal estándar $\mathcal{N}(0, I)$, asumiendo que la estructura del espacio latente es independiente del contexto, dejando que sea el decodificador el encargado de interpretar la relación entre z y $\mathbf{x}$.

Inferencia en *Vanilla* CVAE

Durante la etapa de inferencia, el procedimiento emula al de un VAE estándar con la adición crítica del contexto. Se prescinde del codificador y se muestrea un vector latente $z \sim \mathcal{N}(0, I)$. Este vector se concatena con el contexto $\mathbf{x}$ y se introduce en el decoder para obtener el dato generado $\hat{\mathbf{y}}$. La Figura 2.3 ilustra esquemáticamente el flujo de datos tanto en la fase de entrenamiento como en la de inferencia.

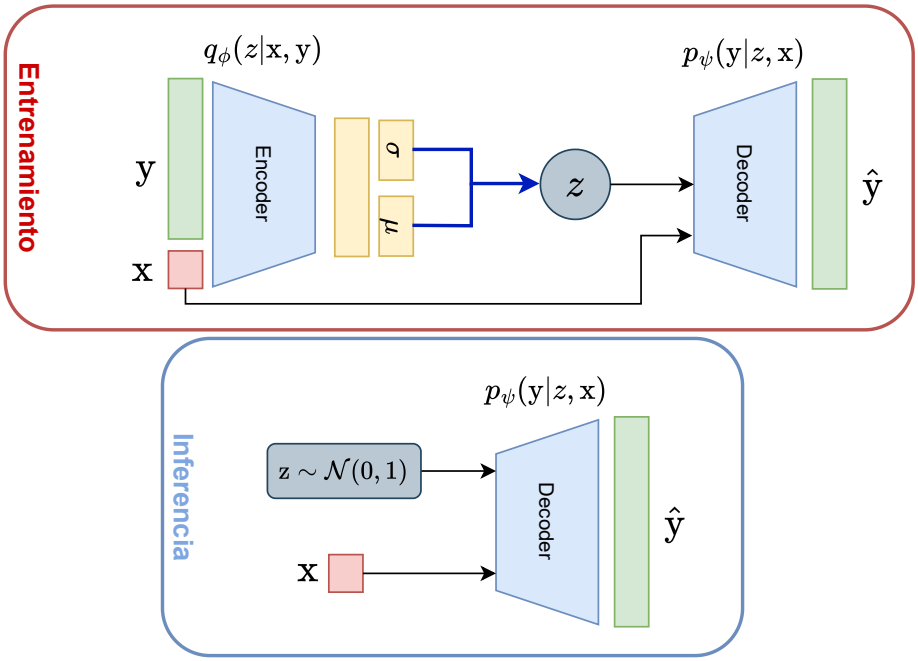


Figura 2.3: Arquitectura de un CVAE *Vanilla*. Durante el entrenamiento, tanto el encoder como el decoder reciben el contexto $\mathbf{x}$. Durante la inferencia, se utiliza solo el decoder con el contexto $\mathbf{x}$ y una muestra aleatoria z.

2.5.2. Formulación 2: CVAE con *Prior* Aprendido (*Learned Prior*)

La formulación anterior presenta una limitación teórica significativa: asume que la distribución a priori del espacio latente $p(z)$ es independiente del contexto $\mathbf{x}$, modelándola genéricamente como una normal estándar $\mathcal{N}(0, I)$. Sin embargo, en problemas de robótica y navegación, la "cantidad" de incertidumbre y las posibilidades de acción dependen fuertemente del entorno.

Para capturar esta dependencia, se adopta la arquitectura de *Learned Prior* CVAE, popularizada en el dominio de predicción de trayectorias por trabajos como *Trajectron++* [21]. En esta formulación, la distribución a priori deja de ser estática y pasa a ser parametrizada por una red neuronal p_θ.

El modelo se compone ahora de tres módulos principales (ver Figura 2.4):

1. **Red de Reconocimiento (*Posterior*):** Aproxima $q_\phi(z|\mathbf{x}, \mathbf{y})$, utilizando tanto el dato objetivo como el contexto.
2. **Red Generativa (Decoder):** Modela la verosimilitud $p_\psi(\mathbf{y}|z, \mathbf{x})$.
3. **Red *Prior* (*Prior Network*):** Es una red neuronal que toma únicamente el contexto $\mathbf{x}$ y predice los parámetros de la distribución latente: $p_\theta(z|\mathbf{x}) = \mathcal{N}(\mu_\theta(\mathbf{x}), \Sigma_\theta(\mathbf{x}))$.

El objetivo de entrenamiento se modifica en el término de regularización. La Divergencia

de Kullback-Leibler ya no "empuja" al posterior hacia el origen $(0, I)$, sino hacia la distribución predicha por la red *prior*. La función de pérdida (ELBO) queda definida como

$$\mathcal{L}_{\text{LP-CVAE}}(\psi, \phi, \theta; \mathbf{x}, \mathbf{y}) = \mathbb{E}_{z \sim q_\phi(z|\mathbf{x},\mathbf{y})}\big[\log p_\psi(\mathbf{y}|z, \mathbf{x})\big] - D_{\text{KL}}\big(q_\phi(z|\mathbf{x}, \mathbf{y}) \,\|\, p_\theta(z|\mathbf{x})\big). \tag{2.7}$$

Esta formulación es más potente porque permite que el espacio latente sea topológicamente consistente con el contexto. Durante el entrenamiento, la red *prior* p_θ aprende a "adivinar" dónde se ubicará el posterior q_ϕ basándose solo en el contexto, cerrando la brecha entre el entrenamiento y la inferencia.

Inferencia con *Prior* Aprendido

A diferencia del caso *Vanilla*, durante la inferencia no se muestrea ruido blanco. El contexto $\mathbf{x}$ se introduce en la red *prior* para obtener la distribución $p_\theta(z|\mathbf{x})$ específica para ese contexto. Luego, se muestrea z de esta distribución informada y se pasa al decoder. Esto resulta en generaciones más coherentes y precisas, ya que el punto de partida en el espacio latente ya está condicionado por la geometría o el estado del entorno.

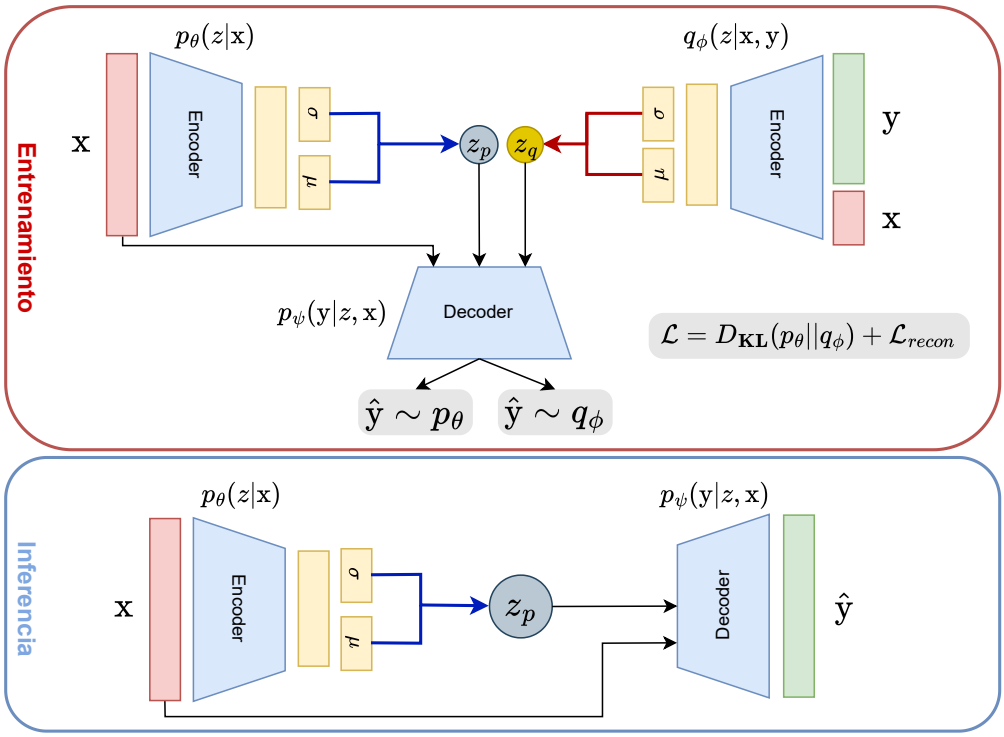


Figura 2.4: Arquitectura de un CVAE con *prior* aprendido. En entrenamiento se aprenden dos espacios latentes $z_q \sim q_\phi(z|\mathbf{x}, \mathbf{y})$ y $z_p \sim p_\theta(z|\mathbf{x})$ para estimar $\hat{\mathbf{y}}$. Durante la inferencia, se utiliza solo el decoder con el contexto $\mathbf{x}$ y una muestra z_p del *prior* aprendido.

2.6. MTG: *Mapless Trajectory Generation*

Para abordar el problema de la navegación global en entornos no estructurados y sin mapas previos, esta tesis usa como marco de partida la arquitectura propuesta en *MTG: Mapless Trajectory Generation* [22]. A diferencia de los planificadores locales tradicionales que buscan optimizar una única ruta inmediata, o de los planificadores globales que requieren un mapa completo, MTG se sitúa en un punto intermedio: genera un abanico de trayectorias candidatas de largo horizonte que exploran el espacio navegable disponible.

El objetivo central de MTG es maximizar la cobertura de áreas transitables (*Traversability Coverage*). En lugar de intentar predecir una única trayectoria "perfecta" (lo cual es ambiguo en espacios abiertos), el modelo busca predecir todas las trayectorias topológicamente posibles (p. ej., ir por la vereda izquierda, por el centro de la calle o por la vereda derecha), delegando la decisión final a un selector posterior. En la formulación original, esta selección se realiza tomando la trayectoria generada que minimiza la distancia al objetivo global.

Formalmente, en base a un conjunto de observaciones sensoriales $\mathbf{x}$ (nube de puntos 3D y velocidades lineales y angulares), MTG busca predecir un conjunto $\mathcal{Y}$ de K trayectorias diversas, donde cada trayectoria $\mathbf{y}_k$ es una secuencia de posiciones en el espacio 2D. Para predecir estas trayectorias, MTG adopta la formulación de CVAE *Vanilla* descrita en la sección 2.5, añadiendo dos componentes clave: un mecanismo para fomentar la diversidad estructural entre las trayectorias generadas y una función de pérdida que asegura cobertura y transitabilidad.

2.6.1. Diversidad Estructural mediante DLOW

Uno de los desafíos inherentes a los CVAE tradicionales es el *colapso de modo*, donde el modelo tiende a ignorar la variable latente z o a promediar las modas, generando trayectorias que van directo hacia los obstáculos o son unimodales. Para mitigar esto y garantizar la generación de hipótesis diversas, MTG integra la técnica de *Diversifying Latent Flows* (DLOW) [23].

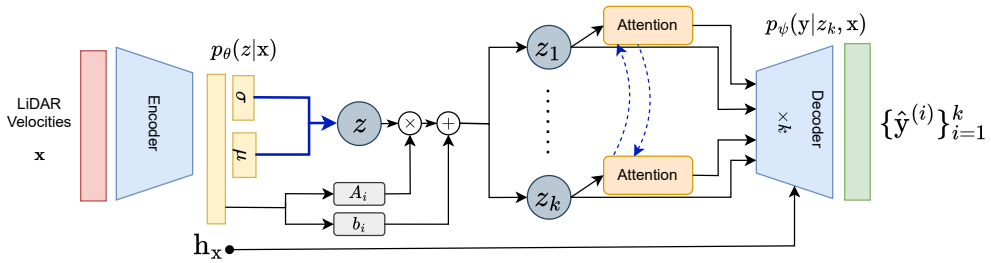


Figura 2.5: Arquitectura general de MTG. Adaptado de [22].

Como se ilustra en el diagrama de la arquitectura mostrada en la Figura 2.5, en lugar de decodificar directamente una única muestra $z \sim \mathcal{N}(0, I)$, se aplica un conjunto de K transformaciones afines aprendibles sobre la variable latente. Para cada modo $k \in \{1, ..., K\}$, se obtiene un vector latente transformado z_k

$$z_k = A_k z + b_k, \tag{2.8}$$

donde A_k y b_k son capas lineales que se optimizan durante el entrenamiento. Estas transformaciones fuerzan a que los códigos latentes z_k se separen en el espacio de características, resultando en trayectorias decodificadas que son geométricamente distintas y que cubren diferentes opciones de navegación (diversidad estructural), evitando que todas las muestras colapsen en una única media segura pero subóptima. Para fomentar la relación entre las trayectorias, se incorporan mecanismos de atención entre cada vector latente transformado z_k, para asegurar redundancia mínima entre ellas.

Dado que cada transformación afín opera inicialmente de manera independiente, los modos generados podrían resultar similares. Para mitigar esto y modelar explícitamente la interacción entre las distintas trayectorias, se aplica un mecanismo de *self-attention* sobre el conjunto de vectores latentes $\{z_1, \ldots, z_K\}$. Este mecanismo permite que cada vector integre información del estado de los demás antes de la decodificación. Al habilitar esta "comunicación cruzada", la red aprende a penalizar la similitud entre los códigos latentes, forzándolos a distribuirse de manera más uniforme en el espacio de características.

2.6.2. Cobertura y Transitabilidad

Para garantizar que las trayectorias generadas sean no solo precisas respecto a la demostración experta, sino también diversas y seguras, el entrenamiento del modelo MTG se rige por una función de pérdida compuesta $\mathcal{L}_{total}$. Esta se divide en tres componentes fundamentales:

- **Pérdida CVAE de Cobertura ($\mathcal{L}_{CVAE}$):** Combina la regularización estándar del espacio latente (Divergencia KL) con un término de reconstrucción modificado. A diferencia de un CVAE tradicional que fuerza una correspondencia uno a uno, aquí se utiliza una pérdida de "mejor coincidencia" (*best-of-many*). Se calcula la distancia de Hausdorff (d_H) entre la trayectoria real $\mathbf{y}$ y la trayectoria generada más cercana $\hat{\mathbf{y}} \in \mathcal{Y}$

$$\mathcal{L}_{CVAE} = \beta_1 D_{KL}(q(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z})) + \beta_2 d_H(\hat{\mathbf{y}}, \mathbf{y}), \tag{2.9}$$

  con β_1 y β_2 como coeficientes de ponderación.

- **Pérdida de Diversidad ($\mathcal{L}_{div}$):** Para evitar el colapso de modos (donde todas las predicciones convergen al promedio), se introduce un término repulsivo. Esta función penaliza a las trayectorias generadas redundantes (aquellas que no son la $\hat{\mathbf{y}}^*$ más cercana al *ground truth*) si se encuentran demasiado próximas entre sí o a la trayectoria óptima. Matemáticamente, utiliza exponenciales negativas de la distancia euclidiana ($d(.)$) entre pares de trayectorias generadas $(\mathbf{y}_c, \mathbf{y}_o)$, forzando al modelo a cubrir el espacio de soluciones posibles mediante las transformaciones afines latentes de DLOW

$$\mathcal{L}_{div} = \exp(-d(\hat{\mathbf{y}}, \mathbf{y}_c) + -d(\hat{\mathbf{y}}, \mathbf{y}_o)). \tag{2.10}$$

- **Pérdida de Transitabilidad ($\mathcal{L}_{trav}$):** Garantiza la seguridad física de las propuestas. Esta pérdida evalúa la distancia mínima entre los puntos $\mathbf{p}_n$ de cada trayectoria generada $\hat{\mathbf{y}}^{(k)}$ y el conjunto de obstáculos u áreas no transitables $\mathcal{C}$ detectadas por el sistema de percepción. La penalización crece exponencialmente si cualquier punto de la trayectoria invade una zona prohibida, orientando el gradiente de optimización hacia el espacio libre

$$\mathcal{L}_{trav} = \exp\left(1 - \text{mín}\left(\text{máx}\left(\frac{1}{N}\sum_{n=1}^{N}\text{mín}\, d(\mathcal{C}, \mathbf{p}_n), 1\right), 0\right)\right). \tag{2.11}$$

La función de pérdida total es una suma ponderada de estos tres componentes

$$\mathcal{L}_{total} = \mathcal{L}_{CVAE} + \lambda_{div}\mathcal{L}_{div} + \lambda_{trav}\mathcal{L}_{trav}, \tag{2.12}$$

donde los coeficientes λ_{trav} y λ_{div} equilibran la importancia de cada término.

2.7. Procesamiento de Nubes de Puntos: PointNet

El procesamiento directo de nubes de puntos 3D mediante aprendizaje profundo presenta desafíos únicos que no se encuentran en el procesamiento de imágenes 2D. A diferencia de los píxeles en una imagen, que residen en una grilla regular y ordenada, una nube de puntos es un conjunto irregular y no ordenado de vectores. Sea una nube de puntos representada como un conjunto $P = \{P_i | i = 1, \ldots, N\}$, donde cada P_i es un vector de coordenadas (x, y, z) más canales extra (p. ej., intensidad). Una arquitectura neuronal diseñada para tomar P debe satisfacer dos propiedades fundamentales:

1. **Invarianza a la Permutación:** Dado que P es un conjunto, el orden en que se alimentan los puntos a la red no debe alterar la salida global. Si π es una permutación de índices, se requiere que $f(\{P_1, \ldots, P_N\}) = f(\{P_{\pi(1)}, \ldots, P_{\pi(N)}\})$.
2. **Invarianza a Transformaciones Rígidas:** La clasificación o segmentación de un objeto no debe variar si la nube de puntos es rotada o trasladada en el espacio.

Para abordar esto, **PointNet** [24] propone una arquitectura que toma nubes de puntos crudas sin necesidad de voxelización o proyecciones previas, evitando así la pérdida de información geométrica y el aumento de costo computacional.

La innovación central de PointNet reside en el uso de una función simétrica para agregar la información de todos los puntos. La red aprende una función espacial h (implementada mediante MLPs) que se aplica idénticamente a cada punto, seguida de una función simétrica de *Max Pooling*

La innovación central de PointNet reside en el uso de una función simétrica para agregar la información de un conjunto de puntos P. El objetivo principal es aproximar una función

teórica global $f(P)$ que represente la nube de puntos. Para lograrlo, la red aprende una función espacial h (implementada mediante MLPs) que se aplica idénticamente a cada punto, seguida de una función agregadora simétrica g, implementada como una operación de *Max Pooling*

$$f(P) \approx g\left(h(P_1), h(P_2), \ldots, h(P_N)\right). \tag{2.13}$$

Esta operación de *Max Pooling* extrae un vector de características globales que representa la firma geométrica de toda la entrada, siendo intrínsecamente invariante al orden de entrada de los puntos.

Para tareas de segmentación semántica, como la requerida en este trabajo (clasificación punto a punto de transitabilidad), la información global no es suficiente, ya que se requiere conocimiento de la geometría local. PointNet resuelve esto concatenando el vector de características globales con las características locales de cada punto. Esto permite que la red clasifique cada punto basándose tanto en su geometría local como en el contexto global de la escena.

2.8. Modelos Fundacionales / Modelos de Visión Lenguaje

Los modelos fundacionales son una categoría de modelos de aprendizaje entrenados en grandes volúmenes de datos para capturar una comprensión profunda y generalizable de un dominio. Su principal fortaleza radica en su capacidad para aprender representaciones ricas y versátiles que pueden aplicarse en una amplia gama de tareas sin requerir un entrenamiento extenso en cada aplicación específica. Esta propiedad los convierte en una base poderosa para múltiples áreas, permitiendo, por ejemplo, resolver tareas complejas de lenguaje, visión y multimodales de manera eficiente [25].

Existen varios tipos de modelos fundacionales, entre los que destacan los Modelos de Lenguaje Extensos (LLMs), los Modelos Fundacionales Visuales (VFMs) y los Modelos Multimodales de Visión y Lenguaje (VLMs). Cada uno de estos modelos está especializado en un dominio específico o en la combinación de dominios, permitiéndoles abordar tareas complejas de manera adaptativa.

- **Modelos de Lenguaje Extensos (LLMs)**: Son modelos diseñados específicamente para procesar y generar lenguaje natural. Estos modelos han sido entrenados en enormes conjuntos de datos textuales, lo que les permite comprender contextos complejos, realizar inferencias y generar texto coherente en una variedad de contextos. Gracias a su estructura, los LLMs pueden aplicarse en tareas como la traducción automática, el resumen de textos, la generación de texto, e incluso en tareas más avanzadas como el análisis de sentimientos y la interacción en lenguaje natural. Un ejemplo de estos modelos es GPT [26].
- **Modelos Fundacionales Visuales (VFMs)**: Los VFMs son modelos especializados en interpretar y procesar información visual, como imágenes y videos. Modelos como

SAM [27] y DINO [28], entrenados en grandes conjuntos de datos de imágenes etiquetadas, pueden identificar y clasificar objetos, reconocer patrones visuales y realizar tareas complejas de visión. Estos modelos son ampliamente utilizados en aplicaciones de reconocimiento de imágenes, segmentación semántica y análisis de video, donde se requiere una comprensión detallada de los elementos visuales. Además, su capacidad para aprender representaciones visuales de alta calidad los hace útiles en áreas como la medicina, donde pueden asistir en el diagnóstico a partir de imágenes.

- **Modelos Multimodales de Visión y Lenguaje (VLMs)**: Los VLMs, integran capacidades tanto de lenguaje como de visión, permitiéndoles comprender y generar información en formatos textuales y visuales de manera combinada. Esto les permite realizar tareas que requieren una interpretación conjunta de texto e imagen, como la generación de descripciones para imágenes, la búsqueda de imágenes mediante lenguaje natural o la respuesta a preguntas visuales (*Visual Question Answering*) [29]. Los VLMs son especialmente útiles en contextos donde la interacción entre texto e imagen es crucial, como en sistemas de recomendación visuales, asistentes virtuales y aplicaciones de accesibilidad. Ejemplos de VLMs incluyen GPT-4 [30] y CLIP [31].

2.8.1. CLIP: *Contrastive Language-Image Pre-training*

El modelo CLIP, desarrollado por OpenAI [31], propone un aprendizaje basado en la alineación de espacios latentes. En lugar de predecir una etiqueta categórica, CLIP entrena conjuntamente un encoder de imágenes y un encoder de texto para predecir qué descripción textual corresponde a qué imagen dentro de un conjunto.

Formalmente, dado un conjunto de N pares (imagen, texto), el modelo busca maximizar la similitud coseno entre los *embeddings* de los pares correctos y minimizarla para los incorrectos, utilizando una función de pérdida contrastiva. Esto resulta en un espacio vectorial compartido donde imágenes visualmente similares y textos semánticamente equivalentes se ubican cerca unos de otros. Esta propiedad permite realizar clasificación *zero-shot*: se puede clasificar una imagen preguntando al modelo cuál de varias descripciones textuales (p. ej., "un camino de tierra", "una vereda de cemento") tiene mayor similitud con la imagen dada, sin haber entrenado explícitamente esas categorías.

2.8.2. CLIPSeg: Segmentación basada en Prompts

Si bien CLIP es potente para clasificación global (entender qué hay en la imagen completa), muchas tareas requieren localización espacial precisa (entender *dónde* está cada elemento). CLIPSeg [32] extiende la capacidad de CLIP para abordar tareas de segmentación densa y referencia visual.

La arquitectura de CLIPSeg reutiliza el encoder de imágenes de CLIP (basado típicamente en ViT - *Vision Transformer* [33]) como *backbone* para extraer características visuales. La innovación radica en del decoder condicional ligero. El funcionamiento es el siguiente:

1. **Codificación del Prompt:** Se introduce un texto arbitrario (el *prompt*, p. ej., "paso peatonal") en el encoder de texto de CLIP para obtener un vector de *embedding* condicional.
2. **Modulación:** Este vector se utiliza para modular las activaciones del decoder a través de capas de atención cruzada (*cross-attention*) o capas FiLM (*Feature-wise Linear Modulation*).
3. **Predicción Densa:** El decoder genera una máscara de probabilidad de un solo canal, donde cada píxel indica la similitud semántica entre esa región de la imagen y el texto de entrada.

Esta arquitectura permite realizar segmentación de vocabulario abierto (*Open-Vocabulary Segmentation*). A diferencia de los métodos clásicos, CLIPSeg no requiere reentrenamiento para detectar nuevos obstáculos o terrenos; simplemente basta con cambiar el texto de entrada. Para esta tesis, CLIPSeg actúa como el puente perceptual que traduce preferencias de navegación (lenguaje) en mapas de costo espaciales (visión), permitiendo distinguir zonas transitables.

Capítulo 3

Estado del Arte

La navegación autónoma en entornos no estructurados representa uno de los desafíos más persistentes en la robótica móvil. A diferencia de los entornos industriales controlados, los escenarios exteriores presentan una variabilidad infinita en términos de terreno, iluminación y dinámica de obstáculos. Históricamente, este problema se abordó mediante enfoques clásicos de "mapeo-planificación-control", donde la construcción de mapas métricos densos era un prerrequisito. Sin embargo, la literatura reciente muestra un cambio de paradigma hacia métodos basados en aprendizaje, los cuales prometen mayor robustez y capacidad de generalización [1].

Esta revisión del estado del arte se estructura en tres pilares fundamentales que sustentan la propuesta de esta tesis: la estimación de transitabilidad (entender el entorno), la generación de trayectorias multimodal (proponer movimientos posibles) y el uso de modelos fundacionales para el razonamiento semántico (decidir el mejor curso de acción).

3.1. Estimación de Transitabilidad y Percepción Semántica

El primer requisito para la navegación autónoma es la capacidad de distinguir entre zonas navegables y obstáculos. Mientras que los métodos tradicionales se basan puramente en la geometría (p. ej., análisis de pendiente o rugosidad en nubes de puntos), los enfoques modernos integran información semántica para diferenciar terrenos que, aunque geométricamente similares, poseen propiedades de fricción o seguridad distintas (p. ej., césped alto vs. rocas).

3.1.1. Enfoques Geométricos y Aprendizaje Autosupervisado

La base de la transitabilidad suele residir en la propiocepción. Trabajos como [34] demostraron la eficacia de aprender la relación entre la apariencia visual del terreno y la vibración experimentada por el vehículo. Recientemente, esta idea ha evolucionado hacia el aprendiza-

je autosupervisado profundo. En [35], se propone el uso de señales de torque de los pies de un robot cuadrúpedo para etiquetar automáticamente imágenes del terreno. En [36] usan el mismo principio de aprendizaje autosupervisado para estimar mapas de transitabilidad directamente de la imagen RGB, proyectando la trayectoria seguida por el robot en la imagen y etiquetando las zonas transitadas como seguras.

De manera similar, [37] y [38] utilizan grandes conjuntos de datos multimodales para aprender funciones de costo de transitabilidad que correlacionan la geometría del entorno con la dinámica del vehículo. En [39], se introduce un enfoque novedoso donde el robot aprende a relacionar sus mediciones propioceptivas con las demostraciones del usuario.

3.1.2. Segmentación Semántica y Fusión de Mapas

Para ir más allá de la rugosidad, es necesario entender la clase del terreno. Arquitecturas de segmentación semántica como *SegFormer* [40] o *Mask2Former* han sido adaptadas para navegación robótica. En [41], se propone una arquitectura híbrida que fusiona mapas de elevación con etiquetas semánticas para la locomoción de cuadrúpedos, permitiendo preferir terrenos blandos sobre asfalto duro para reducir el impacto en las articulaciones. En [4] se utilizan características pre-entrenadas y aprendizaje autosupervisado para clasificar terrenos en función de una señal de transitabilidad calculada en línea, con el fin de obtener mapas de transitabilidad 2.5D.

Otros trabajos como [42] y [43] extienden esto al dominio temporal, utilizando mecanismos de atención para no solo clasificar el terreno estático, sino también predecir el movimiento de agentes dinámicos, generando mapas de costo que codifican tanto la transitabilidad física como la social.

3.2. De la Navegación Reactiva a la Generativa

Una vez que el robot comprende su entorno, debe decidir cómo moverse. Aquí, la literatura ha transitado desde políticas reactivas simples hacia modelos generativos sofisticados capaces de manejar la incertidumbre y la multimodalidad.

3.2.1. Aprendizaje Reforzado y Aprendizaje por Imitación

El Aprendizaje Reforzado (RL) ha sido una herramienta dominante para aprender políticas de navegación complejas. La formulación del problema como un Proceso de Decisión de Markov (MDP) ha permitido éxitos notables. En [44] y [45], se emplean mapas de elevación y nubes de puntos para entrenar agentes capaces de sortear obstáculos sin mapas globales. Para mitigar el alto costo computacional de RL, enfoques como [46] combinan RL con Imitación (IL), mientras que [47] y [48] proponen esquemas de entrenamiento *offline* seguidos de ajustes *online*, logrando despliegues ágiles en el mundo real.

Sin embargo, el RL a menudo sufre de inestabilidad y dificultad para definir funciones de recompensa que capturen comportamientos a largo plazo. Como respuesta, el Aprendizaje por Imitación a gran escala ha ganado tracción. El laboratorio RAIL de UC Berkeley ha liderado esta línea con trabajos como ViKiNG [49], GNM [50] y ViNT [51]. Estos modelos, entrenados con decenas de horas de datos de navegación heterogéneos, aprenden a proponer *waypoints* y velocidades directamente desde imágenes, demostrando una capacidad de generalización superior a la de las políticas de RL tradicionales entrenadas en entornos específicos. Estos modelos son evaluados en múltiples plataformas robóticas con distintas cinemáticas y son adaptables a distintas modalidades de entrada, como imágenes satelitales para navegación a gran escala.

3.2.2. Modelos Generativos de Trayectorias

Una limitación crítica de los enfoques basados en regresión directa (como ViNT) es la *unimodalidad*: tienden a promediar las posibles soluciones. Si un robot puede rodear un árbol por la izquierda o por la derecha, un modelo de regresión podría proponer ir por el centro.

Para resolver esto, los modelos generativos han emergido como el estado del arte. Originalmente diseñados para la predicción de trayectorias humanas —como los influyentes *Social GAN* [6] y posteriormente *Trajectron* [5] y *Trajectron++* [21]—, estos métodos utilizan CVAEs (Conditional Variational Autoencoders), difusión [52, 53] o GANs [54, 55] para modelar la distribución multimodal del movimiento de un agente. En [56] se propone un enfoque jerárquico donde un modelo CVAE predice objetivos de largo plazo y luego un segundo CVAE predice trayectorias entre dichos objetivos, refinando las predicciones. El segundo CVAE es relevante para esta tesis y se utiliza como base para la arquitectura propuesta en las secciones siguientes.

Recientemente, estos conceptos se han transferido al ámbito de planificación robótica. En [22] (*Mapless Trajectory Generation*), se utilizan CVAEs para generar múltiples hipótesis de movimiento viables en entornos urbanos densos. Los mismos autores proponen *Difussion Trajectory Generation* en [57], extendiendo el trabajo a funcionar con modelos de difusión, logrando mejores resultados en navegación, pero sin la posibilidad de predecir múltiples trayectorias diversas.

Otro trabajo relevante con difusión se propone en *NoMaD* [7], donde se emplean políticas de difusión para la exploración autónoma, permitiendo al robot generar representaciones visuales intermedias en su planificación, hasta convertirlas en rutas viables que respetan la geometría del entorno.

3.3. Modelos Fundacionales de Visión-Lenguaje (VLMs)

Mientras que los modelos generativos resuelven el problema de "cómo moverse" de forma segura, a menudo carecen de la capacidad de entender instrucciones de alto nivel o contexto semántico complejo. Aquí es donde los Modelos Fundacionales se proponen como punto de

partida para transferir conocimiento, sin necesidad de entrenar políticas desde cero.

3.3.1. Representaciones Visuales Generalizables

El uso de representaciones pre-entrenadas ha demostrado ser más eficiente que el aprendizaje *end-to-end* desde píxeles puros. En [58] y [4], se utilizan características extraídas por DINO para estimar transitabilidad y velocidad en terrenos desconocidos *zero-shot*. En [59] extiende esto alineando representaciones visuales con atención para obtener segmentaciones semánticas útiles para la navegación bajo la intuición humana.

3.3.2. VLMs como Razonadores de Navegación

La integración de Modelos de Visión-Lenguaje ha permitido la interacción humano-robot en la navegación robótica. En trabajos como [60], [9] y [61] se demuestra como los VLMs y LLMs pueden interpretar instrucciones naturales narrativas para guiar la navegación en interiores y exteriores, construyendo mapas semánticos y/o relacionando observaciones con comandos textuales.

Sin embargo, el avance más relevante para esta tesis se encuentra en la intersección de VLMs y control de bajo nivel en exteriores. En [62] se introduce el concepto de *Iterative Vision-Language Planning*, donde el modelo no genera acciones directas, sino que "selecciona" la mejor trayectoria de un conjunto de candidatos basándose en una instrucción de texto. Paralelamente, en [63] se utilizan VLMs para anotar el contexto del entorno (p. ej., "zona concurrida", "césped") y modular la agresividad del planificador. En [64] se usa una combinación de señales propioceptivas para generar señales de transitabilidad del terreno con una imagen satelital anotada entre el punto inicial y objetivo, para que un VLM seleccione la mejor ruta en base a las preferencias del usuario.

El trabajo más cercano a la propuesta de esta tesis es *TGS* (*Trajectory Generation and Selection*) [65] y su evolución *MOSU* (*Multi-modal Scene Understanding*) [66], ambos trabajos de la Universidad de Maryland. En estos estudios, se emplea un modelo CVAE (específicamente el de [22]) para generar una nube de posibles sub-objetivos geométricamente viables, y posteriormente proyectados al espacio de la imagen, un VLM selecciona cuál de estos puntos maximiza la alineación con una instrucción en lenguaje natural (p. ej., "elegir el mejor camino, evitando caminos de tierra o caminos vehiculares"). Este paradigma de "Generar y Seleccionar" permite combinar la robustez física de los modelos generativos con la flexibilidad semántica de los modelos de lenguaje, estableciendo el punto de partida para la investigación presentada en esta tesis.

Capítulo 4

Navegación Global: Generación y Selección de Trayectorias

El objetivo principal de este trabajo es resolver el problema de navegación global sin mapa, mediante el uso de modelos generativos y modelos de visión lenguaje. La evaluación del módulo propuesto se realiza en el mundo real, en combinación con un planificador local del estado del arte. A continuación, se da una vista general del planificador global y luego se explica cada submódulo de este.

4.1. Descripción general de la solución

La arquitectura del planificador global propuesto se divide en dos etapas secuenciales principales: la **Generación** y la **Selección**.

En la primera etapa, el submódulo de *Generación de Trayectorias con CVAE* (NaVAE - *Navigation VAE*) utiliza las observaciones del estado del robot provenientes del sistema de percepción y el objetivo de navegación global para inferir una distribución de trayectorias futuras posibles. De esta distribución se muestrea un conjunto $\mathcal{Y}$ de trayectorias factibles en un horizonte de tiempo τ.

Posteriormente, el submódulo de selección procesa el conjunto $\mathcal{Y}$ para determinar la mejor opción de navegación. Este proceso inicia con un *Filtrado de Colisiones*, el cual descarta aquellas trayectorias inviables utilizando la información geométrica del entorno (nubes de puntos PCL). Las trayectorias restantes son evaluadas mediante un módulo de *Cálculo de Costos por Trayectoria*, la cual usa un mapa de costos semánticos y la proyección de las trayectorias a dicho mapa. El mapa de costos utilizado proviene del módulo de *Construcción de Mapa de Costos con VLM*, el cual utiliza un modelo de visión lenguaje que integra la información visual de la cámara con las preferencias del usuario para puntuar la navegabilidad del terreno.

Finalmente, se selecciona la trayectoria $\mathbf{y}^*$ que minimiza los costos calculados anteriormente. Esta solución es enviada al módulo de *Actualización de Costos y Trayectorias*, la cual

gestiona la transición asíncrona comparando la trayectoria en ejecución ($\mathbf{y}^*_{current}$) con la nueva mejor candidata ($\mathbf{y}^*_{candidate}$), enviando el resultado final $\mathbf{y}^*$ al planificador local de manera secuencial. En la Figura 4.1 se presenta el esquema de funcionamiento general del sistema completo antes descrito.

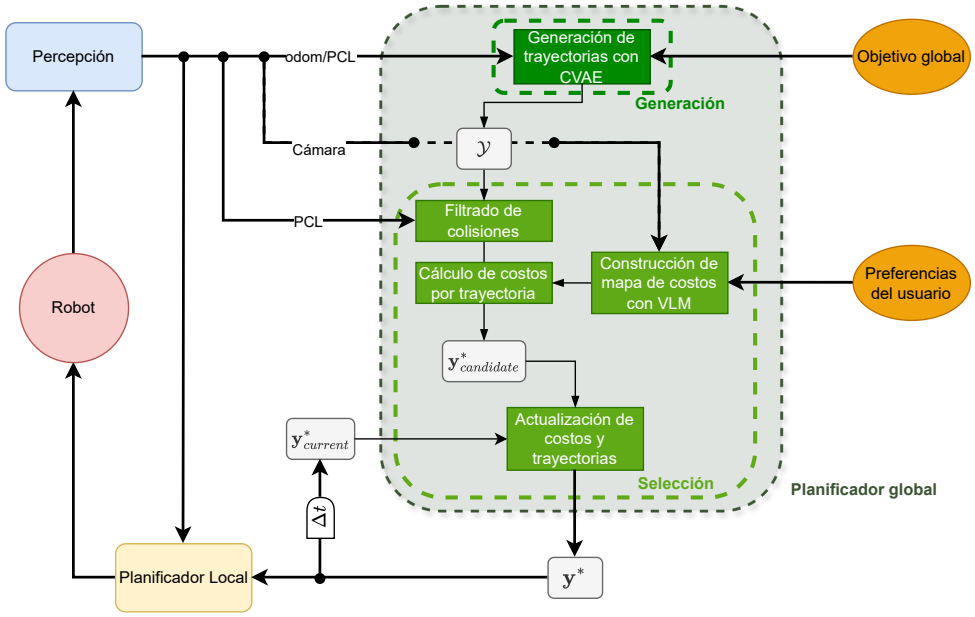


Figura 4.1: Diagrama general de la solución al problema de navegación global.

El resto de este capítulo detalla cada componente: primero se profundiza en el modelo generador, su arquitectura, la lógica probabilística y su entrenamiento; y posteriormente se describe el módulo de selección, detallando los componentes de filtrado, la generación de mapas de costos semánticos y la lógica de actualización continua.

4.2. Generación de trayectorias con CVAE

4.2.1. Formulación del Problema

Se considera un agente robótico navegando en un entorno dinámico, dotado de un sistema de percepción multimodal. En un instante de tiempo t, el agente tiene acceso a un historial de observaciones e información de navegación, denotado como el contexto $\mathbf{x}$. Este contexto se define como una estructura compuesta por tres modalidades

$$\mathbf{x} = \{\mathbf{x}_{\text{lidar}}, \mathbf{x}_{\text{traj}}, \mathbf{x}_{\text{goal}}\}, \tag{4.1}$$

donde:

- $\mathbf{x}_{\text{lidar}} \in \mathbb{R}^{N_l \times N_p \times 4}$ representa la secuencia temporal de las últimas N_l nubes de puntos

procesadas. Cada nube contiene N_p puntos, descritos por sus coordenadas espaciales (x, y, z) y su intensidad: $\{\mathbf{s}^{\text{lidar}}_{t-N_l+1}, \ldots, \mathbf{s}^{\text{lidar}}_{t}\}$.

- $\mathbf{x}_{\text{traj}} \in \mathbb{R}^{N_v \times 4}$ corresponde al historial de los últimos N_v estados del robot, incluyendo posición y velocidadades lineal y angular: $\{\mathbf{s}^{\text{state}}_{t-N_v+1}, \ldots, \mathbf{s}^{\text{state}}_{t}\}$.
- $\mathbf{x}_{\text{goal}} \in \mathbb{R}^2$ denota el objetivo global de navegación, expresado en coordenadas polares (ρ, θ) relativas al marco de referencia actual del robot.

El objetivo es predecir el comportamiento futuro del agente en un horizonte de predicción de longitud temporal τ. Dividiendo este horizonte en intervalos discretos Δt, se define la trayectoria futura verdadera como una secuencia de N_w posiciones espaciales $\mathbf{y} = \{\mathbf{p}_{t+1}, \ldots, \mathbf{p}_{t+\tau}\}$, donde cada $\mathbf{p}_t \in \mathbb{R}^2$ corresponde a las coordenadas 2D en el sistema de referencia local del robot.

Debido a la incertidumbre inherente en la navegación y la interacción con el entorno, el futuro no es determinista. Existe una variedad de trayectorias físicamente plausibles para un mismo contexto temporal $\mathbf{x}$. Por lo tanto, en lugar de buscar una única estimación puntual con una función determinista $\hat{\mathbf{y}} = f(\mathbf{x})$, se modela la incertidumbre futura mediante una distribución de probabilidad condicional

$$p(\mathbf{y} \mid \mathbf{x}). \tag{4.2}$$

Dado que esta distribución subyacente puede ser altamente compleja y multimodal (conteniendo múltiples comportamientos posibles, como girar a la izquierda o continuar recto), el objetivo es aprender un modelo generativo de parámetros ψ que aproxime esta distribución $p_\psi(\mathbf{y} \mid \mathbf{x})$.

Operativamente, se busca que el modelo sea capaz de generar un conjunto diverso de K trayectorias hipotéticas $\mathcal{Y} = \{\hat{\mathbf{y}}^{(k)}\}_{k=1}^{K}$ muestreadas de dicha distribución aprendida

$$\hat{\mathbf{y}}^{(k)} \sim p_\psi(\mathbf{y} \mid \mathbf{x}), \quad \text{para } k = 1, \ldots, K. \tag{4.3}$$

De esta forma, el problema se reduce a maximizar la verosimilitud de las trayectorias reales observadas bajo la distribución predictiva del modelo, asegurando que el conjunto de predicciones $\mathcal{Y}$ cubra los diversos modos de la distribución real del futuro.

Para lo anterior, se propone una arquitectura basada en CVAE con *prior* aprendido (véase la Sección 2.5.2), siguiendo el esquema propuesto en los trabajos [56] y [5].

4.2.2. Arquitectura propuesta (NaVAE)

El objetivo principal del generador CVAE es aprender una distribución de probabilidad sobre las trayectorias futuras condicionadas a las observaciones sensoriales. Dicha distribución se modela mediante un decoder estocástico gaussiano $p_\psi(\mathbf{y}|z, \mathbf{x})$, donde $\mathbf{y}$ es la trayectoria

generada, $\mathbf{x}$ el vector de contexto (observaciones sensoriales codificadas), z es una muestra del espacio latente aprendido y ψ los parámetros del decoder.

Para estimar los parámetros representantes del espacio latente $(\mu, \mathbf{\Sigma})$, se utiliza una combinación de dos distribuciones. En primer lugar, se tiene la distribución *prior* $p_\theta(z|\mathbf{x})$, modelada por un encoder de parámetros θ. Esta distribución tiene como objetivo codificar la información relevante del contexto $\mathbf{x}$ en el espacio latente y es la utilizada al momento de hacer inferencia. Esta red se encarga de procesar las observaciones en $\mathbf{x}$ para generar una representación de menor dimensionalidad $\mathbf{h_x}$ que pueda ser usada como valor condicional para el encoder posterior q_ϕ y el decoder p_ψ (ver Sección 2.5.2).

En segundo lugar, se tiene la distribución *posterior* $q_\phi(z|\mathbf{x}, \mathbf{y})$, modelada por un encoder de parámetros ϕ. Esta distribución tiene como objetivo codificar la información relevante tanto de $\mathbf{x}$ como de la trayectoria real (*ground truth*) $\mathbf{y}_{\text{GT}}$ en un espacio latente (μ_q, Σ_q). Esta distribución es usada solo durante el entrenamiento, de manera que se pueda forzar que la distribución *prior* sea similar a la *posterior* sin contener la trayectoria real.

En la Figura 4.2 se presenta la arquitectura de generación de trayectorias propuesta, donde se pueden observar los tres módulos principales: el encoder *prior*, el encoder *posterior* y el decoder. A continuación, se describe el pre entrenamiento del módulo de percepción de nubes de puntos, los encoders de las distribuciones y finalmente se describen las funciones de pérdida utilizadas para entrenar el modelo.

4.2.3. Pre-entrenamiento del Módulo de Percepción (PointNet)

Como se muestra en la Figura 4.1, el modelo integra una red PointNet pre-entrenada para la extracción de características semánticas del entorno. El objetivo principal de este módulo es realizar una segmentación semántica binaria sobre la nube de puntos voxelizada, clasificando cada punto en las categorías *transitable* y *no-transitable*. Estas predicciones son posteriormente transformadas en mapas de calor de transitabilidad que condicionan el decodificador de trayectorias.

Formalmente, sea $\mathbf{x_{lidar}} = \{\mathbf{p}_i\}_{i=1}^{N_p}$ la nube de puntos de entrada con N_p puntos, donde cada punto $\mathbf{p}_i \in \mathbb{R}^4$ contiene información espacial y de intensidad. La red busca estimar una función de mapeo $f_\phi : \mathbf{x_{lidar}} \to \mathbb{R}^{N\times 2}$, donde la salida corresponde a las log-probabilidades (*logits*) de pertenencia a las clases $\mathcal{C} = \{\text{transitable}, \text{no-transitable}\}$.

Generación de Etiquetas y Entrenamiento

Para el entrenamiento supervisado de este módulo, se generan etiquetas de *Ground Truth* aprovechando la información global de la base de datos descrita en la Sección 5.1. Se define como región transitable estrictamente aquella correspondiente a la clase "camino" (color azul en la Fig. 5.1).

El proceso de etiquetado consiste en una proyección de las coordenadas (x, y) de cada punto $\mathbf{p}_i$ sobre el mapa semántico global, filtrando aquellos puntos cuya altura z excede 1.5

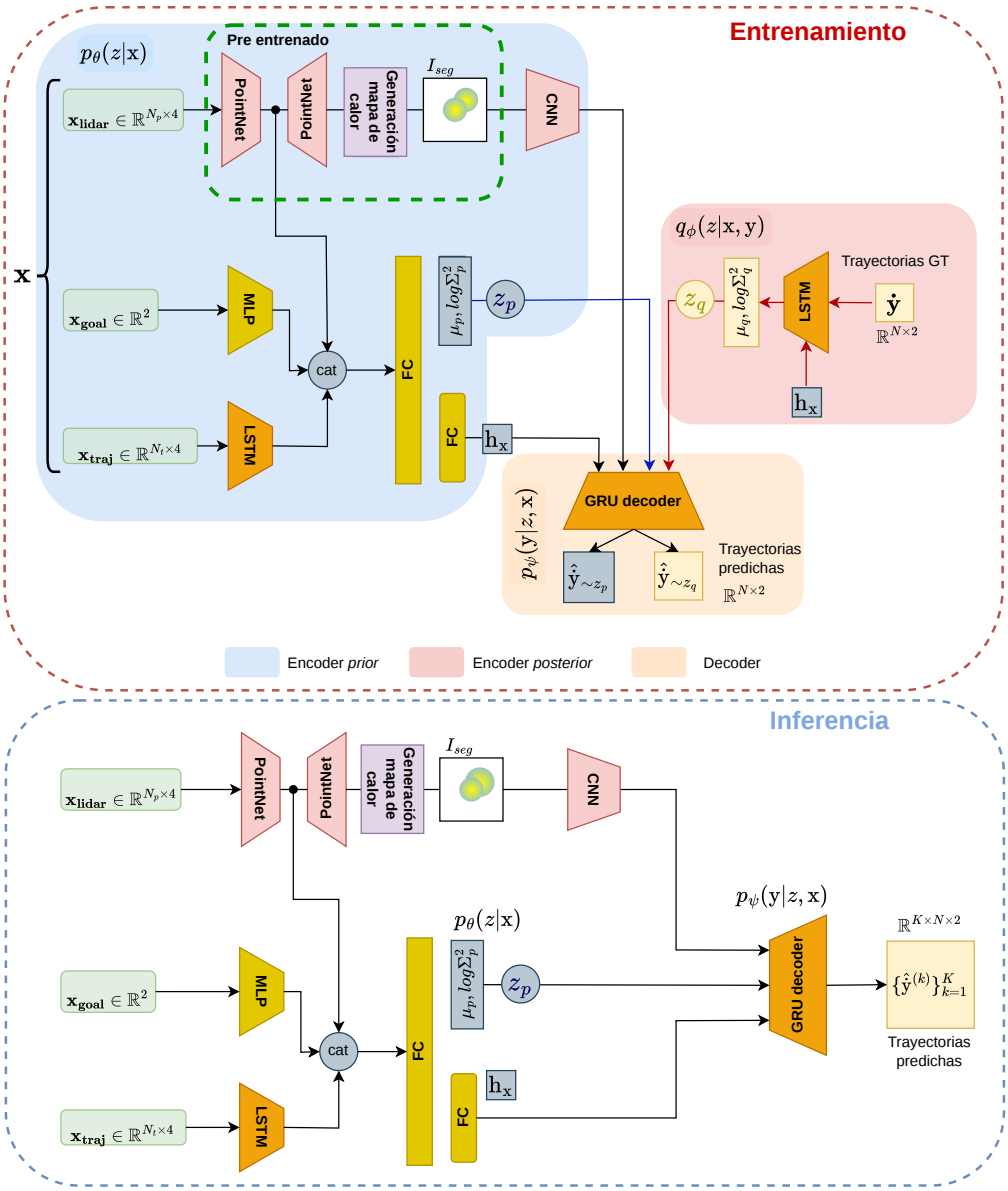


Figura 4.2: Arquitectura del modelo generativo propuesto, basado en CVAE con *prior* aprendido. Durante el entrenamiento se usan las trayectorias reales para calcular la distribución *posterior* $q_\phi(z|\mathbf{x},\mathbf{y})$, mientras que en inferencia solo se usa la distribución *prior* $p_\theta(z|\mathbf{x})$.

veces la altura del robot para descartar obstáculos elevados, techos o copas de árboles. La Figura 4.3 ilustra este proceso de asociación datos-etiqueta.

El entrenamiento se lleva a cabo minimizando la entropía cruzada (*Cross-Entropy Loss*) entre la distribución predicha por la red y las etiquetas generadas por proyección. Sea $y_i \in \{0, 1\}$ la etiqueta binaria real del punto i (donde 1 es transitable) y $\hat{y}_i$ la probabilidad predicha tras aplicar una función *Softmax* sobre los *logits* de salida. La función de pérdida se define como

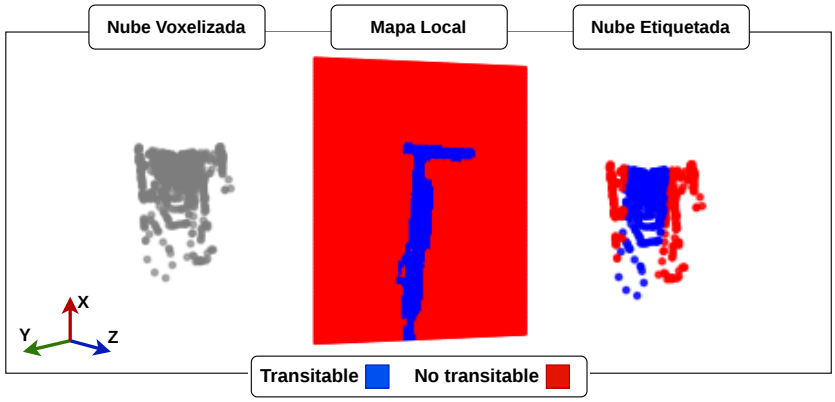


Figura 4.3: Proceso de generación de *Ground Truth* para la nube de puntos. Izquierda: Nube de puntos voxelizada de entrada. Centro: Mapa semántico local correspondiente a la pose del robot. Derecha: Proyección de las etiquetas del mapa semántico sobre la nube de puntos 3D.

$$\mathcal{L}_{\text{seg}} = -\frac{1}{N}\sum_{i=1}^{N}\left[w_1 y_i \log(\hat{y}_i) + w_0(1-y_i)\log(1-\hat{y}_i)\right], \tag{4.4}$$

donde w_0, w_1 son pesos escalares introducidos para compensar el desbalance de clases inherente a las nubes de puntos (donde los puntos de suelo (transitable) suelen ser minoría frente a obstáculos y fondo).

Generación de Mapas de Calor de Segmentación $\mathbf{I}_{seg}$

Para integrar la información semántica dispersa de la nube de puntos de forma condicional en $p_\psi(\mathbf{y}|z,\mathbf{x})$, tanto como entrada como para guiar la decodificación, se realiza un proceso de rasterización sobre una grilla 2D regular de resolución $\delta = 0{,}2(m)$. Sean H y W las dimensiones de la grilla local centrada en el robot.

Primero, definimos una función de discretización $\Phi : \mathbb{R}^2 \to \mathbb{Z}^2$ que mapea la posición (x_k, y_k) de cada punto $\mathbf{p}_k$ en la nube a índices de celda (u, v)

$$u_k = \left\lfloor \frac{x_k - x_{min}}{\delta} \right\rfloor, \quad v_k = \left\lfloor \frac{y_k - y_{min}}{\delta} \right\rfloor. \tag{4.5}$$

Se construyen dos grillas de ocupación, M^+ (transitable) y M^- (no transitable). El valor de cada celda (u, v) en la grilla M se calcula como el promedio de los *scores* s_k (*logits*) de todos los puntos que caen dentro de dicha celda

$$M_{u,v} = \begin{cases} \frac{1}{|\mathcal{K}_{u,v}|}\sum_{k\in\mathcal{N}_{u,v}} s_k & \text{si } |\mathcal{K}_{u,v}| > 0 \\ 0 & \text{en otro caso,} \end{cases} \tag{4.6}$$

donde $\mathcal{K}_{u,v} = \{k \mid \Phi(\mathbf{p}_k) = (u, v)\}$ corresponde al conjunto de índices de puntos proyectados en la celda (u, v). Esta normalización por conteo $|\mathcal{N}_{u,v}|$ asegura que el mapa represente la confianza media de la clasificación y sea invariante a la densidad variable de la nube de puntos.

Finalmente, el mapa de calor de segmentación $\mathbf{I}_{seg}$ se obtiene combinando ambas grillas y aplicando un filtro de suavizado espacial para propagar la información a celdas adyacentes vacías y mitigar el ruido de discretización

$$\mathbf{I}_{seg} = \mathcal{G}_\sigma * (M^+ - M^- + \mathbf{1}), \tag{4.7}$$

donde $\mathcal{G}_\sigma$ denota un kernel Gaussiano con desviación estándar $\sigma = 5$ (correspondiente a $1(m)$ dado δ), $*$ es la operación de convolución, y $\mathbf{1}$ es una matriz de unos, usada como sesgo para centrar el rango dinámico de las probabilidades.

4.2.4. Encoder *prior* $p_\theta(z|\mathbf{x})$

Como se presentó anteriormente, el objetivo de este encoder es procesar las observaciones sensoriales del robot para extraer características relevantes que describan el contexto actual. Estas características son utilizadas para parametrizar la distribución *prior* del espacio latente $p_\theta(z|\mathbf{x})$ y obtener la representación de baja dimensionalidad $\mathbf{h_x}$. En la Figura 4.4 se presenta la arquitectura detallada del encoder *prior*. Los módulos específicos para cada modalidad de entrada se describen a continuación.

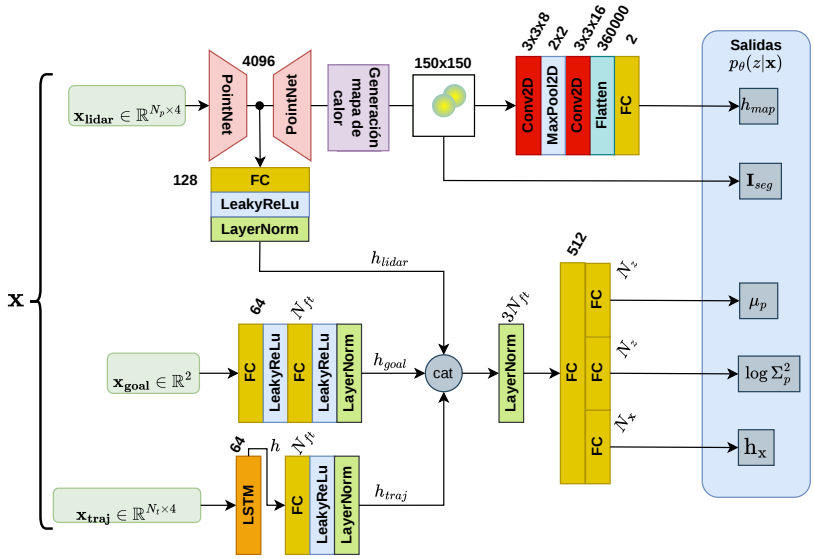


Figura 4.4: Arquitectura detallada del encoder *prior* $p_\theta(z|\mathbf{x})$. Las entradas corresponden a las observaciones sensoriales $\mathbf{x_{lidar}}, \mathbf{x_{traj}}$ y $\mathbf{x_{goal}}$. Las salidas son la representación condicional $\mathbf{h_x}$, los parámetros de la distribución *prior* (μ_p, Σ_p), el mapa de segmentación $\mathbf{I}_{seg}$ y sus características reducidas h_{map}.

Encoder LiDAR

Usa como backbone PointNet [24] pre entrenado como es descrito en la sección anterior. La entrada corresponde a la nube de puntos voxelizada $\mathbf{x_{lidar}} \in \mathbb{R}^{N_l \times N_p \times 4}$, con N_l la cantidad de observaciones anteriores, N_p la cantidad de puntos y la dimensión final corresponde a las coordenadas espaciales (x, y, z) y la intensidad de cada punto. Las entradas son normalizadas por una distancia máxima de medición $R_{\max}$ y las intensidades son estandarizadas para tener media 0 y desviación estándar 1. Las características extraídas por PointNet son pasadas por una capa FC y *LayerNorm*, siendo reducidas a un vector h_{lidar} de tamaño N_{ft}. Por otra parte, la segmentación de la nube es después rasterizada a una grilla 2D gaussiana $\mathbf{I}_{seg}$ de tamaño $H \times W$ y resolución δ (como se muestra en la Sección 4.2.3).

La representación generada es pasada por una capa convolucional para reducir su tamaño y condicionar al decoder, ya que este no puede recibir una imagen directamente, pero sí sus características h_{map}. Esta formulación es una adaptación de los *heatmaps* usados en [56], donde se predicen directamente *waypoints* candidatos. En este caso, la grilla presenta las zonas factibles de navegación según la nube de puntos.

Encoder objetivo

Corresponde a una *Multi Layer Perceptron* (MLP) [67] de dos capas que toma como entrada el objetivo $\mathbf{x_{goal}} \in \mathbb{R}^2$ en coordenadas polares respecto al sistema local del robot. El objetivo es normalizado por una distancia máxima d_{max} y un ángulo máximo $\theta_{max} = \pi$. Dado que este vector representa una instrucción estática y de baja dimensionalidad (coordenadas polares (ρ, θ)), no se requiere memoria temporal. La salida es pasada por una capa *LayerNorm*, obteniendo finalmente una representación h_{goal} de tamaño N_{ft}.

La elección de coordenadas polares facilita el aprendizaje, ya que desacopla la distancia al objetivo de la orientación necesaria para alcanzarlo, alineándose mejor con la cinemática de un robot diferencial.

Encoder trayectoria pasada

Es una red *Long Short Term Memory* (LSTM) [68] de una celda que recibe en paralelo la trayectoria anterior $\mathbf{x_{traj}} \in \mathbb{R}^{N_v \times 4}$ en posiciones y velocidades, en un horizonte de tiempo τ_{past} con N_v muestras. La entrada es normalizada por la velocidad lineal máxima v_{max} y las posiciones se mantienen en el sistema local del robot, sin normalización. Se usa como características el estado oculto de la celda, que se pasa por una capa *LayerNorm* para luego obtener una representación h_{traj} de tamaño N_{ft}.

Para este encoder se selecciona una LSTM sobre una *Recurrent Neural Network* (RNN) [69] estándar o una MLP debido a su capacidad para mitigar el problema del desvanecimiento del gradiente (*vanishing gradient*) en secuencias temporales. La LSTM puede aprender dependencias a largo plazo, permitiendo que se modele no solo la velocidad instantánea actual, sino también la tendencia de aceleración y la suavidad del movimiento previo, descartando

ruido de la odometría gracias a las compuertas de olvido.

Los vectores de características luego se concatenan para formar una representación intermedia $\mathbf{h}$

$$\mathbf{h} = [h_{lidar}; h_{goal}; h_{traj}] \in \mathbb{R}^{3 \cdot N_{ft}}, \tag{4.8}$$

donde N_{ft} es la dimensión de cada componente. El vector $\mathbf{h}$ es luego procesado por una capa *LayerNorm* y una capa *fully-connected*, obteniendo la representación condicional final $\mathbf{h_x}$ de tamaño $N_\mathbf{x}$, la cual resume toda la información relevante del estado actual del robot y su entorno. En una capa FC por separado, el resultado es utilizado para obtener los parámetros de media y log-varianza de la distribución *prior* p_θ del espacio latente ($\mu_\mathbf{p}$, $\log \mathbf{\Sigma_p}^2$) ambos en $\mathbb{R}^{N_z}$ con N_z la dimensión del espacio latente, para luego construir una muestra z_p. Durante el entrenamiento se muestrea de p_θ pero únicamente para calcular la función de pérdida. El uso de *LayerNorm* es crucial aquí para estabilizar el entrenamiento estocástico, evitando que la varianza de las características concatenadas desestabilice el espacio latente.

4.2.5. Encoder *posterior* $q_\phi(z|\mathbf{x}, \mathbf{y})$

Durante el entrenamiento, la trayectoria real $\mathbf{y} \in \mathbb{R}^{N_w \times 2}$ es procesada por una LSTM bidireccional [70] de una celda, cuyo estado oculto (h, c) corresponde a una reducción por capas FC de $\mathbf{h_x}$. En la Figura 4.5 se muestra la arquitectura detallada del encoder, junto con los tamaños de salida de cada capa.

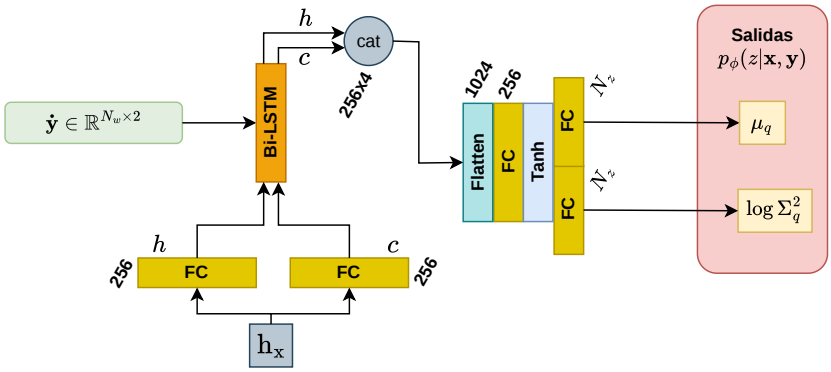


Figura 4.5: Arquitectura detallada del encoder *posterior* $q_\phi(z|\mathbf{x}, \mathbf{y})$. La entrada corresponde a la trayectoria futura real $\mathbf{y}$ y el contexto $\mathbf{h_x}$. Las salidas son los parámetros de la distribución *posterior* (μ_q, Σ_q).

Se usa para obtener los parámetros de la distribución *posterior* q_ϕ del espacio latente ($\mu_\mathbf{q}$, $\log \mathbf{\Sigma_q}^2$) de dimensión N_z, de manera análoga a la distribución *prior*, generando muestras z_q. A diferencia del historial pasado, la trayectoria futura es una curva geométrica completa disponible durante el entrenamiento. Una LSTM bidireccional procesa la secuencia de t a $t + N$ y viceversa, permitiendo que el estado latente z_q capture dependencias globales de la curva, generando una representación más robusta que la LSTM unidireccional del estado

pasado del robot. El contexto $\mathbf{h_x}$ se usa como segunda variable condicionadora dentro del estado oculto inicial de la Bi-LSTM, siendo pasado primero por capas lineales.

4.2.6. Decoder $p_\psi(\mathbf{y}|z,\mathbf{x})$

En la práctica, en lugar de predecir directamente las posiciones absolutas $\mathbf{y}$, la red modela una distribución sobre la secuencia de velocidades $\dot{\mathbf{y}} \in \mathbb{R}^{N_w \times 2}$, donde $\dot{\mathbf{y}} = \{\mathbf{v}_1, \ldots, \mathbf{v}_{N_w}\}$. Esta decisión de diseño se debe a que la predicción en el espacio de velocidades permite obtener trayectorias que son cinemáticamente más suaves y continuas, evitando los saltos espaciales bruscos que serían físicamente irrealizables para el robot. Este es un enfoque respaldado por arquitecturas como Trajectron++ [21].

Para recuperar la trayectoria $\hat{\mathbf{y}} = \{\mathbf{p}_1, \ldots, \mathbf{p}_{N_w}\}$, las velocidades predichas se integran iterativamente a partir de la posición actual del robot $\mathbf{p}_t = (0, 0)$, utilizando la relación:

$$\hat{\mathbf{p}}_{t+k} = \hat{\mathbf{p}}_{t+k-1} + \mathbf{v}_k \Delta t, \ \forall k \in \{1, \ldots, N_w\} \tag{4.9}$$

de este modo, se mantiene la coherencia entre el objetivo espacial del problema y la estabilidad cinemática de la red. En la Figura 4.6 se muestra en detalle el proceso autoregresivo de predicción de trayectorias tanto en entrenamiento como inferencia, descrito a continuación.

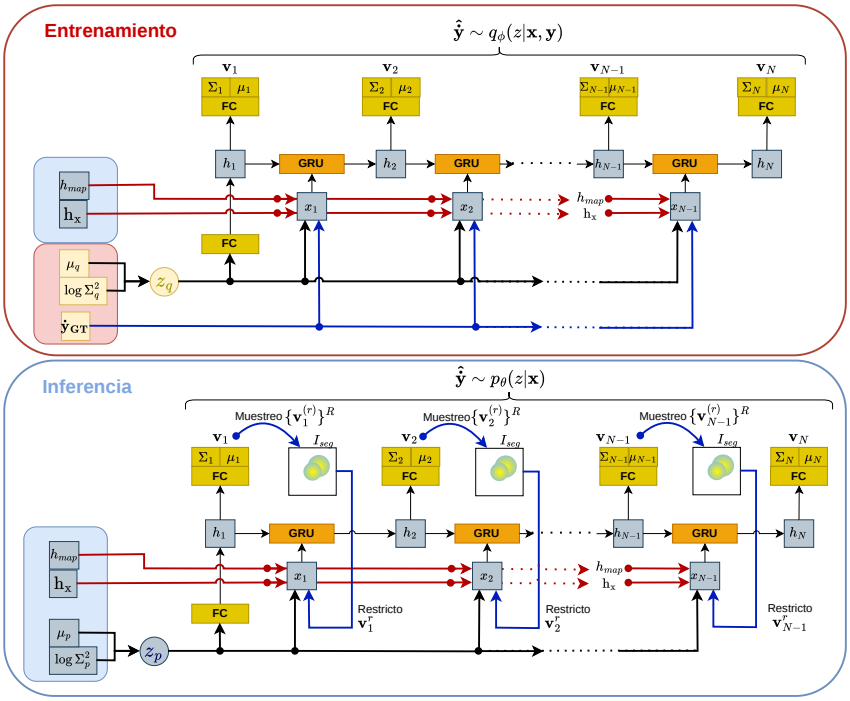


Figura 4.6: Arquitectura detallada del decoder $p_\psi(\mathbf{y}|z,\mathbf{x})$. Durante el entrenamiento se usa la trayectoria real para condicionar las predicciones mediante *teacher forcing*, mientras que en inferencia se muestrean R velocidades y se restringen con el mapa de segmentación $\mathbf{I}_{seg}$.

Las entradas del decoder corresponden a una muestra del espacio latente $z \in \mathbb{R}^{N_z}$ (muestreado de q_ϕ en entrenamiento $[z_q]$ y de p_θ en inferencia $[z_p]$), el vector de características $\mathbf{h_x}$ y las características h_{map} de $\mathbf{I}_{seg}$. El decoder corresponde a una GRU [71] de N_w celdas, donde en cada celda se concatena la entrada con la salida de la celda anterior y se realiza *teacher forcing* [72] para usar solamente las predicciones que caen en espacios factibles. Durante el entrenamiento, este proceso se lleva a cabo usando la trayectoria real $\mathbf{y}_{\mathrm{GT}}$ para condicionar las predicciones. En inferencia, se muestrean R velocidades (fijado en 1.000) en cada paso y se filtran según $\mathbf{I}_{seg}$, eligiendo aquella que se encuentre dentro de zonas positivas y su aceleración $\Delta\mathbf{v}_i$ sea menor a un límite (fijado en 0,5 $\mathrm{m/s^2}$).

La salida final es pasada por una capa *fully-connected* para obtener los parámetros de una distribución gaussiana $(\mu_{\dot{y}}, \mathbf{\Sigma}_{\dot{y}}) \in \mathbb{R}^{N_w \times 2}$, correspondiente a la media y desviación estándar de las velocidades en cada *waypoint*, separados por un tiempo Δt.

La elección de una GRU se sustenta en su menor cantidad de parámetros respecto a LSTM (carece de puerta de salida separada), lo que acelera la inferencia en tiempo real, crítico para la tarea a resolver. Por otro lado, dado que el contexto fuerte ya ha sido extraído por los encoders y comprimido en z, el decoder actúa principalmente como un generador autoregresivo. Empíricamente, las GRU demuestran un rendimiento comparable a las LSTM en tareas de generación de trayectorias, pero con menor costo computacional. Las GRU son usados en trabajos similares como [56] y [22].

4.2.7. Función de pérdida

Dado que la arquitectura propuesta se basa en un CVAE con *prior* aprendido, la función de pérdida se formula inicialmente de la manera descrita en la Sección 2.5.2, usando la divergencia KL entre los encoder *prior* y *posterior* y la verosimilitud de las predicciones usando el decoder $q_\phi(z|\mathbf{x}, \mathbf{y})$. Siguiendo el objetivo propuesto en [56], se agrega además la log-probabilidad negativa (NLL) del encoder *prior* $p_\theta(z|\mathbf{x})$, de manera que ambas distribuciones se alineen con la distribución real de las trayectorias

$$\mathcal{L}_{\mathrm{ELBO}} = -\mathbb{E}_{z_p}[\log(p_\psi(\mathbf{y}|z_p, \mathbf{x}))] - \mathbb{E}_{z_q}[\log(p_\psi(\mathbf{y}|z_q, \mathbf{x}))] + \beta D_{\mathrm{KL}}(p_\theta(z|\mathbf{x})||q_\phi(z|\mathbf{x}, \mathbf{y})). \quad (4.10)$$

La pérdida NLL mostrada anteriormente fuerza que las distribuciones resultantes de trayectorias $p_\psi(\mathbf{y}|z_p, \mathbf{x}), z_p \sim p_\theta(z|\mathbf{x})$ y $p_\psi(\mathbf{y}|z_q, \mathbf{x}), z_q \sim q_\phi(z|\mathbf{x}, \mathbf{y})$ se ajusten lo suficiente para ser capaces de producir muestras $\mathbf{y}$ provenientes del *ground truth*. Sin embargo, en el marco de este trabajo, la base de datos utilizado contiene múltiples trayectorias *ground truth* $\{\mathbf{y}^{(m)}\}_{m=1}^{M}$ (representando las direcciones factibles de movimiento), por lo que se desea poder forzar que la salida sea siempre lo más similar a al menos una de las referencias, en vez de al valor promedio de todas estas. Por esta razón, la NLL para el encoder *prior* se reformula como una suma sobre exponenciales logarítmicas (*log-mean-exp*). Notar que la exponencial desaparece al ser el inverso del logaritmo. De esta manera, la función de pérdida de reconstrucción para el encoder *prior* queda como

$$\log \frac{1}{M} \sum_{m=1}^{M} p_\psi(\mathbf{y}^{(m)}|z, \mathbf{x}). \tag{4.11}$$

Esta formulación es matemáticamente análoga a la utilizada en los Importance Weighted Autoencoders (IWAE) [73], donde se demuestra que mover la sumatoria dentro del logaritmo constituye una cota inferior más ajustada de la verosimilitud real. Operativamente, esto actúa como un mecanismo de "*Multiple Choice Learning*" o "*Winner-Takes-All*" suave: el gradiente de la pérdida es dominado por la muestra $\mathbf{y}^{(m)}$ que obtiene la mayor probabilidad bajo la distribución actual. Esto permite que el modelo se especialice en un modo específico de la distribución de futuros, sin ser penalizado por las otras variantes válidas.

Es importante mencionar que dicha modificación debe ser aplicada únicamente a las predicciones hechas con muestras z_p provenientes del decoder *prior* $p_\theta(z|\mathbf{x})$, ya que durante el entrenamiento el decoder *posterior* recibe una muestra de *ground truth* como entrada, y por lo tanto su log-verosimilitud debe ajustarse a esa muestra específica. De esta manera, la pérdida de reconstrucción queda como

$$\mathcal{L}_{recon} = -\log \frac{1}{M} \sum_{m=1}^{M} p_\psi(\mathbf{y}^{(m)}|z_p, \mathbf{x}) - \mathbb{E}_{z_q}[\log(p_\psi(\mathbf{y}|z, \mathbf{x}))]. \tag{4.12}$$

En la práctica se usan k muestras z_p de p_θ, de manera que el entrenamiento sea más estable ante perturbaciones de *ground truth* y predicciones.

Por otra parte, se agrega una función de pérdida adicional que penaliza las trayectorias que se encuentran en áreas no transitables. Para esto se muestrean velocidades a partir de las predicciones y se proyectan a un mapa local $\mathbf{M}_{sem}$ que contiene etiquetas reales de las áreas transitables y no transitables (véase la base de datos en la Sección 5.1). Para hacer suave la función, se usa el logaritmo sobre un kernel gaussiano $\mathbf{GK}(.)$ sobre el mapa $\mathbf{M}_{\text{sem}}$, con (u_x, v_y) las coordenadas de cada trayectoria $\mathbf{y}$ proyectadas en los mapas locales usando la resolución y origen del mapa. De esta manera, la función de pérdida de colisiones $\mathcal{L}$ queda como

$$\mathcal{L}_{\text{col}} = \log \left(\sum_{(p_x, p_y) \in \mathbf{y}'} \mathbf{GK}(\mathbf{M}_{\text{sem}}(p_x, p_y)) \right). \tag{4.13}$$

La función de pérdida total es la suma de las pérdidas $\mathcal{L}_{\text{NaVAE}} = \beta D_{\text{KL}} + \mathcal{L}_{\text{recon}} + \lambda \mathcal{L}_{\text{col}}$, con β, λ factores regularizadores.

4.3. Selección de trayectorias

El objetivo de esta sección es describir la metodología utilizada para seleccionar la mejor trayectoria $\mathbf{y}^* \in \mathcal{Y}$ de las K generadas en el paso anterior. La Figura 4.7 muestra una

vista general del proceso, el cual se divide en (1) filtrar las trayectorias que no cumplan con restricciones cinemáticas del robot (*Filtrado de colisiones*), (2) generar un mapa de costos semántico en base a un modelo de visión lenguaje (*Construcción de mapa de costos con VLM*) (3) elegir la trayectoria que minimice un costo semántico y geométrico (*Cálculo de costos por trayectoria*), y finalmente (4) actualización de las trayectorias en base a la comparación entre la trayectoria en ejecución y la nueva candidata óptima (*Actualización de costos y trayectorias*). Adicionalmente, se describe un mecanismo de recuperación (*recovery behavior*) en caso de que se detecte una falla en la navegación.

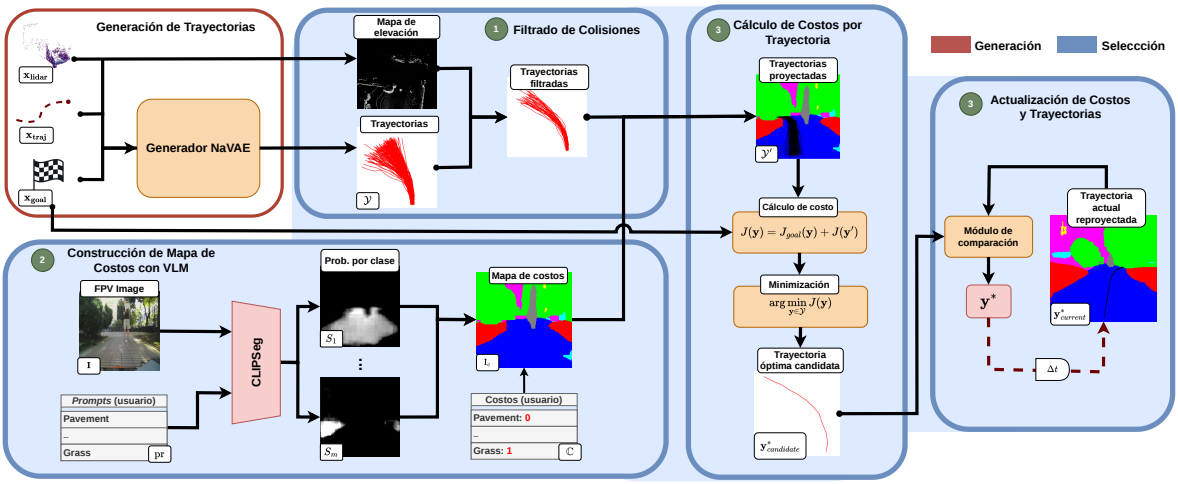


Figura 4.7: Diagrama general del sistema de selección de trayectorias. Se muestra de manera secuencial de izquierda a derecha el proceso de filtrado de colisiones, construcción de mapa de costos con VLM, cálculo de costos por trayectoria y actualización de costos y trayectorias.

4.3.1. Filtrado de colisiones

Es posible que algunas de las trayectorias generadas por el módulo CVAE no sean factibles, ya sea por restricciones cinemáticas específicas de la plataforma robótica o porque pasen por zonas de colisión. Para esto, se utiliza un filtro de colisiones que descarta todas las trayectorias que no cumplan con estas restricciones. El filtro utiliza la nube de puntos del LIDAR para construir un mapa de elevación local en base a [18] y verificar si alguna trayectoria intersecta con una zona de colisión, utilizando el *footprint* del robot para dicho cálculo.

El mapa de elevación tiene un tamaño de $S_x \times S_y$ m^2 y una resolución de r_{elv} $m/celda$. El mapa está desplzado en $S_x/2$ m hacia adelante, debido a que las trayectorias se generan en un campo de visión (FOV) positivo en el eje x del robot, por lo que la información detrás de este no es relevante.

Dado que el mapa de elevación tiene espacios sin información por la forma en que los escaneos del LiDAR son proyectados, se considera como colisiones los *waypoints* de las trayectorias que tienen un cambio de elevación mayor a θ_c. Esto también ayuda en terrenos con pendientes leves, donde el mapa de elevación puede no reflejar correctamente la transitabilidad del terreno.

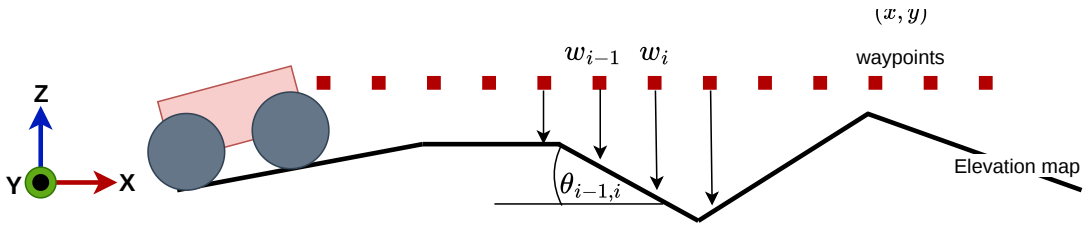


Figura 4.8: Diagrama x-z del robot y la proyección de los *waypoints* al mapa de elevación.

La Figura 4.8 muestra un diagrama del robot y la proyección de los *waypoints* al mapa de elevación. Si se considera un *waypoint* w_i con coordenadas (x_i, y_i, z_i) y su *waypoint* anterior $(x_{i-1}, y_{i-1}, z_{i-1})$, se definen las diferencias $\Delta x_{i,i-1} = x_i - x_{i-1}$, $\Delta y_{i,i-1} = y_i - y_{i-1}$ y $\Delta z_{i,i-1} = z_i - z_{i-1}$. El ángulo de inclinación entre ambos *waypoints* se calcula según la Ec. 4.14. Si este valor es mayor a θ_c, se considera que el *waypoint* i está en colisión. En vez de hacer uso del valor (x_i, y_i) exactos en el mapa de elevación, se usa el valor máximo alrededor de un área cuadrada de lado d_{foot}, correspondiente al *footprint* del robot.

$$\theta_c = \text{atan}\left(\frac{\Delta z_{i,i-1}}{\sqrt{(\Delta x_{i,i-1})^2 + (\Delta y_{i,i-1})^2}}\right) \tag{4.14}$$

4.3.2. Construcción de mapa de costos con VLM

Una vez obtenidas las trayectorias dinámicamente factibles, se debe seleccionar la que cumpla de mejor manera con las restricciones impuestas por el usuario. Para esto, se utiliza CLIPSeg [32], que permite obtener mapas de probabilidades por píxel de que un objeto o región esté presente en una imagen, en base a una consulta en lenguaje natural.

Si se considera una imagen de entrada en primera persona (FPV) $\mathbf{I}$ y un conjunto de restricciones de vocabulario abierto $\mathbf{pr} = \{pr_1, pr_2, \ldots, pr_m\}$, el modelo de lenguaje visual (VLM) entrega un conjunto de mapas de probabilidad $\mathbf{S} = \{S_1, S_2, \ldots, S_m\}$, donde cada mapa S_i corresponde a la probabilidad de que la restricción pr_i esté presente en cada píxel de la imagen $\mathbf{I}$. Estas restricciones pueden referirse a clases simples posibles de encontrar en un ambiente de exterior, como caminos, personas, escaleras, etc., las cuales son intercambiables dependiendo de la tarea de navegación.

Para obtener el mapa de costos se le asigna un valor arbitrario a cada clase presente $\mathbf{pr}$, $\mathbb{C} = \{c_1, c_2, \ldots, c_m\}$, la cual es después asignada a un mapa resultante del máximo valor entre todos los mapas de probabilidad, es decir, $\mathbf{I}_c = \mathbb{C}[\arg\max_i(\mathbf{S_i})]$, donde $i = 1, 2, \ldots, m$ e $\mathbf{I}_c$ es el mapa de costos resultante.

Tanto para asignar los valores de $\mathbb{C}$ como para el cálculo de métricas de navegación, se categoriza en tres niveles de transitabilidad, asignando a cada clase un costo de navegación específico:

- **No-transitable estricto**: Corresponde a clases semánticas que representan obstáculos

físicos infranqueables o que provocarían daños al robot (p. ej. paredes, árboles, escaleras). A estas zonas se les asignan costos comparativamente altos a las otras clases.

- **Transitable suave**: Incluye áreas que son físicamente posibles de navegar sin causar colisiones, pero que podrían no ser categorizadas por el usuario como óptimas (p. ej. césped, tierra suelta). A estas clases se les asigna un costo penalizado mayor a cero ($c > 0$), pero menor a las estrictas.
- **Transitable preferido**: Representa las superficies ideales y seguras para la navegación según la preferencia del usuario (p. ej. veredas, caminos pavimentados). Estas zonas constituyen el área transitable óptima, y se les asigna un costo igual a cero ($c = 0$).

4.3.3. Cálculo de costos por trayectoria

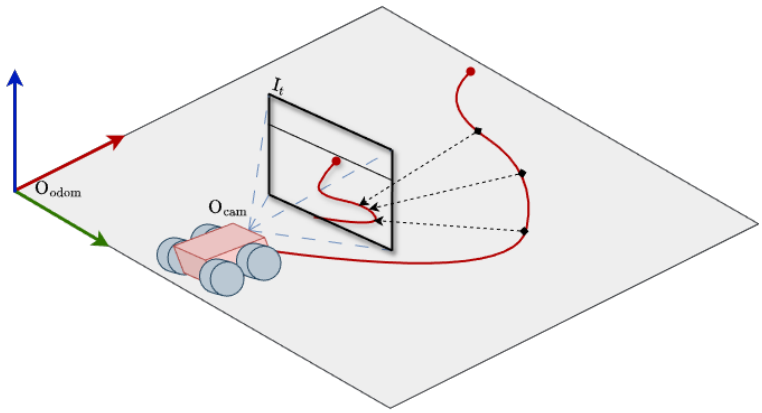


Figura 4.9: Ilustración de un robot en un sistema $\mathcal{O}_{odom}$, que proyecta una trayectoria en el sistema $\mathcal{O}_{robot}$ al plano de la imagen $\mathbf{I}'$.

Las trayectorias $\mathcal{Y}$ se generan en el sistema de referencia local del robot (véase Figura 4.9). Si se considera una de estas trayectorias $\mathbf{y}$, se puede transformar al sistema de referencia de la cámara $\mathcal{O}_{cam}$ mediante la matriz de transformación homogénea $\mathbf{T_{r-c}}$ (robot-cámara), definida por la geometría del robot. El resultado de esta transformación es la trayectoria $\mathbf{y}_c = \mathbf{T_{r-c}}\mathbf{y}$ desde el sistema $\mathcal{O}_{cam}$. A continuación, utilizando la matriz intrínseca de la cámara K, las trayectorias se pueden proyectar al plano de la imagen según la Ecuación 4.15, que corresponde al modelo *pinhole* de una cámara en coordenadas homogéneas

$$\mathbf{y}' = K[I_3|0]\mathbf{y}_c, \tag{4.15}$$

donde I_3 representa a la matriz identidad 3×3. Luego, $\mathbf{y}'$ contiene las coordenadas en píxeles $(x, y) \rightarrow (p_x, p_y)$ en el espacio de la imagen, específicamente en el mapa de costos. Finalmente, para obtener la mejor trayectoria se calcula el costo semántico de cada trayectoria como la suma descontada de los costos en cada *waypoint* proyectado al mapa de costos. Es decir,

$$J(\mathbf{y}') = \sum_{j=1}^{N} \gamma^j \mathbf{I}_c(p_x^j, p_y^j) \quad \forall (p_x^j, p_y^j) \in \mathbf{y}', \tag{4.16}$$

donde $\mathbf{I}_c(p_x^j, p_y^j)$ es el valor del mapa de costos del *waypoint* proyectado (p_x^j, p_y^j), $N = |\mathbf{y}'|$ y $\gamma \in [0, 1]$ es un factor de descuento que penaliza los costos en *waypoints* más alejados, ya que $J(\mathbf{y}')$ es una suma de costos futuros.

Un punto importante a considerar es que al pasar del espacio 3D a 2D existe pérdida de información (profundidad) y puede ocurrir que haya oclusiones de las trayectorias con objetos que estén delante de *waypoints* futuros (p. ej. una trayectoria que rodea un árbol en frente del robot). Teniendo en cuenta que las trayectorias con colisiones son filtradas en el paso anterior, se asigna un costo diferenciado intermedio C_u de oclusión a dichos *waypoints*, debido a que no se tiene certeza de su costo real

$$J(\mathbf{y}') = \sum_{j=1}^{N} \gamma^j [\mathbf{I}_c(p_x^j, p_y^j) \mathbb{1}_{u_j=0} + C_u \mathbb{1}_{u_j=1}] \quad \forall (p_x^j, p_y^j) \in \mathbf{y}', \tag{4.17}$$

con u_j correspondiente a una máscara binaria que indica si el *waypoint* j está ocluido o no. Para detectar si un *waypoint* en el espacio de la imagen está ocluido o no, se compara su valor semántico con un umbral T_{occ}, el cual es el valor mínimo asignado a las clases consideradas como no transitables estrictas. Es decir, sólo se consideran las clases no transitables estrictas para determinar las oclusiones. Si $\mathbf{I}_c(p_x, p_y) > T_{occ}$, entonces $u_j = 1$.

Por último, para fomentar que se seleccionen trayectorias que acerquen al robot al objetivo global, tanto en posición como orientación, se agrega un término de costo J_{goal}

$$J_{goal}(\mathbf{y}) = \alpha_1 \cdot \log(1 + d(\mathbf{y}, goal)) + \alpha_2 \cdot \frac{|\theta_{w_N - goal}|}{\pi}, \tag{4.18}$$

con α_1 y α_2 factores de escala, $d(\mathbf{y}, goal)$ la distancia euclidiana entre el último *waypoint* de la trayectoria $\mathbf{y}$ y el objetivo y $\theta_{w_N - goal}$ el ángulo entre el objetivo y la orientación final del robot en la trayectoria. Finalmente, la selección está dada por la minimización de la suma de ambos costos

$$\mathbf{y}^* = \arg\min_{\mathbf{y} \in \mathcal{Y}} J(\mathbf{y}') + J_{goal}(\mathbf{y}). \tag{4.19}$$

4.3.4. Actualización de costos y trayectorias

Para la actualización de costos y trayectorias es necesario tener en cuenta dos puntos: el primero es cuándo realizar una actualización y la segunda es cómo comparar los costos. Estos puntos se relacionan con la naturaleza de las trayectorias y el método para obtener costos, ya que actualizar periódicamente para ejecutar solo el primer *waypoint* puede resultar en comportamientos subóptimos si las trayectorias tienen mayoritariamente secciones iniciales rectas hacia adelante. Por otra parte, los costos calculados con oclusiones pueden dejar de serlo a medida que se avanza.

En la Figura 4.10 se muestran dos fotogramas simulados al ejecutar una trayectoria que fue seleccionada en un instante de tiempo t_n. Dado que las trayectorias proyectadas pueden

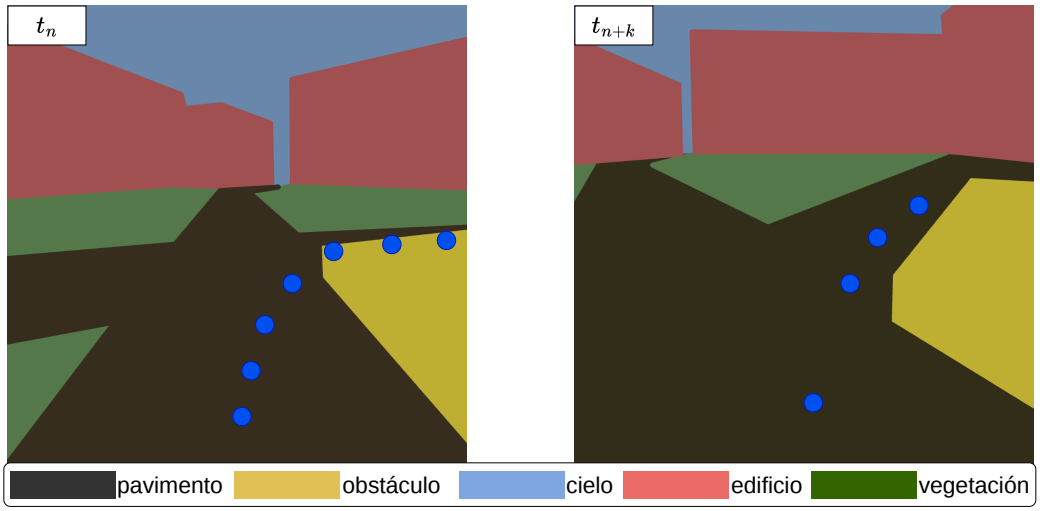


Figura 4.10: Referencia de una trayectoria seleccionada en un mapa de costos con clases arbitrarias. A la izquierda el tiempo en que se selecciona, donde algunos *waypoints* están ocluidos. A la derecha en k pasos más adelante, donde se pueden recalcular los costos de las oclusiones.

presentar oclusiones en el momento que se seleccionan, en vez de mantener constante sus costos durante la ejecución, se vuelven a calcular en cada nueva segmentación recibida de CLIPSeg. Esto se hace congelando los costos por *waypoint* $\{c_f^1, \ldots, c_f^m\}$ hasta t_{n+k}, con k el *waypoint* actual, y actualizando solo los siguientes en base a la reproyección de la trayectoria a medida que se van superando los *waypoints*

$$J(\mathbf{y}^*) = \sum_{j=1}^{k} c_f^j + \sum_{j=k}^{N} \gamma^j [\mathbf{I}_c(p_x^j, p_y^j)\mathbb{1}_{u_j=0} + C_u \mathbb{1}_{u_j=1}] \quad \forall (p_x^j, p_y^j) \in \mathbf{y}^*. \tag{4.20}$$

Lo anterior es necesario ya que, para comparar la trayectoria actual con las nuevas estas deben tener el mismo largo, y mientras más cerca se esté de un *waypoint*, más fidedigno es el valor de su costo semántico. Luego, al obtener un nuevo conjunto de trayectorias $\mathcal{Y}$, se compara el costo de la mejor de estas ($\mathbf{y_{new}}$) con el costo de la trayectoria actual ($\mathbf{y_{curr}}$), y si esta es mejor en un factor de histéresis $\varepsilon > 0$, se actualiza

$$J(\mathbf{y_{new}}) < J(\mathbf{y_{curr}}) - \varepsilon \tag{4.21}$$

En el caso que se ejecute por completo una trayectoria, al llegar al último *waypoint* se reinician los costos y se selecciona la mejor de las nuevas recibidas.

Es importante mencionar que cada paso se ejecuta en un bucle por separado con su propia frecuencia. La generación de trayectorias se realiza a una frecuencia f_{gen}, mientras que la actualización de costos está dada por la frecuencia a la que se crean los mapas de costos f_{clip} (dado por el tiempo de inferencia de CLIP).

A diferencia de los planificadores clásicos que suelen recalcular rutas completas de manera

síncrona, esta propuesta desacopla la frecuencia de generación (f_{gen}) de la frecuencia de actualización semántica (f_{clip}). Esto constituye una ventaja significativa, ya que permite reevaluar la viabilidad de la ruta a la alta frecuencia de la percepción visual, corrigiendo errores de interpretación (p. ej., una sombra que parecía obstáculo o una oclusión) sin la sobrecarga computacional de generar nuevas geometrías constantemente. Este diseño híbrido otorga al sistema la robustez necesaria para operar en entornos con oclusiones dinámicas, asegurando que las decisiones de navegación se basen siempre en la evidencia visual más reciente disponible.

4.3.5. *Recovery behavior*

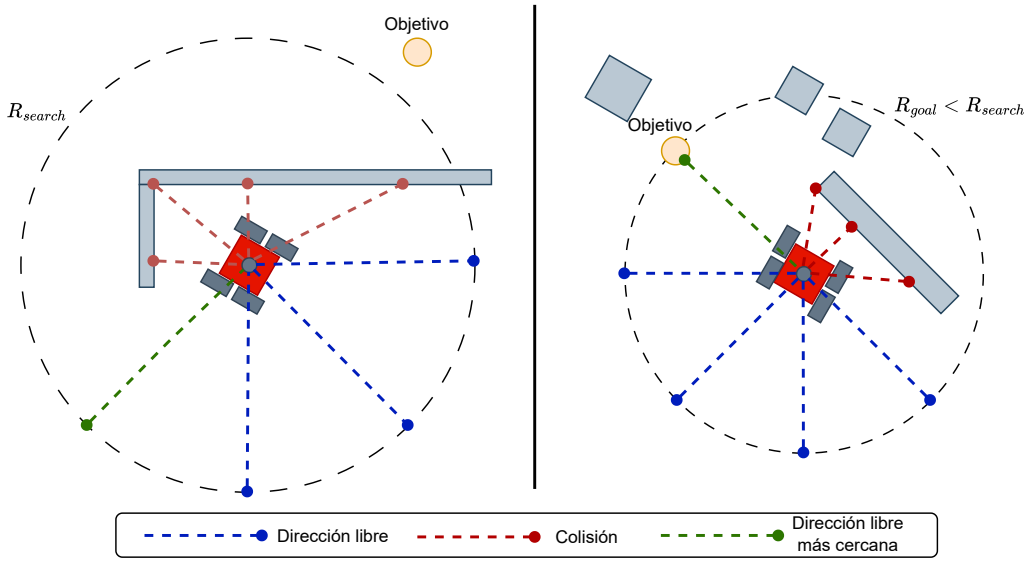


Figura 4.11: Diagrama de búsqueda de direcciones libres. A la izquierda un caso en que el objetivo está más lejos que el radio de búsqueda, a la derecha el caso contrario. En ambos casos, se busca la dirección de giro libre que minimice la distancia angular al objetivo.

Dado que el sistema no necesariamente propone trayectorias globalmente óptimas (al no contar con información global más que la pose en un sistema de referencia), existen casos en que el robot puede quedar "atrapado" en mínimos locales. En dichos casos, ninguna trayectoria generada es factible y por ende no es posible actualizar a una nueva. Para abordar estos casos específicos, se propone un *recovery behavior* simple, que rota al robot en la dirección libre más cercana al objetivo en caso necesario. La Figura 4.11 muestra un diagrama de cómo se obtiene dicha dirección en base al mapa de elevación.

Se busca en un radio R_{search} a menos que el robot se encuentre a una distancia menor a aquel radio, y en ese caso, $R_{search} = R_{goal}$. Se busca en N direcciones usando el mapa de elevación local, haciendo *raycasting* con búsqueda binaria para mayor eficiencia. Se elige la dirección de giro que deje al robot con un campo de visión libre pero angularmente más cercano al objetivo global.

El algoritmo se activa si el robot se encuentra "atorado" por más de τ_{stuck} ciclos de

planificación consecutivos. Es decir, si no se encuentran trayectorias factibles (sin colisiones) por τ_{stuck} ciclos. Una vez que se encuentra una dirección libre, se reinicia el contador.

Es importante mencionar que este método de recuperación es una heurística que puede no ser suficiente en todos los casos e incluso podría ocurrir que el robot se oriente en una dirección menos óptima. Sin embargo, sigue siendo un apoyo en la mayoría de los casos.

En los Algoritmos 1, 2 y 3 se resume el funcionamiento completo del sistema de navegación global y local conjunto[1].

Algoritmo 1 Bucle de Planificación Global

Initialize: $\mathbf{y}_{current} \leftarrow$ initial trajectory
$J_{current} \leftarrow$ UpdateCost($\mathbf{y}_{current}$) ▷ Actualización de costo trayectoria actual, Sec. 4.3.4
$t_{stuck} \leftarrow 0$
while planning is active **do**
  $\mathcal{Y}_{candidates} \leftarrow$ GenerateNewTrajectories(M, current_pose) ▷ Inferencia de NaVAE, Sec. 4.2
  $\mathcal{R}_{results} \leftarrow \emptyset$
  for each trajectory $\mathbf{y}_i \in \mathcal{Y}_{candidates}$ **do**
    EvaluateTrajectory($\mathbf{y}_i$) ▷ Véase Alg. 2
  end for
  $\mathcal{R}_{results} \leftarrow$ Collect all results (cost, validity, masks)
  $\mathcal{Y}_{valid} \leftarrow$ Filter for valid (non-collision) results from $\mathcal{R}_{results}$ ▷ Filtrado de trayectorias, Sec. 4.3.1
  if $|\mathcal{Y}_{valid}| = 0$ **then**
    $t_{stuck} \leftarrow t_{stuck} + 1$
    if $t_{stuck} > \tau_{stuck}$ **then**
      RecoveryBehavior ▷ Véase Alg. 3
    end if
    continue ▷ Repetir o activar recovery, Sec. 4.3.5
  end if
  $(\mathbf{y}_{candidate}, J_{candidate}) \leftarrow$ FindBestTrajectory($\mathcal{Y}_{valid}$) ▷ Bloques de proyección y cálculo de costos, Sec. 4.3.3
  $J_{current} \leftarrow$ UpdateCost($\mathbf{y}_{current}$)
  if $J_{candidate} < J_{current} - \varepsilon$ **then** ▷ Bloque de actualización asíncrona, Sec. 4.3.4
    $\mathbf{y}_{current} \leftarrow \mathbf{y}_{candidate}$
    $J_{current} \leftarrow J_{candidate}$
    UpdateLocalPlannerGoal($\mathbf{y}_{current}$) ▷ Enviar nueva trayectoria al planificador local
  end if
  if IsGoalReached($\mathbf{y}_{current}$) **then** ▷ Cancelar trayectorias al llegar al objetivo
    CancelLocalPlanner
    break
  end if
end while

Algoritmo 2 Hilo de Evaluación de Trayectorias

procedure EvaluateTrajectory($\mathbf{y}$)
  if CheckCollision($\mathbf{y}$, elevation_map) **then**
    return
  end if
  ▷ Cálculo de costos
  $U_{mask} \leftarrow$ GetOcclusionMask($\mathbf{y}$)
  $J_{cost} \leftarrow$ CalculateTrajectoryCost($\mathbf{y}, U_{mask}$, costmap)
  return (valid $\leftarrow$ **true**, cost $\leftarrow J_{cost}$, mask $\leftarrow U_{mask}$)
end procedure

[1]**Nota**: el algoritmo es solo de referencia. En la práctica se implementa en ROS con cada bucle relevante por separado con su respectiva frecuencia de actualización y el cálculo de los costos es realizado en paralelo, para mayor eficiencia.

Algoritmo 3 Recovery behavior

```
1: procedure RecoveryBehavior
2:     Y_candidates ← GenerateNewTrajectories(M, current_pose)
3:     while |Y_candidates| = 0 do
4:         u_dir ← GetNearestFreeDirection(current_pose, goal)
5:         RotateRobot(u_dir)
6:         Y_candidates ← GenerateNewTrajectories(M, current_pose)
7:     end while
8:     t_stuck = 0
9: end procedure
```

4.4. Modificación *baseline* MTG

4.4.1. Arquitectura

Con el fin de probar la arquitectura de MTG (véase Sección 2.6) en la misma tarea de navegación presentada en este trabajo, se realizan algunas modificaciones a la arquitectura original. Las modificaciones se relizan debido a que el modelo base entrega trayectorias con poca variabilidad incluso para observaciones distintas, como se verá en la sección de resultados.

La Figura 4.12 muestra el diagrama de la arquitectura del modelo modificado. Se agrega PointNet pre entrenado como se formula en la Sección 4.2.3, tal que se pueda capturar una mejor representación de la geometría del ambiente y evitar que se "ignore" el contexto (problema conocido como *posterior collapse* [74]). El objetivo no es cubrir todas las zonas navegables, sino más bien direccionar la generación de trayectorias hacia el objetivo. Por esta razón, se agrega un encoder adicional que procesa la posición relativa del objetivo en coordenadas polares $(d_{goal}, \theta_{goal})$, donde d_{goal} es la distancia Euclidiana entre la posición actual del robot y el objetivo, de la misma manera que en NaVAE.

El decoder usado es una GRU que se usa en modo autoregresivo, donde en cada paso se le entrega como entrada la velocidad generada en el paso anterior, junto con el estado oculto de la GRU. La salida en cada paso corresponde a la predicción de las diferencias de posiciones en cada *waypoint*. El decoder es determinístico, por lo que las trayectorias predichas son fijas en número, determinada por la cantidad k de transformaciones afines aplicadas al espacio latente.

Al igual que en NaVAE, el valor condicional para el encoder corresponde a una representación compacta de las observaciones sensoriales $\mathbf{h_x}$ de dimensión N_{ft}, El espacio latente tiene una dimensión de 512 y el resto de los hiperparámetros se mantienen iguales al trabajo original presentado en [22].

4.4.2. Función de Pérdida y Entrenamiento

Para el entrenamiento del modelo base, se adopta la función de objetivo original propuesta en la arquitectura MTG (detallada en la Sección 2.6), pero con una modificación crítica en el término de reconstrucción.

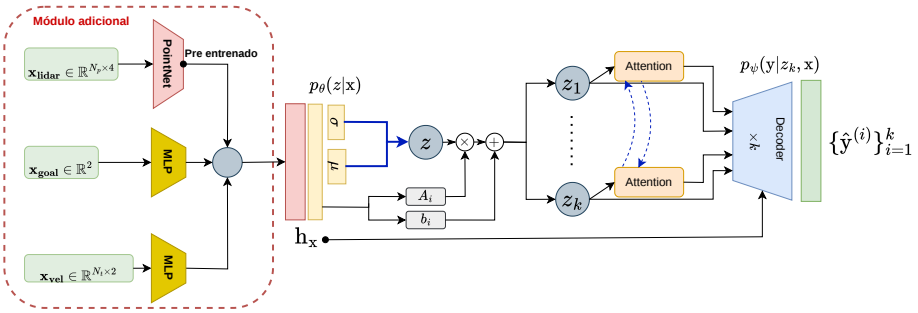


Figura 4.12: Diagrama de la arquitectura MTG modificada (MTG') para la tarea de navegación. Se agregan módulos para procesar la nube de puntos y la posición del objetivo global de navegación (encerrado en linea punteada roja).

Originalmente, MTG emplea la distancia de Hausdorff para alinear las curvas. Sin embargo, en este trabajo a modo de comparación, se propone la sustitución de este término por la Distancia de Chamfer ($\mathcal{L}_{CD}$). Esta métrica es computacionalmente más eficiente y numéricamente más estable para conjuntos de puntos desordenados o con correspondencias no lineales.

Matemáticamente, si definimos la trayectoria real como un conjunto de puntos $\mathbf{y} = \{p_1, \ldots, p_N\}$ y la trayectoria generada como $\hat{\mathbf{y}} = \{\hat{p}_1, \ldots, \hat{p}_N\}$, donde $|\hat{\mathbf{y}}| = |\mathbf{y}| = N$, la pérdida de reconstrucción Chamfer busca minimizar la distancia promedio desde cada punto de la predicción a su vecino más cercano en la realidad, y viceversa

$$\mathcal{L}_{CD}(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{N} \sum_{p \in \mathbf{y}} \min_{\hat{p} \in \hat{\mathbf{y}}} ||p - \hat{p}||_2^2 + \frac{1}{N} \sum_{\hat{p} \in \hat{\mathbf{y}}} \min_{p \in \mathbf{y}} ||\hat{p} - p||_2^2. \tag{4.22}$$

Esta formulación asegura dos propiedades geométricas deseables:

1. **Precisión (Forward):** El primer término garantiza que cada punto real sea "cubierto" por algún punto generado.
2. **Cobertura (Backward):** El segundo término penaliza los puntos generados que se alejan demasiado de la trayectoria real (alucinaciones o desvíos).

Finalmente, la función de pérdida total integra esta reconstrucción junto con la divergencia KL para la regularización del espacio latente y las restricciones de diversidad y transitabilidad, manteniendo los hiperparámetros de ponderación originales de MTG (disponibles en [22]). El entrenamiento se ejecuta utilizando el mismo conjunto de datos multimodal de NaVAE, configurando el modelo para generar $K = 6$ predicciones simultáneas por cada instancia de inferencia. La función de pérdida total queda entonces formulada como

$$\mathcal{L}_{MTG'} = \beta_1 D_{\text{KL}} + \beta_2 \mathcal{L}_{CD} + \lambda_{div} \mathcal{L}_{div} + \lambda_{trav} \mathcal{L}_{trav}, \tag{4.23}$$

donde los ponderadores β_1, λ_{div} y λ_{trav} se mantienen iguales a los valores propuestos en el trabajo original de MTG.

Capítulo 5

Resultados

La presente sección describe los resultados obtenidos tras la implementación y evaluación del módulo de navegación global propuesto. En primer lugar, se presenta la base de datos usada para el entrenamiento y evaluación del modelo generativo. Luego, se muestran los resultados del pre entrenamiento del modelo PointNet para segmentación semántica. Posteriormente, se presentan los resultados del entrenamiento y evaluación del modelo generativo NaVAE. Finalmente, se describen las pruebas y métricas utilizadas para evaluar la navegación global en el mundo real, junto con los resultados cuantitativos y cualitativos obtenidos.

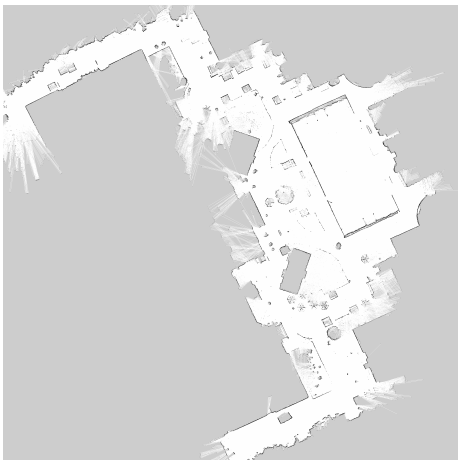
5.1. Base de datos

Para el entrenamiento del modelo generativo es necesario contar con una base de datos de interacciones en el mundo real, junto con mapas que permitan calcular métricas y las funciones de pérdida descritas en la Sección 4.2. Siguiendo el formato de la base de datos GND, presentada en [75], se utilizan trayectorias del mundo real en ambientes de exterior. Dichas trayectorias son recolectadas teleoperando una base móvil Husarion Panther [76], equipado con un LiDAR Ouster de 64 canales y una cámara USB de bajo costo. Además, el robot cuenta con una IMU y un GPS RTK (ver Sección 5.4.1 para más detalles del *setup* experimental).

El ambiente usado para obtener los datos corresponde a la Facultad de Ciencias Físicas y Matemáticas de la Universidad de Chile[1], el cual es un ambiente urbano con caminos peatonales, áreas verdes y múltiples cambios de altura. El robot se teleopera en dos zonas distintas, las cuales se muestran en la Fig. 5.1. Los mapas son construidos a partir de imágenes satelitales y un mapa de ocupación hecho con un algoritmo estándar de SLAM 2D [77]. Se distingue entre sector A (derecha) y sector B (izquierda). Los colores indican clase: edificio, camino, vegetación, obstáculos y escaleras.

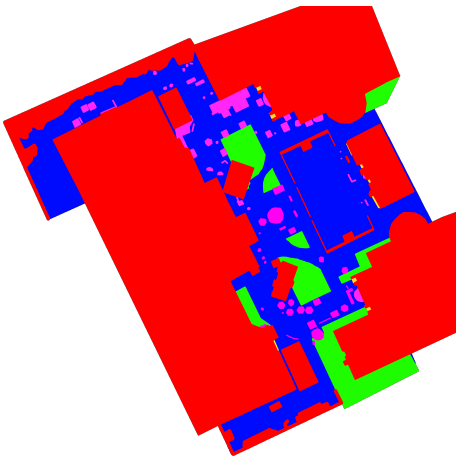
[1] Av. Beauchef 850,Santiago, Chile

(a) Imagen satelital

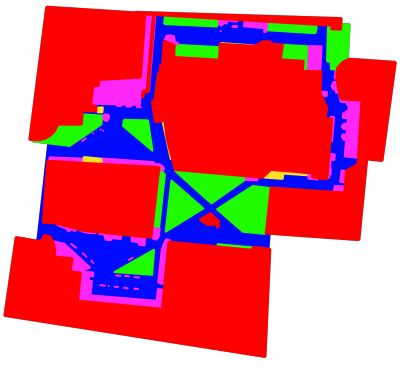


(b) AMCL sector B

(c) AMCL sector A



(d) Semántico sector B



(e) Semántico sector A

Figura 5.1: Foto satelital de la FCFM, junto con los mapas de ocupación 2D generados con SLAM y los mapas semánticos resultantes. Foto satelital extraída de `www.google.com/maps`.

Un punto importante a considerar es que se quiere que cada muestra de la base de datos contenga más de una trayectoria futura posible, idealmente una por cada dirección factible dada la pose del robot en una muestra dada. Para esto, se usa la funcionalidad de Rosbag[2] para recrear las trayectorias del robot en el mapa semántico. Usando una distancia máxima de búsqueda $R_{max} = 15m$ se encuentran todos los puntos alrededor del robot que sea posible alcanzar, considerando su footprint. Luego, se llama al planificador global base NavFN de ROS[3] para obtener los *waypoints* entre los objetivos y la pose actual del robot. Finalmente, se guardan todas las trayectorias obtenidas y se eliminan aquellas que tengan un FDE (*Final Displacement Error*) [78] menor a 0,5 entre ellas. Este proceso se repite para todas las muestras de la base de datos.

En total se cuenta con 15 trayectorias completas del sector A, de las cuales cada una se divide en muestras de 0,5 s, obteniendo un total de 1.706 muestras. La Tabla 5.1 presenta la información de cada muestra de la base de datos.

Tabla 5.1: Desglose de la información contenida en cada muestra temporal de la base de datos (ventana de 0,5 s).

Tipo de Dato	**Resolución / Formato**	**Dimensión por muestra**
Datos Sensoriales y de Contexto		
Imagen FPV RGB	1280×720 px	1 fotograma
Nube de puntos LiDAR	$\approx$ 300,000 puntos/scan	3 escaneos
IMU (9 ejes)	Vector $\in \mathbb{R}^9$	3 lecturas
Pose actual	(x, y, θ) global	1 estado
Mapa semántico local	Grid 150×150 px (15×15 m)	1 mapa
Secuencias de Trayectoria (Estados: x, y, v_x, v_y***)***		
Trayectoria pasada	Horizonte de 5 s	10 pasos
Trayectoria futura	Horizonte de 20 s	40 pasos
Trayectorias factibles	6 opciones de 20 s	6×40 pasos

Es importante mencionar que la base de datos es generada únicamente para entrenar el modelo generador presentado en las secciones siguientes, así como su evaluación. Durante la navegación, el sistema **no tiene acceso** a los mapas semánticos, ni a las trayectorias presentadas anteriormente.

5.2. Pre-entrenamiento de PointNet

Para el entrenamiento del modelo de segmentación semántica PointNet se usa la totalidad de la base de datos descrita en la Sección 5.1. El entrenamiento se realiza en una NVIDIA RTX 4070 (al igual que el resto de los modelos presentados), con un *batch size* de 24 muestras y un *learning rate* de 0,001. El modelo se entrena por 40 épocas, tomando alrededor de 10 minutos en total. La Figura 5.2 muestra las curvas de entrenamiento y validación del modelo.

[2] `https://wiki.ros.org/rosbag`
[3] `https://wiki.ros.org/navfn`

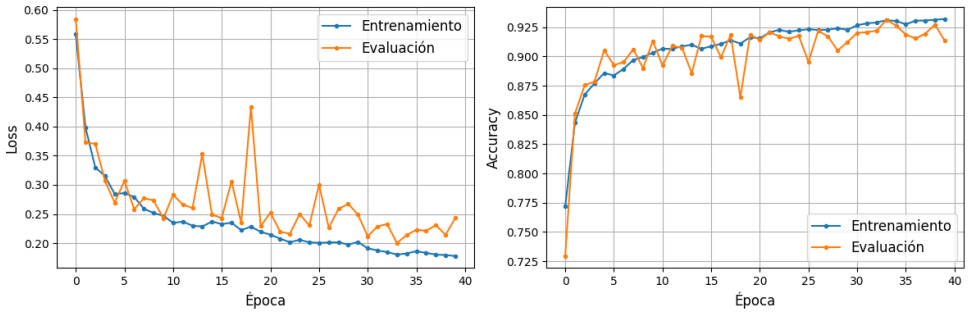


Figura 5.2: Curvas de entrenamiento de *Loss* y *Accuracy* del modelo PointNet.

Se presenta además en la Tabla 5.2 las principales métricas (*Accuracy* e IoU) del mejor modelo en evaluación.

Tabla 5.2: Resultados del mejor modelo de PointNet en evaluación.

meanIoU	**IoU transitable**	**IoU no transitable**	**Acc. Punto**	**Acc. promedio clase**
0,811	0,705	0,918	0,931	0,890

Como se describe en la sección anterior, uno de los objetivos del modelo de segmentación es generar mapas de calor que sirvan para condicionar al generador de trayectorias. La Figura 5.3 muestra algunos ejemplos de segmentación y mapas de calor generados en datos de test, donde se puede observar en blanco las zonas positivas M^+ y en negro las zonas negativas M^-.

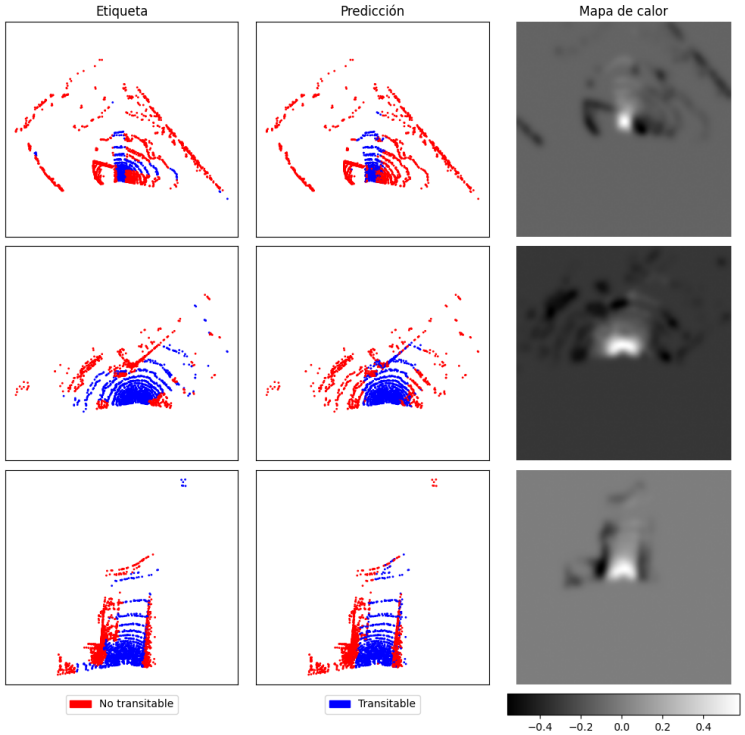


Figura 5.3: Ejemplos de segmentación de PointNet y mapas de calor $\mathbf{I}_{seg}$ generados en los datos de test.

5.3. Entrenamiento NaVAE

Para el entrenamiento del modelo propuesto se usan datos únicamente del sector derecho descrito en la Figura 5.1, tal que se tenga un ambiente de prueba fuera de distribución. El entrenamiento usa los parámetros descritos en la Tabla 5.3 y toma alrededor de 7 horas en 1000 épocas. Se usa KL *annealing* [79] para incrementar gradualmente el peso del término de divergencia KL en la función de pérdida. Esto permite que el modelo se enfoque inicialmente en la reconstrucción de las trayectorias antes de regularizar el espacio latente. Se emplea también una tasa de decaimiento exponencial de la tasa de aprendizaje cada 10 épocas. La evaluación se realiza cada 10 épocas.

Tabla 5.3: Parámetros usados para el entrenamiento del modelo generador NaVAE.

Parámetro		**Valor**
Hiperparam.	lr, β, λ	0,001, 1, 10
Arquitectura	N_{ft}, N_z, N_{x}	384, 128, 128
Física	N_l, N_p, R_{max}, $H \times W$ (I_f), r	3, 2560, 50(m), 180×180, 0,2(m/px)
	N_v, τ_{past}, v_{max}, d_{max}	10, 5(s), 2(m/s), 300(m)
	N_w, K, Δt	12, 200, 1(s)

La Figura 5.4 muestra las curvas de pérdida principales durante el entrenamiento, correspondientes a la divergencia KL $\mathbf{D_{KL}}$, las pérdidas del encoder prior y posterior $\mathcal{L}_{recon}$ y la pérdida de colisión $\mathcal{L}_{col}$. Se observa una convergencia estable tras las primeras 200 épocas. Se muestra además las curvas de evaluación para las métricas FDE y ADE (*Average Displacement Error*) [78].

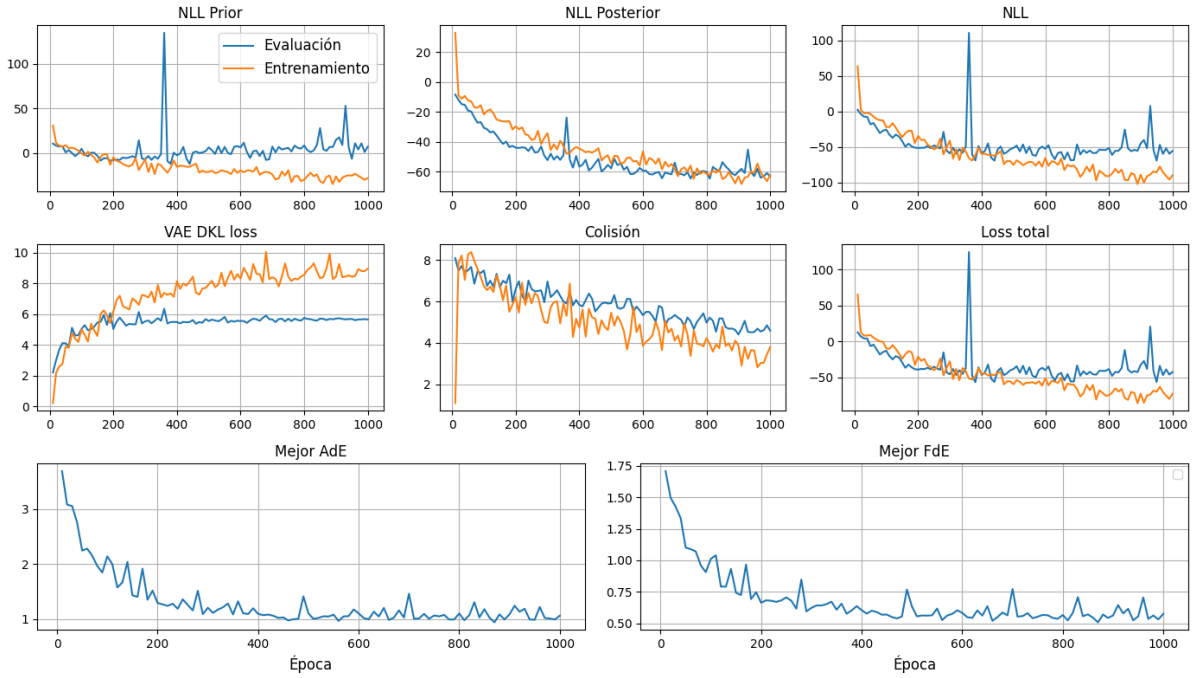


Figura 5.4: Curvas de entrenamiento y evaluación del modelo generador NaVAE.

Para validar la capacidad de generalización del modelo propuesto, se realiza una evaluación preliminar utilizando el conjunto de validación. La Figura 5.5 presenta una visualización del desempeño del generador con un mapa local semántico de fondo, comparando las predicciones del modelo contra la trayectoria real ejecutada por el experto. Se incluye el ADE, FDE y el porcentaje de trayectorias generadas que se encuentran en zonas transitables según el mapa semántico local.

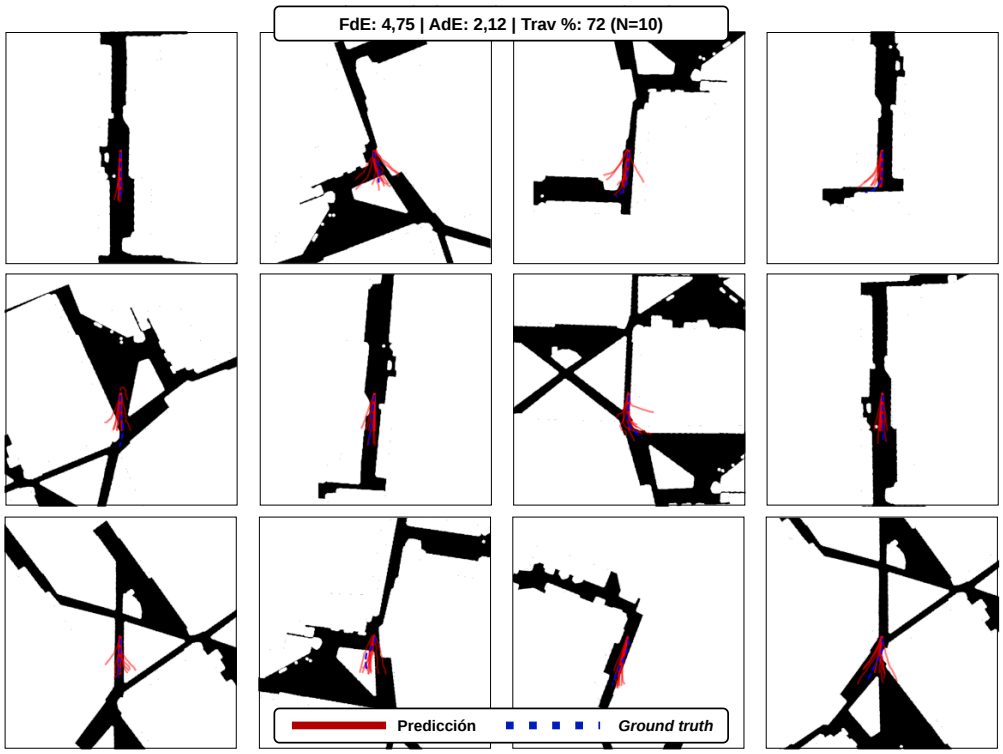


Figura 5.5: Ejemplos de trayectorias generadas por el modelo NaVAE en los datos de evaluación. Se muestra el ADE, FDE y porcentaje de trayectorias generadas en zonas transitables según el mapa semántico local.

5.4. Evaluación en el mundo real

5.4.1. *Setup* experimental

Para todos los experimentos realizados se utiliza una plataforma robótica Husarion Panther [76], que cuenta con su propio módulo de odometría que fusiona encoders e IMU mediante un EKF. Está equipado con un LiDAR 3D Ouster OS-0 de 64 canales con 512 puntos por canal a 10Hz. Además, el robot posee una cámara frontal USB genérica. En su interior el robot cuenta con una Intel NUC que ejecuta el planificador local en ROS Noetic en Ubuntu 20.04. Para el despliegue de algoritmos se prueba intercaladamente con una NVIDIA Jetson Orin AGX 64Gb y un laptop Lenovo LOQ equipado con una GPU RTX4060. El robot está

también equipado con dos módulos GNSS UBlox Zed F9P. La Figura 5.6 muestra el robot con los sensores y los equipos anteriormente mencionados.

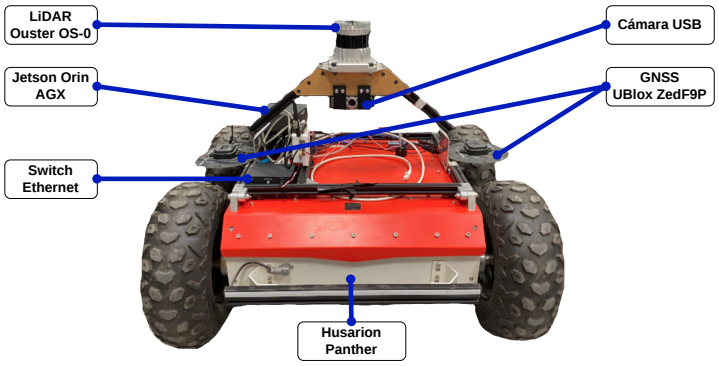


Figura 5.6: *Setup* experimental del robot Husarion Panther usado para las pruebas de navegación.

5.4.2. Pruebas y métricas

Para la evaluación en el mundo real se usan ambos sectores descritos en la Figura 5.1, teniendo en cuenta que el sector A (derecha) es el de entrenamiento y el sector B (izquierda) es el de evaluación. La Figura 5.7 muestra una imagen satelital de ambos sectores juntos, con las posiciones objetivo enumeradas. Los puntos se encuentran todos a una distancia mayor a 40 metros entre sí y poseen curvas y sectores de variable dificultad de navegación por el terreno considerado como transitable.

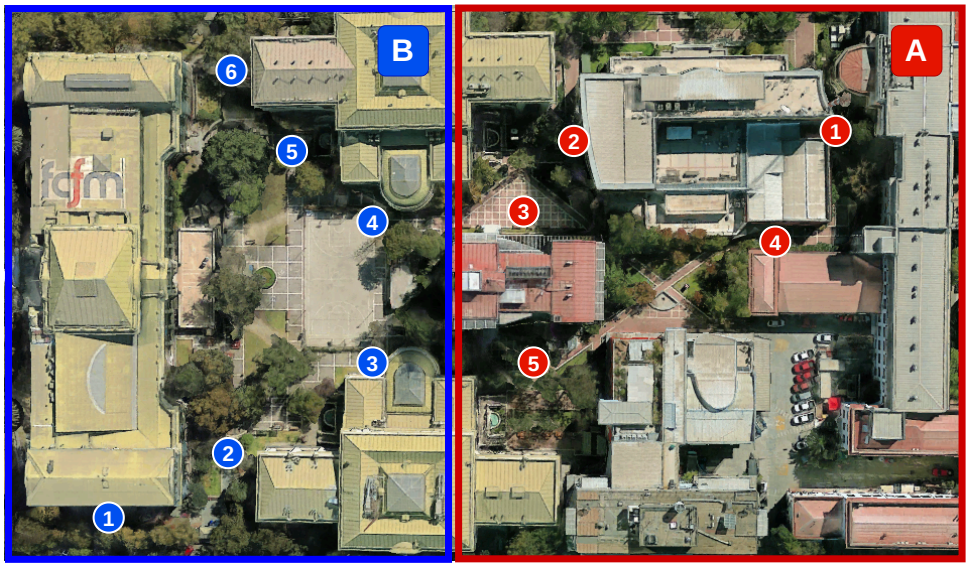


Figura 5.7: Vista satelital del sector usado para evaluar el módulo de navegación global. Los números representan las distintas posiciones objetivo de inicio y fin de las pruebas. En rojo el sector de entrenamiento (A) y en azul el de evaluación (B).

Las pruebas a realizar se listan en la Tabla 5.4, junto con su largo aproximado por una teleoperación experta (dado que es un entorno universitario que tiene modificaciones constantes, puede haber variaciones en la posición de algunos obstáculos entre prueba y prueba). Se realizan 3 pruebas en el sector A (entrenamiento) y 5 pruebas en el sector B (evaluación). Cada prueba se repite 6 veces para obtener resultados estadísticamente significativos.

Tabla 5.4: Orden de las pruebas de navegación realizadas y su largo aproximado por una demostración experta. El objetivo es abarcar distintos tipos de trayectorias y dificultades de navegación en ambos sectores.

Sector	**Experimento**	**Inicio-Fin**	**Largo de referencia (m)**
A	1	1 - 2	122
	2	2 - 4	193
	3	1 - 5	110
B	1	2 - 6	125
	2	4 - 1	157
	3	6 - 1	202
	4	1 - 4	119
	5	6 - 3	112

Para evaluar el desempeño del planificador global se utilizan una serie de métricas que permiten analizar tanto la eficiencia como la efectividad del sistema en diferentes escenarios. Estas métricas incluyen indicadores de éxito, transitabilidad de las rutas generadas y ejecutadas, así como el tiempo requerido para completar las tareas de navegación. A continuación, se describen en detalle las métricas empleadas para esta evaluación:

- ***Success Rate*** **(SR, tasa de éxito)**: Mide el porcentaje de instancias en que el robot alcanza un objetivo dentro de un umbral de distancia euclidianapredefinido (5 metros en este caso) desde su posición inicial.
- ***Average Path Length*** **(PL, longitud promedio de trayectoria)**: Mide la longitud promedio de todas las trayectorias ejecutadas por el robot, para una misma prueba.
- ***SPL*** **(Success weighted by Path Length)**: Mide la eficiencia de la navegación considerando tanto el éxito como la longitud de la trayectoria. Se calcula como la razón entre la distancia teleoperada[4] objetivo d_i y el máximo entre d_i y la longitud de la trayectoria seguida p_i, ponderada por el éxito de la prueba S_i

$$SPL = \frac{1}{N}\sum_{i=1}^{N} S_i \frac{d_i}{\text{máx}(p_i, d_i)}. \tag{5.1}$$

- ***Executed path traversability*** **% (EPT, porcentaje de trayectoria ejecutada transitable)**: Mide el porcentaje de la trayectoria ejecutada por el robot que se encuentra en espacios transitables, según los mapas semánticos usados. Si τ es la trayectoria

[4]Generalmente se usa una distancia bajo un algoritmo de planificación global clásico, como RRT o A*, pero en este caso los obstáculos del ambiente cambian de posición constantemente al ser un ambiente universitario.

completa ejecutada por el robot en una prueba, rasterizada al mapa semántico, de largo N, se tiene que

$$EPT = \frac{1}{N}\sum_{n=0}^{N} g(\tau(n), \mathcal{A}), \tag{5.2}$$

donde $g(\cdot,\cdot)$ es una función que devuelve 1 si el punto se encuentra en una zona transitable preferida y 0 en caso contrario, y $\mathcal{A}$ corresponde al área transitable preferida del mapa (véase Sección 4.3.2 para definiciones de transitabilidad).

- ***Non-traversable Rate* % (NTR, porcentaje de trayectoria generada no transitable)**: Mide el porcentaje de las trayectorias generadas por el modelo que no se encuentran en espacios transitables (tanto estrictos como suaves), según los mapas semánticos usados. Esto se mide al momento de realizar la inferencia y es independiente de la trayectoria seleccionada y ejecutada por el robot. Si $\mathbf{y}^{(k)}$ es la k-ésima trayectoria generada en una prueba, rasterizada al mapa semántico, de largo N, se tiene que

$$NTR = \frac{1}{NK}\sum_{n=0}^{N}\sum_{k=0}^{K} \frac{g(\mathbf{y}^{(k)}(n), \tilde{\mathcal{A}})}{|\mathbf{y}^{(k)}(n)|}, \tag{5.3}$$

  donde $g(.)$ es la función descrita anteriormente y $\tilde{\mathcal{A}}$ corresponde al área no transitable (suave y estricta). Es importante mencionar que esta métrica solo mide la calidad del generador, por ende, las áreas usadas como no transitables deben ser las mismas que se usaron como no transitables durante el entrenamiento del modelo. En este caso, aquellas áreas son solo las que definen clase preferida, como el pavimento (véase Tabla 5.6 para el detalle de las clases usadas).

- ***Navigation time* (T_{nav}, tiempo de navegación)**: Mide el tiempo total en segundos que tarda el robot en alcanzar el objetivo desde su posición inicial.

- ***Navigation time ratio* (T_{ratio}, relación de tiempo de navegación)**: Mide la relación entre el tiempo de navegación del robot y el tiempo de teleoperación.

$$T_{ratio} = \frac{T_{nav}}{T_{teleop}}. \tag{5.4}$$

- **Número de llamadas al *recovery behaviour* (#RB)**: Mide la cantidad de veces que el robot ha tenido que activar el comportamiento de recuperación durante una prueba.

5.4.3. Evaluación del sistema de navegación global

Las pruebas anteriores se realizan comparando el modelo propuesto (**NaVAE**) con los *baselines* de MTG Base (**MTG**) como es presentado en [22] y la versión modificada de MTG (**MTG'**) como es presentada en la Sección 4.4. Todos los modelos usan el mismo sistema de selección de trayectorias descrito en la Sección 4.3. Los parámetros usados se muestran en las Tablas 5.5 y 5.6, los cuales son escogidos empíricamente tras una serie de pruebas preliminares en simulación y en el entorno real.

Tabla 5.5: Parámetros generales usados para el módulo de selección de trayectorias.

Parámetro		**Valor**
Filtrado	$r_{elev}, S_x \times S_y$	$0,1(m/celda)$, $18 \times 18(m^2)$
	θ_c, h_c, d_{foot}	14^o, $0,18(m)$ $0,85(m)$
Cálculo de costos/actualización	$\gamma, \alpha_1, \alpha_2$	0,8, 2, 0,2
	f_{gen}, f_{clip}	0,5, 2,5
Recovery Behaviour	R_{search}, τ_{stuck}	10(m), 3(s)

Tabla 5.6: Valores de las clases usadas para la construcción de mapas de costos.

Clase/color $\mathbf{p_r}$	**pavement**	**grass**	**trees**	**wall**	**stairs**	**person**	**sky**	C_u	T_{occ}
Valor $\mathbb{C}$	0	2	3	3	3	3	4	2	2
Categoría	preferencia	suave	crítico					-	-

Además, para todas las pruebas se utiliza un planificador local reactivo basado en aprendizaje reforzado (RL), propuesto en [46], el cual usa percepción LiDAR 2D. Para localización global se usa AMCL[5] con el mapa 2D mostrado en las Figs. 5.1b y 5.1c, con el objetivo de poder repetir con precisión las posiciones iniciales y finales y calcular las métricas presentadas anteriormente. El sistema de localización no influye en el proceso de generación de trayectorias ni en el de selección (más que para calcular la posición del objetivo), es solo para referencia global, el cual puede ser reemplazado por GPS u otro sistema.

La Tabla 5.7 presenta los resultados cuantitativos obtenidos en las pruebas de navegación global, comparando el modelo propuesto NaVAE con MTG y MTG'. Las métricas PL, T_{nav} y #RB se presentan como promedios de las 6 repeticiones de cada prueba individual, mientras que las demás métricas se promedian en total por sector de pruebas. Los mejores resultados por experimento se muestran en negrita.

[5] `https://wiki.ros.org/amcl`

Tabla 5.7: Resultados cuantitativos de navegación global en ambos sectores de prueba. Los experimentos Ax corresponden a las pruebas en el sector A (entrenamiento) y los experimentos Bx corresponden a las pruebas en el sector B (evaluación).

Modelo	**Exp**	**SR ↑**	**PL (m)**	**SPL ↑**	**EPT % ↑**	**NTR % ↓**	**T_{nav}(s)**	**T_{ratio} ↓**	**#RB**
NaVAE	A1	**0,83**	178,2	**0,44**	89,9	**32,9**	170	**1,59**	**1**
	A2	**0,66**	223,5	**0,33**	**87,5**	41,2	225	**1,70**	**2**
	A3	**1,00**	134,8	**0,76**	**92,6**	40,8	161	**1,77**	**1**
	total A	**0,83**	-	**0,51**	**90,0**	38,3	**-**	**1,69**	-
	B1	**0,83**	143,1	**0,80**	94,2	**20,4**	171	**1,39**	**2**
	B2	**1,00**	115,7	**0,91**	**93,9**	25,4	170	**1,32**	**1**
	B3	**0,66**	210,1	**0,49**	**89,7**	35,2	215	**1,29**	**3**
	B4	**1,00**	170,5	0,72	**95,0**	22,1	196	**1,36**	**0**
	B5	**1,00**	116,0	0,86	**99,5**	**14,3**	140	**1,29**	**2**
	total B	**0,90**	**-**	**0,76**	**95,0**	23,4	**-**	**1,33**	-
MTG'	A1	0,50	188,1	0,39	**94,7**	37,3	220	2,06	**1**
	A2	**0,66**	260,3	0,31	75,7	40,9	307	2,33	3
	A3	0,33	130,0	0,66	82,1	**28,1**	204	2,24	2
	total A	0,50	-	0,45	84,2	**35,4**	-	2,21	-
	B1	0,66	134,9	0,73	**95,5**	23,2	347	2,73	**2**
	B2	0,83	161,1	0,75	86,7	**19,7**	320	2,51	3
	B3	0,00	-	0,00	79,3	**25,8**	-	-	5
	B4	0,66	136,2	**0,82**	94,1	**19,6**	295	2,05	2
	B5	**1,00**	120,1	**0,89**	97,9	22,2	198	1,80	**2**
	total B	0,63	-	0,63	90,7	**22,1**	-	2,41	-
MTG	A1	0,33	191,9	0,21	89,8	45,6	242	2,26	2
	A2	0,00	-	0,00	70,2	**39,9**	-	-	5
	A3	0,17	177,2	0,16	78,4	45,8	201	2,21	4
	total A	0,28	-	0,22	79,4	43,8	-	2,24	-
	B1	0,33	166,5	0,28	66,0	43,7	323	2,65	4
	B2	0,00	-	0,00	59,9	39,8	-	-	4
	B3	0,00	-	0,00	77,2	27,3	-	-	6
	B4	0,33	159,2	0,12	88,3	28,9	278	1,92	2
	B5	0,66	114,8	0,32	96,1	23,8	344	3,30	4
	total B	0,17	-	0,16	77,5	32,7	-	2,62	-

En la Figura 5.8 se muestran ejemplos de una trayectoria exitosa seguida por el robot usando los distintos modelos MTG (verde), MTG' (azul) y NaVAE (rojo), en el sector A (entrenamiento) para las pruebas 1, 2 y 3. En los casos que no se logra llegar al objetivo, se grafica la trayectoria ejecutada más cercana al objetivo, específicamente en la prueba A2 para MTG (verde). Las imágenes satelitales son solo referenciales, ya que las trayectorias graficadas usan los datos de localización por AMCL, los cuales no están perfectamente alineados con las imágenes satelitales usadas. Dado que se usa un umbral de 5 metros para considerar la navegación como exitosa, en algunos casos las trayectorias no terminan exactamente en el punto objetivo.

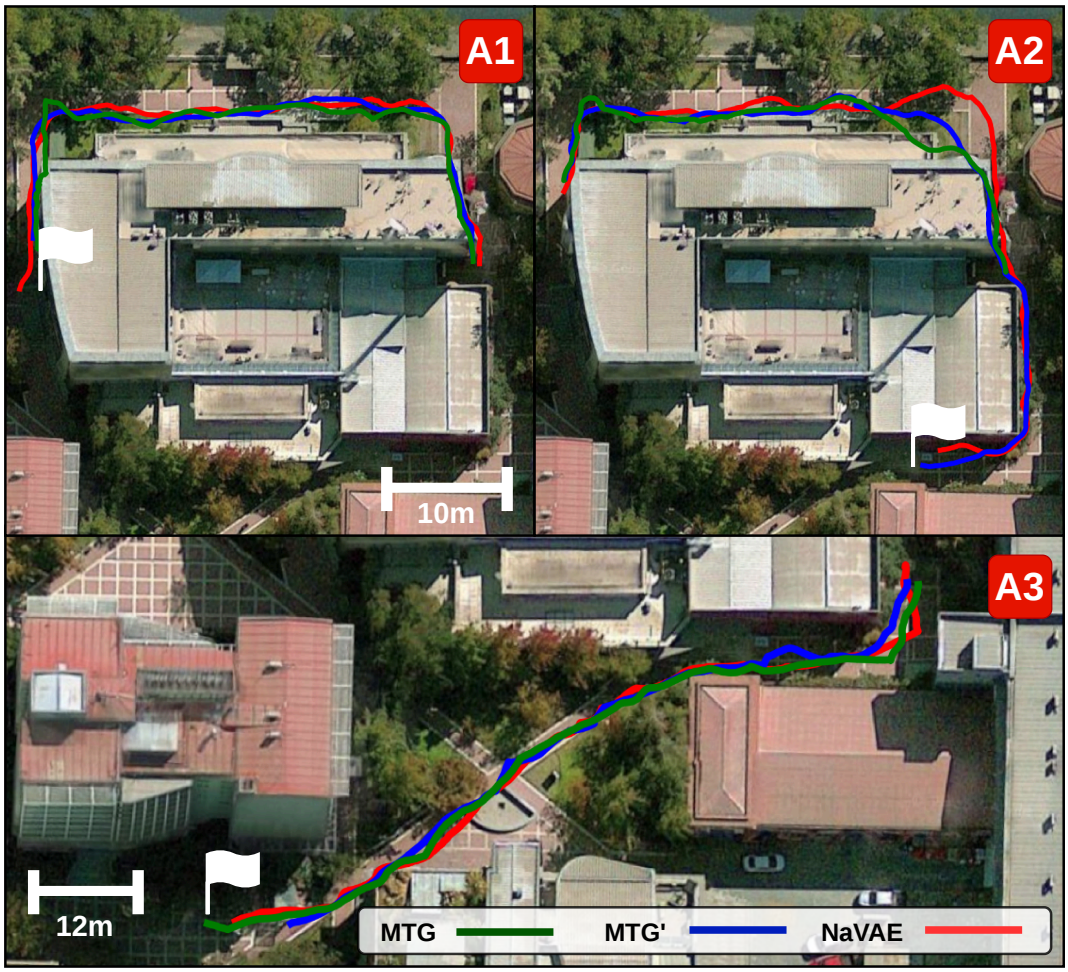


Figura 5.8: Comparativa de trayectorias exitosas ejecutadas de los modelos MTG (verde), MTG' (azul) y NaVAE (rojo) para las pruebas 1, 2 y 3 en el sector A.

Se repiten los mismos gráficos de trayectorias ejecutadas en el sector B (evaluación), para las 5 pruebas descritas en la Tabla 5.4, y se muestran en la Fig. 5.9 con el mismo código de colores. Nuevamente se muestran las trayectorias más cercanas al objetivo en caso de no lograr llegar al mismo, en específico en las pruebas B2 para MTG (verde) y en B3 para MTG (verde) y MTG' (azul).

A modo de comparación entre el abanico de las trayectorias propuestas por cada modelo, se escoge una de las pruebas en el sector B (prueba 1) y se grafican los caminos seguidos, además de las trayectorias generadas proyectadas tanto al mapa de elevación como a la imagen FPV del robot. Esto se muestra en la Figura 5.10. Los mapas de elevación se encuentran en escala de grises, siendo los valores más cercanos al blanco obstáculos de mayor altura. Las trayectorias se pueden obervar en color verde, y en azul se marcan los segmentos no transitables según lo calculado por el filtro de colisiones. En la columna del modelo propuesto (NaVAE) se puede observar además en los puntos rojos las direcciones de movimiento libre calculadas en caso de ser necesario activar el *recovery behavior*.

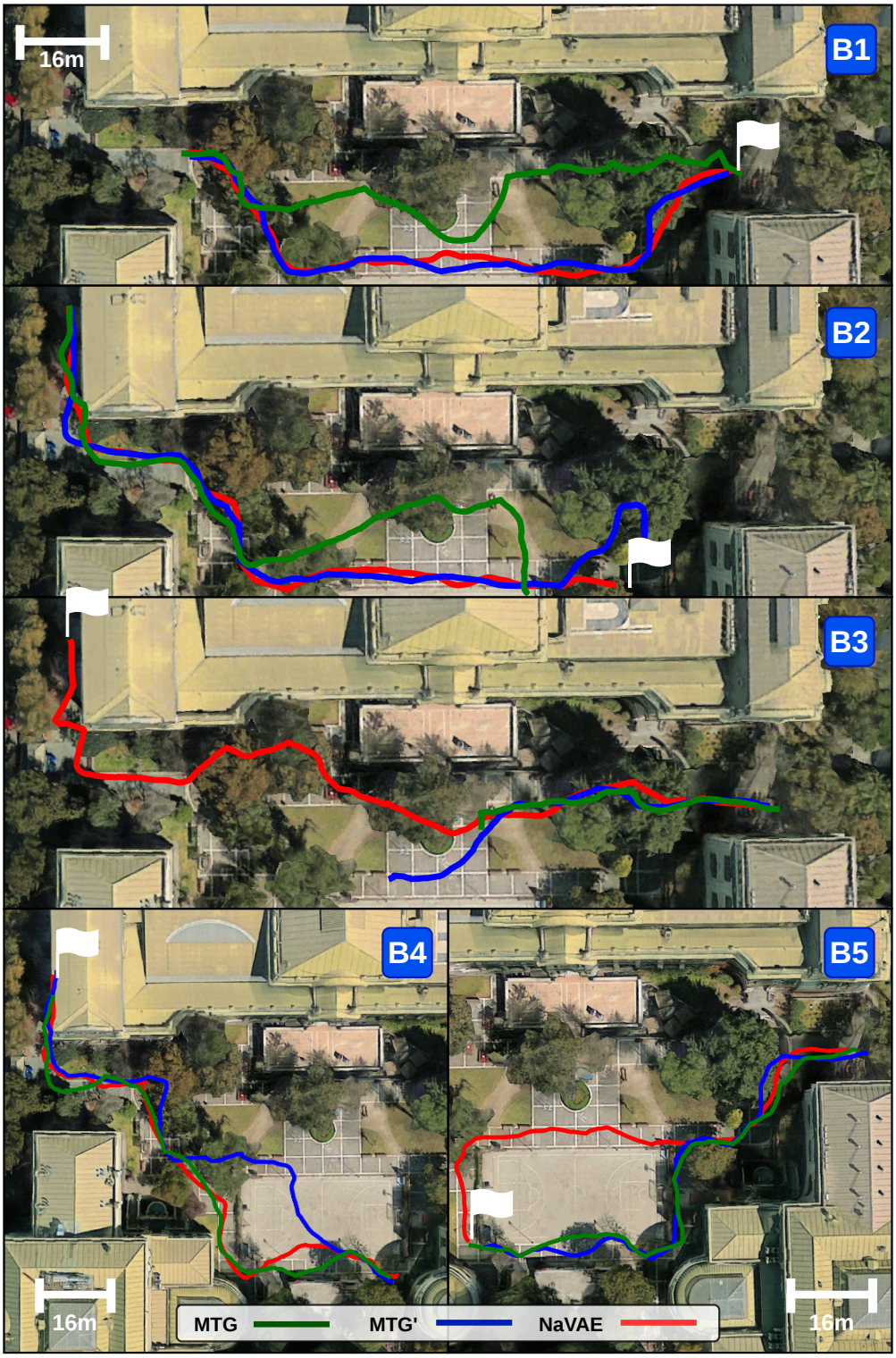


Figura 5.9: Comparativa de trayectorias exitosas ejecutadas de los modelos MTG base (verde), MTG' (azul) y NaVAE (rojo) para las pruebas 1 a 5, en el sector B.

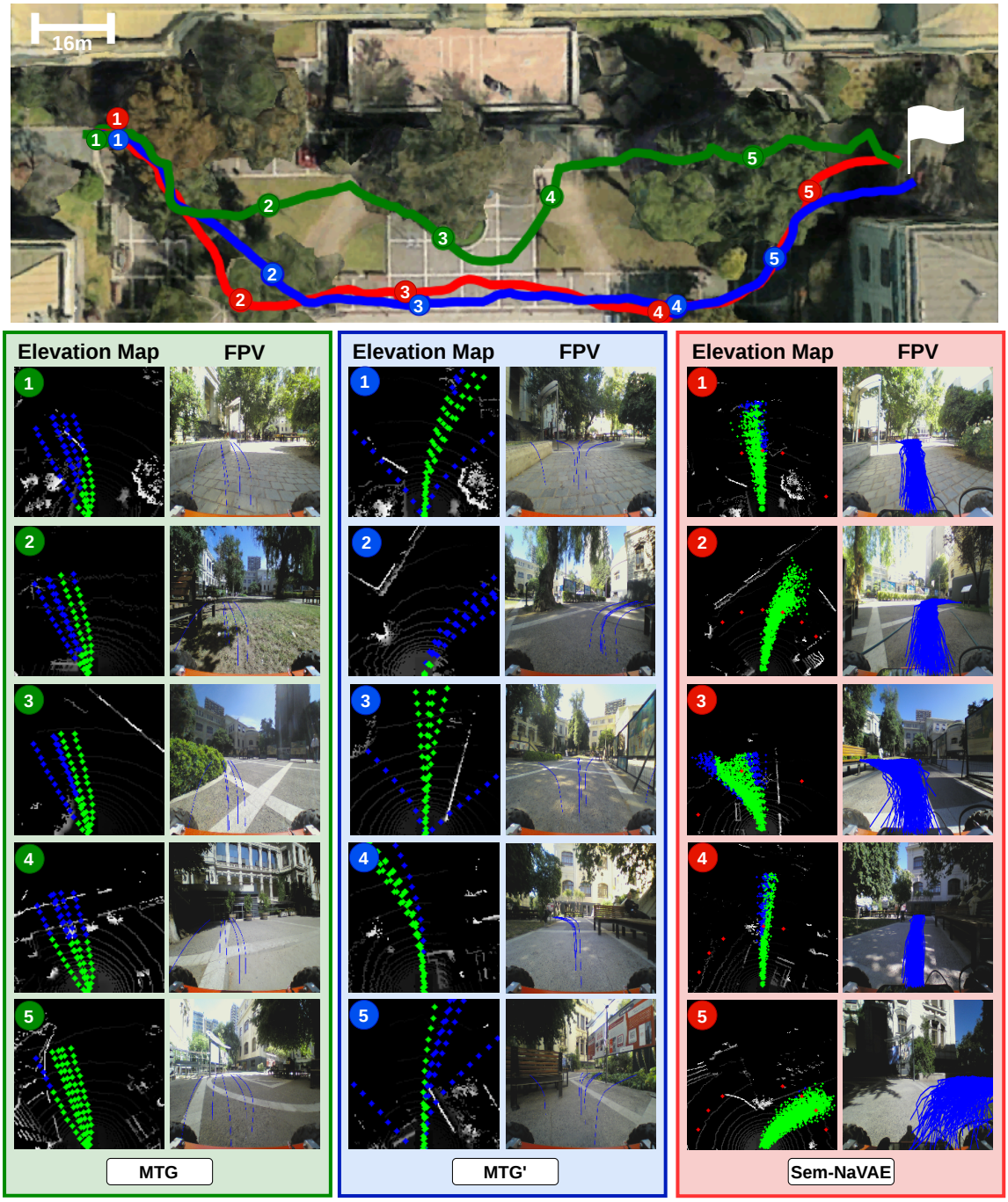


Figura 5.10: Comparación entre las trayectorias ejecutadas y las generadas en el mapa de elevación y proyectadas a la imagen FPV, para los tres modelos en tres pruebas. La columna del mapa de elevación muestra las trayectorias sin colisiones en verde y los *waypoints* con colisión en azul. La columna FPV muestra todas las trayectorias generadas proyectadas a la imagen.

Por último, en la Figura 5.11 se muestra una comparación entre las trayectorias seleccionadas y las trayectorias ejecutadas para los tres modelos en las pruebas B1, B4 y A3. Se utilizan los mapas semánticos en escala de grises para una mayor claridad visual. Las pruebas elegidas son las que tienen una mayor cantidad de clases transitables suaves (césped, tierra, etc.), para observar cómo el módulo de selección escoge las trayectorias en función de los costos asignados a cada clase.

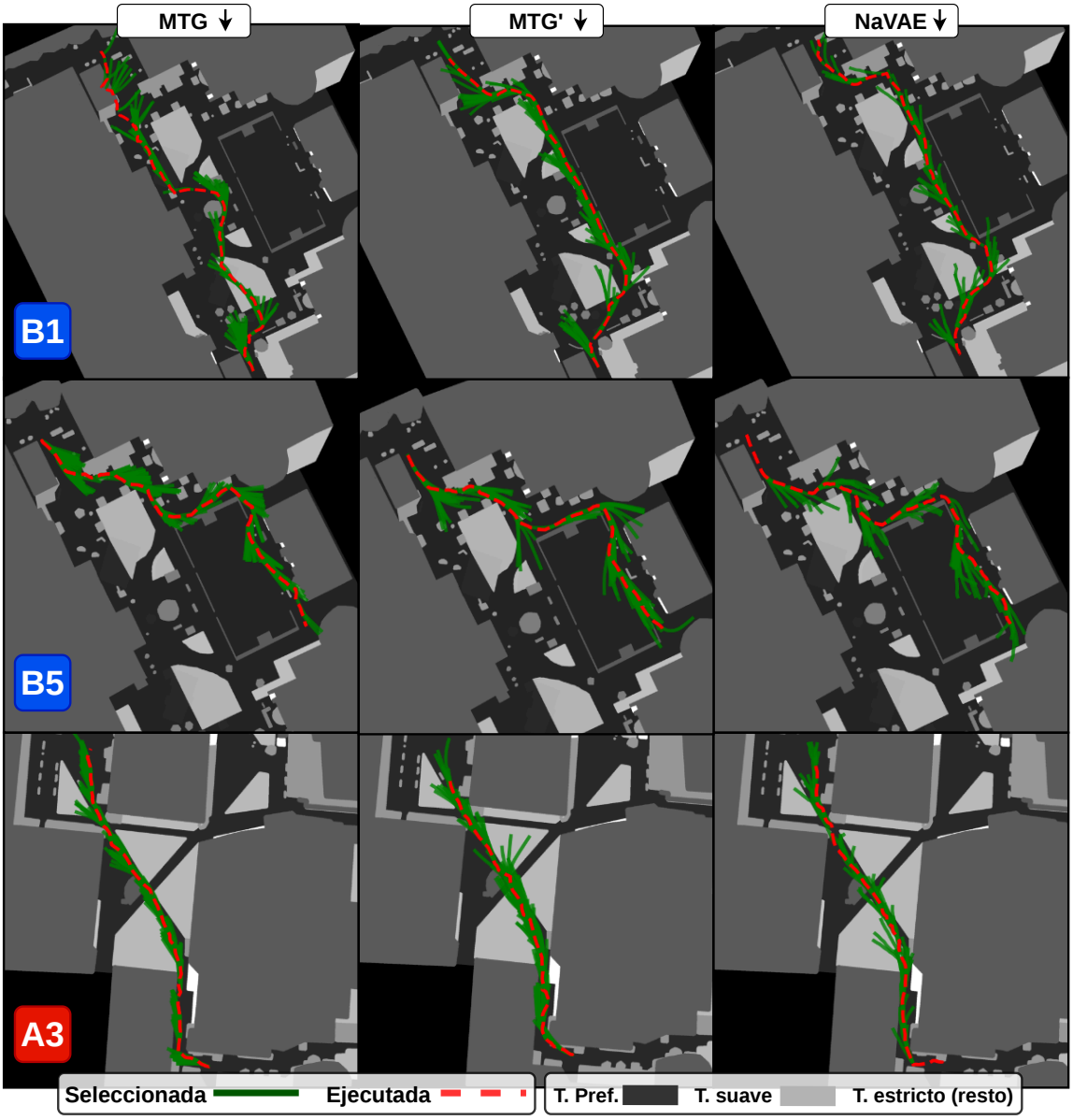


Figura 5.11: Comparativa de trayectoria seleccionada (en verde) con trayectoria ejecutada (en rojo) para los tres modelos en las pruebas B1, B5 y A3. Se muestran los mapas semánticos en escala de grises para una mayor claridad visual, marcando la clase libre, la transitable suave y las clases no transitables (críticas).

5.4.4. Variación parámetros de selección

En términos de los comportamientos emergentes del sistema de navegación completo, es importante evaluar el efecto de variar los parámetros del módulo de selección de trayectorias. En específico, se evalúa el efecto de variar los costos para una clase de transtabilidad suave, y el efecto de agregar una nueva clase al mapa de costos, representando una nueva preferencia del usuario.

Manteniendo los parámetros presentados en la Tab. 5.5, se varía el costo asignado a la clase *grass* (césped) y se prueba en dos sectores distintos que contienen grandes extensiones de dicha clase. La Figura 5.12 muestra ejemplos de trayectorias ejecutadas al variar el costo de la clase *grass* de 0 a 2, considerando que en los parámetros originales el costo 0 se le asigna a la preferencia principal (pavimento) y 2 a las clases críticas (pared, escalera, etc.). Se grafica tanto la imagen satelital como el mapa semántico para mayor claridad ante las clases presentes.

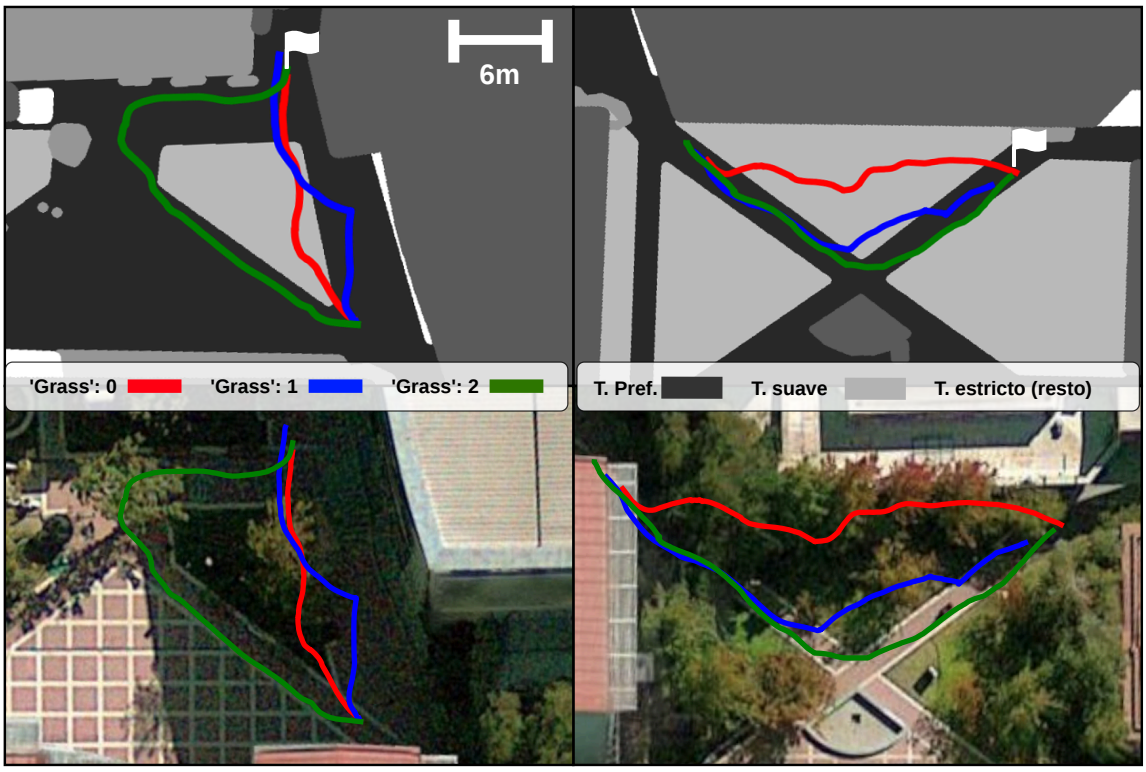


Figura 5.12: Ejemplos de trayectorias ejecutadas variando el costo de la clase *grass*.

Luego, se prueba el efecto de agregar una clase nueva al mapa de costos, representando una nueva preferencia del usuario. En este caso se agrega la clase *sand* (arena) como *prompt* al VLM, para tomar en cuenta por separado los caminos de gravilla de los pavimentados. En la Figura 5.13 se muestra la imagen FPV junto con las probabilidades de los pixeles por clase y los mapas de costos resultantes, al usar y no usar la clase *sand*. En la misma figura se muestran las trayectorias ejecutadas al variar el costo asignado a la clase *sand* de 0 a 2. La idea es observar cómo el sistema evita o prefiere los caminos de gravilla según el costo

asignado, que en un inicio se mantiene igual al de la preferencia principal (pavimento) y se aumenta el costo hasta el valor de las clases críticas.

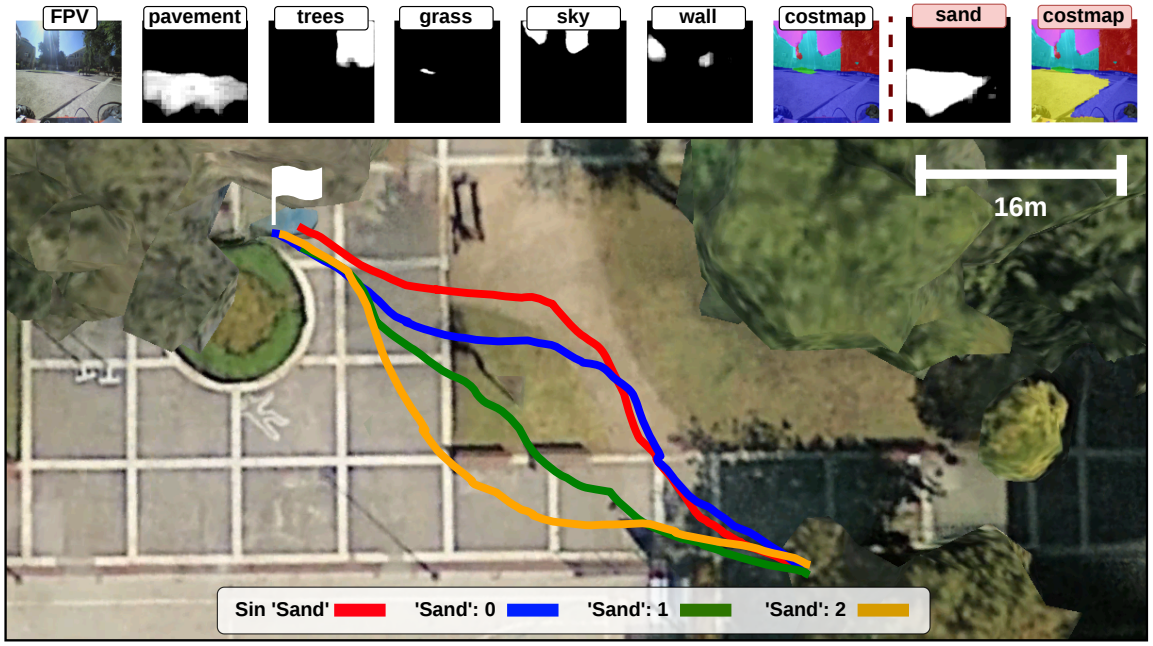


Figura 5.13: Ejemplos de mapas de costos obtenidos al agregar la clase *sand* a la segmentación. Abajo las trayectorias ejecutadas al usar y no usar la clase y variar su costo.

5.4.5. Variación del planificador local

Un punto importante a evaluar es el desempeño del sistema de navegación global al variar el planificador local. Para esto, se realizan pruebas usando dos planificadores locales distintos: DWA (*Dynamic Window Approach*)[6] y el planificador basado en RL usado para pruebas presentadas anteriormente. Para las pruebas con DWA se usa el *stack* de navegación de ROS con un mapa global vacío, tal que los *waypoints* enviados al planificador sean exactamente los mismos de las trayectorias seleccionadas por el módulo de planificación global. La Figura 5.14 muestra ejemplos de trayectorias ejecutadas usando ambos planificadores locales en las pruebas B1 y A2. Es importante mencionar que ambos planificadores locales probados usan percepción LiDAR 2D únicamente, aparte de sus mediciones de odometría.

[6] `https://wiki.ros.org/dwa_local_planner`

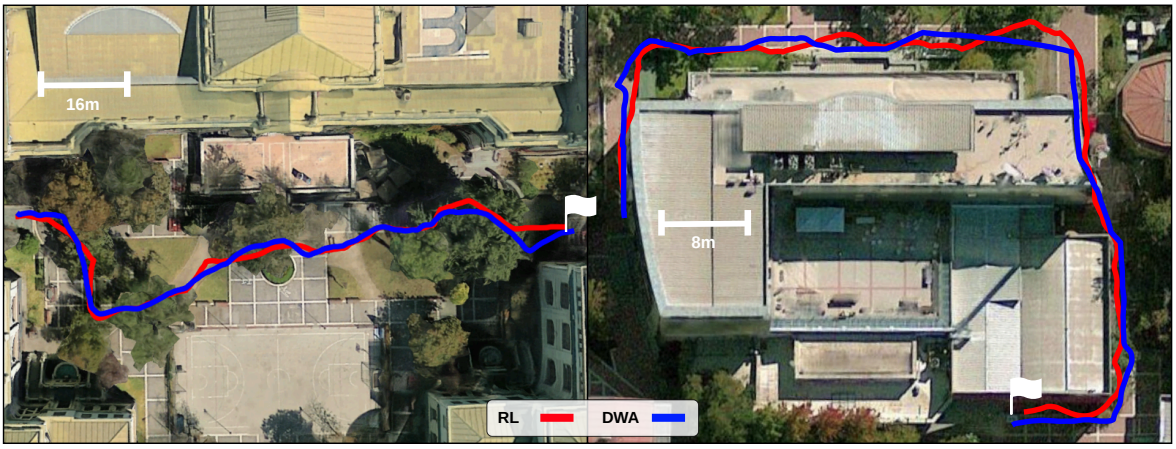


Figura 5.14: Ejemplos de trayectorias ejecutadas usando dos planificadores locales distintos: uno con DWA y otro basado en aprendizaje reforzado.

Capítulo 6

Análisis y discusión de resultados

Entrenamiento NaVAE y PointNet

De los resultados presentados en la Figura 5.2 es posible notar como el modelo PointNet logra converger exitosamente en el conjunto de datos de prueba, alcanzando *Accuracy* del 93 % en la clasificación de las nubes de puntos. En la Figura 5.3 se pueden ver ejemplos de segmentación y mapas de calor generados en los datos de test, donde se tiene una buena clasificación de las distintas clases del entorno, con algunos errores puntuales en los bordes de las clases. Estos errores pueden atribuirse a la naturaleza ruidosa de los datos de entrada y a la complejidad del entorno, pero en general el modelo demuestra una capacidad sólida para segmentar el entorno de manera efectiva.

Por otra parte, desde una perspectiva cualitativa, se observa en la Fig. 5.5 cómo el modelo propuesto NaVAE es capaz de interpretar la estructura del entorno local. Las trayectorias generadas (en verde) no se limitan a replicar una única ruta, sino que exhiben la multimodalidad esperada del CVAE, proponiendo un abanico de rutas alternativas. Es notable que la distribución de estas predicciones respeta las restricciones impuestas por el mapa de ocupación en su mayoría (72 % de generación en espacios transitables tanto suave como críticamente), curvándose adecuadamente para evitar las zonas de obstáculos (en blanco) y manteniéndose en general dentro del espacio libre, lo que evidencia que el modelo ha aprendido en gran parte la relación entre la percepción visual local y la viabilidad de movimiento.

Generador CVAE y resultados de navegación

A partir de los resultados de la Tabla 5.7 se puede notar una mejora considerable del desempeño del modelo propuesto, exceptuando la generación en áreas transitables NTR. En particular, la tasa de éxito, SPL y el tiempo de navegación son las métricas con mayor porcentaje de mejora, obteniendo un 33 %, 10 % y 23 % de mejora respectivamente en el sector A y un 20 %, 8 % y 44 % respectivamente en el sector B, en comparación al mejor *baseline*. Lo anterior se debe principalmente a la capacidad de NaVAE para generar una mayor cantidad de posibles direcciones de movimiento y, por ende, una mayor tasa de éxito al momento de seleccionar una trayectoria factible. El tiempo de navegación se reduce considerablemente

gracias a que los *waypoints* seleccionados requieren menores correcciones por parte del planificador local y porque la tasa de selección es menor (detallado más adelante). La mejora de las métricas en el sector B respecto al A se deben principalmente a que el último tiene mayor complejidad, al ser pasillos más estrechos y con curvas más cerradas.

Aun así, es posible notar que en términos de generación de trayectorias en espacios transitables, NaVAE no obtiene resultados considerablemente mejores que el mejor *baseline* (MTG') en todos los casos. Esto debido a que el modelo genera una gran cantidad de trayectorias donde muchas de estas no son factibles, lo que reduce el promedio. Sin embargo, es importante notar que la generación de trayectorias que residen en espacios no transitables (tanto estrictos como suaves) no afecta directamente el desempeño del sistema de navegación siempre y cuando haya una cantidad suficiente de trayectorias factibles, ya que el módulo de selección se encarga de filtrar estas trayectorias y seleccionar la mejor opción disponible. Esto puede verse reflejado en la métrica de EPT, donde NaVAE obtiene los mejores resultados generales, demostrando que a pesar de generar trayectorias no factibles, el módulo de selección es capaz de elegir trayectorias óptimas para la navegación.

De la Figura 5.8 y 5.9 se puede notar que las trayectorias ejecutadas por el robot presentan movimientos oscilatorios en varios puntos del recorrido. Esto se debe a una combinación de factores. Primero, el momento en que se selecciona una trayectoria puede que, si bien sea el óptimo en términos de costo, no sea el ideal dada la pose del robot, provocando que se generen movimientos tipo zigzag al corregir. Segundo, el planificador local no está pensado para seguir de manera perfecta los *waypoints* entregados como objetivo, sino que busca llegar a estos haciendo las correcciones que el ambiente requiera. No obstante, a pesar de estos movimientos oscilatorios, el robot es capaz de llegar a su objetivo en todos los casos de prueba, probando la robustez del sistema de navegación global propuesto. Es posible observar además que el modelo propuesto es el que presenta la menor cantidad de movimientos zigzag, al comparar con los *baselines*.

Se puede observar también en las Figuras 5.8 y 5.9 que las trayectorias ejecutadas no son necesariamente óptimas globalmente, ni idénticas entre repeticiones de la misma prueba. Esto se debe a que el planificador no tiene acceso a un mapa global para elegir la ruta de menor distancia o con menores obstáculos, sino que se basa en las trayectorias generadas localmente y en la selección de estas. Por lo tanto, es posible que el planificador elija diferentes rutas en diferentes repeticiones, dependiendo de las trayectorias generadas y seleccionadas en cada instancia. Aun así, el robot es capaz de adaptarse a los obstáculos del entorno y alcanzar los objetivos de manera efectiva.

En la Figura 5.10 se pueden apreciar las diferentes trayectorias generadas por los tres modelos en un mismo escenario. Se puede notar que el *baseline* de MTG base no tiene variabilidad en las trayectorias generadas, produciendo siempre el mismo abanico. Esto se atibuye al problema de *posterior collapse* y a que el modelo entrenado es tiene una dependencia fuerte al formato de entrada de la nube de puntos, siendo este mejor ajustado al usado en el trabajo original, distinto al de esta tesis. Por otro lado, MTG' presenta una mayor diversidad en las trayectorias generadas, mostrando multimodalidad en algunos casos (como en 1, 3, y 5), sin embargo las trayectorias redundantes no son factibles en su mayoría. Finalmente, el modelo propuesto NaVAE muestra un abanico que es "modulado" por el entorno, donde en los sectores más estrechos tiende a generar trayectorias más alineadas (1 y 4), mientras que

en los sectores más abiertos la diversidad es mayor (2, 3, 5), pero sin mostrar modos separados como en MTG'. Esto indica que el modelo ha aprendido a adaptar sus predicciones enfocándose en direcciones específicas de movimiento, pero cubriendo la mayoría del espacio transitable.

Por otra parte, en la Figura 5.11 se puede notar algo interesante. Gran parte de las trayectorias seleccionadas poseen *waypoints* que se encuentran en lugares no factibles según el mapa de transitabilidad (NTR mayor). Hay dos puntos a considerar respecto a esto. El primero, menos relevante y con menor presencia, es que el mapa de transitabilidad no es perfecto y puede poseer cambios respecto al momento en que se generó (y por ende las métricas de evaluación respecto a transitabilidad pueden tener pequeñas imprecisiones porcentuales). La segunda y más importante, es que el modelo de navegación global está pensado como un planificador de alto nivel "grueso", que entrega una guía al planificador local para que este la refine. A partir de los resultados, es posible notar que la combinación tanto del sistema de selección como el planificador local son capaces de manejar las imprecisiones en las trayectorias generadas por el modelo NaVAE.

Por otra parte, en los casos que ambos niveles de planificación fallan, el *recovery behaviour* es capaz de compensar. Ejemplos de esto pueden observarse en la Figura 5.11, en los sectores que hay curvas cerradas, ya que el robot para y ejecuta el *recovery* previo a volver a moverse. En estos casos las trayectorias previamente seleccionadas llevan directo a una pared en su sector final, y una vez que el planificador local no es capaz de avanzar y las nuevas trayectorias generadas tampoco son factibles, el *recovery behaviour* toma el control y rota el robot en la dirección correcta hasta que una nueva trayectoria factible es seleccionada. Es importante mencionar que, si bien este procedimiento es heurístico, su implementación es crucial para la robustez del sistema de navegación completo propuesto, ya que como se muestra en la Tab. 5.7, se usa en casi todos los casos al menos una vez.

Sistema de selección de trayectorias

Complementario a lo anterior, el sistema de selección logra elegir la trayectoria óptima de manera no periódica. Es decir, se actualiza solo cuando el costo de continuar ejecutando la trayectoria actual es mayor que el costo de cambiar a una nueva trayectoria, en vez de actualizar a una frecuencia fija. Esto se puede notar especialmente en las curvas (véase la columna NaVAE para los experimentos B1 y B5 de la Figura 5.11), donde la densidad de trayectorias seleccionadas (cantidad de trayectorias nuevas en un determinado desplazamiento del robot) es mayor que en los sectores rectos, debido a que la dificultad es mayor. Por otra parte, es posible observar en los todos los casos mostrados en la Fig. 5.11 como la trayectoria real ejecutada por el planificador local es una versión suavizada de la seleccionada, a pesar de presentar los movimientos oscilatorios mencionados anteriormente.

De los resultados presentados en la Sección 5.4.4 se puede observar cómo variar los costos de las clases del mapa de costo influye directamente en las trayectorias seleccionadas y, por consecuencia, en la trayectoria ejecutada por el robot. En particular, al aumentar el costo de la clase *grass* (césped), se puede notar cómo las trayectorias seleccionadas tienden a evitar esta clase, prefiriendo caminos alternativos que pueden ser más largos, pero que evitan terrenos

con mayor dificultad.

Por otra parte, también se observa como agregar una clase puede alterar el comportamiento del sistema de selección. En el caso de la Fig. 5.13, al agregar la clase *sand* (arena) con un costo alto, se observa que las trayectorias seleccionadas evitan en mayor medida el camino de gravilla, prefiriendo caminos más abiertos aunque estos impliquen atravesar césped. Esto demuestra la flexibilidad del sistema de selección para adaptarse a diferentes escenarios y preferencias de navegación.

Planificador local

Los resultados presentados en la sección 5.4.5 demuestran la capacidad del módulo de navegación global para adaptarse a diferentes planificadores locales. A pesar de las diferencias en el comportamiento y las capacidades de los planificadores locales evaluados, el módulo de navegación global propuesto es capaz de integrarse efectivamente con ambos. En particular, de la Figura 5.14 se puede observar que las trayectorias ejecutadas por ambos planificadores son similares tanto en longitud como en las zonas transitadas.

Si bien el módulo de planificación global no está diseñado específicamente para un planificador local en particular, el desempeño final del sistema de navegación depende en gran medida de las capacidades del planificador local para corregir las trayectorias seleccionadas. En este sentido, se refuerza la importancia de una planificación multinivel, donde cada nivel complementa las capacidades del otro para lograr una navegación efectiva y robusta.

En relación a lo anterior, ambos planificadores locales usados presentan limitaciones para ambientes de exterior considerando que usan percepción bidimensional, que no ocurrirían con un planificador local que tenga capacidades de percepción 3D. Puede ocurrir (y ocurre) que falle una tarea de navegación por obstáculos que no son registrados correctamente por ninguno de los dos niveles de planificación, como por ejemplo escalones pequeños o terrenos con terrenos poco regulares, que presenten un gran número de variaciones de altura. En ese caso también, el filtro de colisiones puede tener problemas con sus cálculos. Una posible solución a eso último es orientar correctamente la nube de puntos usando las mediciones IMU, o yendo más allá, teniendo una odometría robusta tridimensional.

Capítulo 7

Conclusiones

El desarrollo de esta tesis ha abordado uno de los desafíos más relevantes en la robótica móvil: la navegación autónoma en entornos no estructurados sin la dependencia de mapas preexistentes. A través del diseño, implementación y validación de una arquitectura híbrida, que combina la formulación probabilística de los modelos generativos con la segmentación de vocabulario abierto de los modelos de visión-lenguaje, se ha demostrado que es posible dotar a un agente robótico de una capacidad de discriminación semántica ausente en los enfoques geométricos tradicionales.

El trabajo se formuló en torno a la premisa de separar la generación de movimiento de la toma de decisiones semánticas, validando el paradigma de "Generar y Seleccionar". A continuación, se detalla el cumplimiento de los objetivos específicos y las conclusiones derivadas de cada etapa.

En relación al primer objetivo específico, se generó exitosamente un conjunto de datos multimodal capturado en entornos reales. La sincronización entre la odometría, las nubes de puntos LiDAR, las imágenes RGB y los mapas semánticos construidos permitió entrenar modelos de aprendizaje profundo que capturan el comportamiento cinemático y las decisiones del operador en una plataforma tipo *skid-steer*. Esta base de datos no solo sirvió como base para el entrenamiento supervisado, sino que constituye en sí mismo un aporte, dado que es reutilizable para entrenar nuevos modelos de predicción de trayectorias robóticas en escenarios similares.

Respecto al segundo objetivo, el desarrollo del planificador global basado en CVAE con *Learned Prior* demostró ser una solución eficaz para el problema de la multimodalidad. Los resultados experimentales evidencian que, al condicionar la generación al contexto local, el modelo fue capaz de proponer múltiples hipótesis de movimiento geométricamente viables. Más aún, el planificador generativo logró cubrir gran parte del espacio navegable disponible en dirección al objetivo global, ofreciendo un abanico de opciones seguras sobre las cuales decidir. Esto confirma la evidencia en la literatura [56, 21] de que los modelos generativos son herramientas altamente efectivas para modelar la incertidumbre y la multimodalidad inherentes a la navegación en exteriores."

El tercer objetivo se cumplió mediante la integración del modelo CLIPSeg como me-

canismo de intermedio de selección semántica. La capacidad de este modelo para realizar segmentación de vocabulario abierto permitió la correcta discriminación de las hipótesis de movimiento, con la capacidad de variar las preferencias de navegación de manera simple. La estrategia de actualización asincrónica de costos y trayectorias permitió manejar las oclusiones temporales y la incertidumbre de la percepción visual, otorgando al robot una capacidad de reacción semántica (evasión de categorías específicas) que no se tiene en los planificadores puramente geométricos.

Finalmente, en cuanto al cuarto objetivo, la validación experimental en una plataforma real confirmó la viabilidad del sistema integrado. Las pruebas en entornos exteriores demostraron que la arquitectura propuesta es capaz de cerrar el ciclo de percepción-acción en tiempo real, validando la robustez de la propuesta en condiciones del mundo real.

En vista de los resultados obtenidos, se considera que las hipótesis planteadas han sido validadas. La integración de modelos generativos de trayectorias (CVAE) con modelos de visión-lenguaje permitió, efectivamente, superar las limitaciones de unimodalidad y rigidez semántica de los planificadores clásicos. La comparación con el modelo base (MTG) evidencia que, mientras un generador geométrico puede proponer rutas viables, carece de la facultad para priorizarlas según el contexto variable (lenguaje natural). Por tanto, la propuesta valida una adaptabilidad cualitativamente superior, al permitir que el robot modifique su comportamiento de navegación basándose en la interpretación del entorno y no solo en la estructura física del mismo.

7.1. Trabajo Futuro

A pesar de los resultados satisfactorios, existen distintas áreas de mejora que pueden ser abordadas en trabajos futuros.

En primer lugar, si bien el objetivo no era tener un modelo generador con cobertura de todas las áreas transitables como MTG, sino más bien un modelo que proponga rutas diversas y semánticamente informadas, la cobertura del espacio navegable puede ser mejorada. Esto podría lograrse mediante la incorporación de funciones de pérdida específicas para esta tarea y también cambiando la distribución de velocidades predichas de una gaussiana multivariable a una mixtura de gaussianas (GMM), de modo que la multimodalidad de las predicciones sea aún mayor.

Por otra parte, como se mencionó en la discusión, uno de los principales limitantes del sistema de navegación completo en ambientes no estructurados es el planificador local 2D. Aunque el planificador global propuesto genera trayectorias diversas, la ejecución de estas trayectorias depende críticamente de la capacidad del planificador local para refinar dichas secuencias. En este sentido, una versión siguiente del sistema de navegación podría incluir un planificador local con capacidades tridimensionales y semánticas, posiblemente de manera acoplada al módulo global. Esto permitiría una mejor gestión de terrenos irregulares y obstáculos complejos, abriendo la posibilidad a compartir observaciones de manera que se reduzca la complejidad total del sistema.

Por último, el enfoque presentado es eminentemente reactivo; el robot "olvida" lo que vio hace unos metros, aun cuando usa información sensorial pasada. Esto puede provocar que las trayectorias vuelvan a un punto de inicio o que nunca se alcance el objetivo. Lo anterior podría abordarse desde el módulo de selección, donde se construya incrementalmente un grafo topológico (o similar) semántico ligero, como en [49, 80]. A diferencia de los mapas métricos densos, esta representación almacenaría nodos asociados a características visuales clave o instrucciones ejecutadas previamente. De esta forma, el módulo de selección podría reconocer si el agente está revisitando una zona o si se ha desviado de una instrucción global, permitiendo una planificación de retorno o reorientación basada en hitos visuales reconocidos por el modelo CLIPSeg. Dicho enfoque requeriría usar un VLM más potente o entrenar un modelo capaz de razonar sobre el mapa topológico semántico construido, haciendo el sistema más complejo.

Bibliografía

[1] Sorin Grigorescu, Bogdan Trasnea, Tiberiu Cocias, et al. A Survey of Deep Learning Techniques for Autonomous Driving. *Journal of Field Robotics*, 37(3):362–386, 2020.

[2] Ye-Hoon Kim, Jun-Ik Jang, and Sojung Yun. End-to-End Deep Learning for Autonomous Navigation of Mobile Robot. In *IEEE International Conference on Consumer Electronics (ICCE)*, pages 1–6, 2018.

[3] Xiaogang Ruan, Dingqi Ren, Xiaoqing Zhu, et al. Mobile Robot Navigation Based on Deep Reinforcement Learning. In *Chinese Control And Decision Conference (CCDC)*, pages 6174–6178, 2019.

[4] Jonas Frey, Matias Mattamala, Nived Chebrolu, et al. Fast Traversability Estimation for Wild Visual Navigation. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.

[5] Boris Ivanovic and Marco Pavone. The Trajectron: Probabilistic Multi-Agent Trajectory Modeling With Dynamic Spatiotemporal Graphs. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 2375–2384, 2019.

[6] Agrim Gupta, Justin Johnson, Li Fei-Fei, et al. Social GAN: Socially Acceptable Trajectories with Generative Adversarial Networks. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2635–2644, 2018.

[7] Ajay Sridhar, Dhruv Shah, Catherine Glossop, et al. NoMaD: Goal Masked Diffusion Policies for Navigation and Exploration. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 12224–12230, 2024.

[8] Daeun Song, Jing Liang, Amirreza Payandeh, et al. VLM-Social-Nav: Socially Aware Robot Navigation Through Scoring Using Vision-Language Models. *IEEE Robotics and Automation Letters*, 10(1):508–515, 2025.

[9] Dhruv Shah, Blazej Osinski, Brian Ichter, and Sergey Levine. LM-Nav: Robotic Navigation with Large Pre-Trained Models of Language, Vision, and Action. In *Proceedings of the 6th Conference on Robot Learning (CoRL)*, pages 259–269. PMLR, 2022.

[10] Kihyuk Sohn, Honglak Lee, and Xinchen Yan. Learning structured output representation using deep conditional generative models. In C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 28. Curran Associates, Inc., 2015.

[11] Roland Siegwart, Illah Reza Nourbakhsh, and Davide Scaramuzza. *Introduction to autonomous mobile robots*. MIT press, 2011.

[12] Peter E Hart, Nils J Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

[13] Edsger W Dijkstra. A note on two problems in connexion with graphs. In *Edsger Wybe Dijkstra: his life, work, and legacy*, pages 287–290. 2022.

[14] Steven LaValle. Rapidly-exploring random trees: A new tool for path planning. *Research Report 9811*, 1998.

[15] Johann Borenstein, Hobart R Everett, Liqiang Feng, and David Wehe. Mobile robot positioning: Sensors and techniques. *Journal of robotic systems*, 14(4):231–249, 1997.

[16] Dieter Fox. Adapting the sample size in particle filters through kld-sampling. *The international Journal of robotics research*, 22(12):985–1003, 2003.

[17] Armin Hornung, Kai M. Wurm, Maren Bennewitz, Cyrill Stachniss, et al. Octomap: an efficient probabilistic 3d mapping framework based on octrees. *Autonomous Robots*, 34(3):189–206, Apr 2013.

[18] Péter Fankhauser, Michael Bloesch, Christian Gehring, et al. Robot-Centric Elevation Mapping with Uncertainty Estimates. In *International Conference on Climbing and Walking Robots (CLAWAR)*, pages 433–440, 2014.

[19] Diederik P. Kingma and Max Welling. Auto-Encoding Variational Bayes. In *2nd International Conference on Learning Representations, (ICLR)*, 2014.

[20] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT press, 2016.

[21] Tim Salzmann, Boris Ivanovic, Punarjay Chakravarty, and Marco Pavone. Trajectron++: Multi-Agent Generative Trajectory Forecasting With Heterogeneous Data for Control. In *European Conference on Computer Vision (ECCV)*, pages 683–700, 2020.

[22] Jing Liang, Peng Gao, Xuesu Xiao, et al. MTG: Mapless Trajectory Generator with Traversability Coverage for Outdoor Navigation. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 2396–2402, 2024.

[23] Ye Yuan and Kris Kitani. DLow: Diversifying Latent Flows for Diverse Human Motion Prediction. In *European Conference on Computer Vision (ECCV)*, pages 346–364. Springer, 2020.

[24] Charles R. Qi, Hao Su, Kaichun Mo, et al. PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 77–85, 2017.

[25] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, et al. On the opportunities and risks of foundation models, 2022.

[26] Tom B. Brown, Benjamin Mann, Nick Ryder, et al. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901, 2020.

[27] Alexander Kirillov, Eric Mintun, Nikhila Ravi, et al. Segment Anything. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4015–4026, 2023.

[28] Mathilde Caron, Hugo Touvron, Ishan Misra, et al. Emerging Properties in Self-Supervised Vision Transformers. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9630–9640, 2021.

[29] Aishwarya Agrawal, Jiasen Lu, Stanislaw Antol, et al. VQA: Visual Question Answering. In *IEEE International Conference on Computer Vision (ICCV)*, pages 2425–2433, 2015.

[30] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, et al. Gpt-4 technical report, 2024.

[31] Alec Radford, Jong Wook Kim, Chris Hallacy, et al. Learning Transferable Visual Models From Natural Language Supervision. In *International Conference on Machine Learning (ICML)*, pages 8748–8763. PMLR, 2021.

[32] Timo Lüddecke and Alexander Ecker. Image segmentation using text and image prompts. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7086–7096, June 2022.

[33] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *International Conference on Learning Representations (ICLR)*, 2021.

[34] Sebastian Thrun, Mike Montemerlo, Hendrik Dahlkamp, David Stavens, et al. Stanley: The robot that won the darpa grand challenge. *Journal of Field Robotics*, 23(9):661–692, 2006.

[35] Lorenz Wellhausen, Alexey Dosovitskiy, René Ranftl, et al. Where Should I Walk? Predicting Terrain Properties From Images Via Self-Supervised Learning. *IEEE Robotics and Automation Letters*, 4(2):1509–1516, 2019.

[36] Chanyoung Jung and David Hyunchul Shim. Incorporating Multi-Context Into the Traversability Map for Urban Autonomous Driving Using Deep Inverse Reinforcement Learning. *IEEE Robotics and Automation Letters*, 6(2):1662–1669, 2021.

[37] Samuel Triest, Matthew Sivaprakasam, Sean J. Wang, et al. TartanDrive: A Large-Scale Dataset for Learning Off-Road Dynamics Models. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 2546–2552, 2022.

[38] Philipp Schoch, Fan Yang, Yuntao Ma, et al. IN-Sight: Interactive Navigation through Sight. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 7794–7800, 2024.

[39] Haresh Karnan, Elvin Yang, Garrett Warnell, et al. Wait, That Feels Familiar: Learning to Extrapolate Human Preferences for Preference-Aligned Path Planning. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 13008–13014, 2024.

[40] Enze Xie, Wenhai Wang, Zhiding Yu, et al. SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 12077–12090, 2021.

[41] Pascal Roth, Julian Nubert, Fan Yang, et al. ViPlanner: Visual Semantic Imperative Learning for Local Navigation. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 5243–5249, 2024.

[42] Haitong Wang, Aaron Hao Tan, and Goldie Nejat. NavFormer: A Transformer Architecture for Robot Target-Driven Navigation in Unknown and Dynamic Environments. *IEEE Robotics and Automation Letters*, 9(8):6808–6815, 2024.

[43] Noriaki Hirose, Dhruv Shah, Ajay Sridhar, et al. SACSoN: Scalable Autonomous Control for Social Navigation. *IEEE Robotics and Automation Letters*, 9(1):49–56, 2024.

[44] Han Hu, Kaicheng Zhang, Aaron Hao Tan, et al. A Sim-to-Real Pipeline for Deep Reinforcement Learning for Autonomous Robot Navigation in Cluttered Rough Terrain. *IEEE Robotics and Automation Letters*, 6(4):6569–6576, 2021.

[45] Francisco Leiva and Javier Ruiz-del Solar. Robust RL-Based Map-Less Local Planning: Using 2D Point Clouds as Observations. *IEEE Robotics and Automation Letters*, 5(4):5787–5794, 2020.

[46] Francisco Leiva and Javier Ruiz-del Solar. Combining RL and IL Using a Dynamic, Performance-Based Modulation over Learning Signals and Its Application to Local Planning, 2024. arXiv preprint arXiv:2405.09760.

[47] Kyle Stachowicz, Dhruv Shah, Arjun Bhorkar, et al. FastRLAP: A System for Learning High-Speed Driving via Deep RL and Autonomous Practicing. In *Proceedings of the 7th Conference on Robot Learning (CoRL)*, pages 3100–3111. PMLR, 2023.

[48] Kyle Stachowicz, Lydia Ignatova, and Sergey Levine. Lifelong Autonomous Improvement of Navigation Foundation Models in the Wild. In *Proceedings of the 8th Conference on Robot Learning (CoRL)*, 2024.

[49] Dhruv Shah and Sergey Levine. ViKiNG: Vision-Based Kilometer-Scale Navigation with Geographic Hints. In *Proceedings of Robotics: Science and Systems (RSS)*, 2022.

[50] Dhruv Shah, Ajay Sridhar, Arjun Bhorkar, et al. GNM: A General Navigation Model to Drive Any Robot. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 7226–7233, 2023.

[51] Dhruv Shah, Ajay Sridhar, Nitish Dashora, et al. ViNT: A Foundation Model for Visual Navigation. In *Proceedings of the 7th Conference on Robot Learning (CoRL)*, pages 2620–2634. PMLR, 2023.

[52] Tianpei Gu, Guangyi Chen, Junlong Li, et al. Stochastic Trajectory Prediction via Motion Indeterminacy Diffusion. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 17092–17101, 2022.

[53] Chiyu Max Jiang, Andre Cornman, Cheol Eon Park, et al. MotionDiffuser: Controllable Multi-Agent Motion Prediction Using Diffusion. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9644–9653, 2023.

[54] Mingkun Wang, Xinge Zhu, Changqian Yu, et al. GANet: Goal Area Network for Motion Forecasting. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 1609–1615, 2023.

[55] Parth Kothari and Alexandre Alahi. Safety-Compliant Generative Adversarial Networks for Human Trajectory Forecasting. *IEEE Transactions on Intelligent Transportation Systems*, 24(4):4251–4261, 2023.

[56] Mihee Lee, Samuel S. Sohn, Seonghyeon Moon, et al. Multi-Scale VAE for Environment-Aware Long Term Trajectory Prediction. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8110–8119, 2022.

[57] Jing Liang, Amirreza Payandeh, Daeun Song, Xuesu Xiao, and Dinesh Manocha. Dtg : Diffusion-based trajectory generation for mapless global navigation. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5340–5347, 2024.

[58] Samuel Triest, Matthew Sivaprakasam, Shubhra Aich, et al. Velociraptor: Leveraging Visual Foundation Models for Label-Free, Risk-Aware Off-Road Navigation. In *Proceedings of the 8th Conference on Robot Learning (CoRL)*. PMLR, 2024.

[59] Mohammad Nazeri, Junzhe Wang, Amirreza payandeh, and Xuesu Xiao. VANP: Self-supervised vision-action pretraining for navigation. In *Bridging the Gap between Cognitive Science and Robot Learning in the Real World: Progresses and New Directions*, 2024.

[60] Matthew Chang, Theophile Gervet, Mukul Khanna, et al. GOAT: Go to Any Thing. In *Robotics: Science and Systems (RSS)*, 2024.

[61] Amirreza Payandeh, Anuj Pokhrel, Daeun Song, et al. Narrate2Nav: Real-Time Visual Navigation with Implicit Language Reasoning in Human-Centric Environments, 2025. arXiv preprint arXiv:2506.14233.

[62] Soroush Nasiriany, Fei Xia, Wenhao Yu, et al. PIVOT: Iterative Visual Prompting Elicits Actionable Knowledge for VLMs, 2024. arXiv preprint arXiv:2402.07872.

[63] Adarsh Jagan Sathyamoorthy, Kasun Weerakoon, Mohamed Elnoor, et al. CoNVOI: Context-Aware Navigation Using Vision Language Models in Outdoor and Indoor Environments. In *Proceedings of the 8th Conference on Robot Learning (CoRL)*. PMLR, 2024.

[64] Mohamed Elnoor, Kasun Weerakoon, Gershom Seneviratne, et al. VLM-GroNav: Robot Navigation Using Physically Grounded Vision-Language Models in Outdoor Environments. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 2391–2398, 2025.

[65] Daeun Song, Jing Liang, Xuesu Xiao, and Dinesh Manocha. VL-TGS: Trajectory Generation and Selection Using Vision Language Models in Mapless Outdoor Environments. *IEEE Robotics and Automation Letters*, 2025.

[66] Jing Liang, Kasun Weerakoon, Daeun Song, et al. MOSU: Autonomous Long-Range Robot Navigation with Multi-Modal Scene Understanding, 2025. arXiv preprint arXiv:2507.04686.

[67] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.

[68] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[69] M. I. Jordan. Serial order: a parallel distributed processing approach. technical report, june 1985-march 1986. Technical report, California Univ., San Diego, La Jolla (USA). Inst. for Cognitive Science, 1986.

[70] Alex Graves and Jürgen Schmidhuber. Framewise phoneme classification with bidirectional lstm and other neural network architectures. *Neural Networks*, 18(5):602–610, 2005. IJCNN 2005.

[71] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, et al. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734. Association for Computational Linguistics, 2014.

[72] Ronald J. Williams and David Zipser. A Learning Algorithm for Continually Running Fully Recurrent Neural Networks. *Neural Computation*, 1(2):270–280, 1989.

[73] Yuri Burda, Roger B. Grosse, and Ruslan Salakhutdinov. Importance weighted autoencoders. In Yoshua Bengio and Yann LeCun, editors, *Conference on Learning Representations (ICLR)*, 2016.

[74] James Lucas, George Tucker, Roger B Grosse, and Mohammad Norouzi. Don't blame the ELBO! a linear VAE perspective on posterior collapse. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

[75] Jing Liang, Dibyendu Das, Daeun Song, Md Nahid Hasan Shuvo, et al. Gnd: Global navigation dataset with multi-modal perception and multi-category traversability in outdoor campus environments. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 2383–2390, 2025.

[76] Husarion. *Panther Manual*. En línea, Accedido: 20-10-2025.

[77] Giorgio. Grisetti, Cyrill. Stachniss, and Wolfram. Burgard. Improving grid-based slam with rao-blackwellized particle filters by adaptive proposals and selective resampling. In *Proceedings of the IEEE International Conference on Robotics and Automation*, pages 2432–2437, 2005.

[78] S. Pellegrini, A. Ess, K. Schindler, and L. van Gool. You'll never walk alone: Modeling social behavior for multi-target tracking. In *International Conference on Computer Vision*, pages 261–268, 2009.

[79] Samuel R. Bowman, Luke Vilnis, Oriol Vinyals, Andrew M. Dai, et al. Generating sentences from a continuous space. In *Conference on Computational Natural Language Learning*, 2015.

[80] Rey Reza Wiyatno, Anqi Xu, and Liam Paull. Lifelong topological visual navigation. *IEEE Robotics and Automation Letters*, 7(4):9271–9278, 2022.